

UNIVERSIDAD SERGIO ARBOLEDA

APLICACIÓN WEB DE RESERVAS DE RESTAURANTES CON PATRONES CREACIONALES Y ESTRUCTURALES

Parcial CR-2: análisis y diseño orientado a objetos en tres capas

Patrones de Diseño de Software

Camilo Andrés Díaz García
Nicolás Pérez Chuscano
Santiago José Rodríguez Gutiérrez

Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá D.C., 2026

Índice

1. Introducción **8**

1.1. Objetivos 8

1.2. Metodología 8

1.3. Alcance y entregables 9

2. Análisis **9**

2.1. Descripción del sistema 9

2.2. Actores y roles 10

2.3. Casos de uso de alto nivel 10

2.4. Casos de uso extendidos y su especificación 11

2.5. Reglas de negocio 16

2.6. Requerimientos funcionales 17

2.7. Requerimientos no funcionales 22

3. Diseño **23**

3.1. Arquitectura en tres capas y MVC 23

3.2. Modelo relacional 25

3.3. Diagrama de clases de la capa de entidades 30

3.4. Diagrama de clases de la capa de datos (DAO) 32

3.5. Diagrama de clases de control (patrones) 33

3.6. Diagramas de estados 35

3.7. Diagramas de colaboración 37

3.8. Diagramas de secuencia 39

3.9. Raíz de composición 44

4. Patrones de diseño aplicados **44**

4.1. Patrones creacionales 45

4.1.1. Singleton: `ConexionBD` 45

4.1.2. Factory Method: `FabricaUsuario` 47

4.1.3. Builder: `ReservaBuilder` 49

4.2. Patrones estructurales 51

4.2.1. Facade: `ReservaFacade` 51
4.2.2. Proxy de protección: `ReservaServiceProxy` 53
4.2.3. Decorator: `Cobro` 56

5. Implementación de la presentación y la seguridad **59**
5.1. Controladores 59
5.2. Seguridad 60
5.3. Esquema de la reserva en SQL 61
5.4. Acceso a datos 62

6. Robustez y calidad **63**
6.1. Doble reserva bajo concurrencia 63
6.2. Límite de intentos de inicio de sesión 63
6.3. Consultas por lote en las listas de reservas 64
6.4. Cabeceras de seguridad y configuración 64

7. Pruebas automáticas **65**

8. Programa en ejecución **71**
8.1. Instalación y ejecución 71
8.2. Flujo del cliente 71
8.3. Flujo del administrador 83

9. Despliegue **89**

10. Limitaciones **90**

11. Conclusiones **90**

Referencias **92**

A. Créditos **93**

Índice de figuras

1. Diagrama de casos de uso de alto nivel (CU-01 a CU-15), por rol. Elaboración propia. 11

2. Caso de uso extendido CUE-01: iniciar sesión. Elaboración propia. 12

3. Caso de uso extendido CUE-02: reservar mesa. Elaboración propia. . . . 13

4. Caso de uso extendido CUE-03: gestionar las reservas del restaurante. Elaboración propia. 15

5. Arquitectura en tres capas y ubicación de los patrones. Elaboración propia a partir de la estructura del proyecto. 24

6. Modelo relacional, vista 1: usuarios, restaurantes, mesas, horarios y reservas. Generado a partir de `schema.sql`. 26

7. Modelo relacional, vista 2: carta y pre-pedido (`categoria`, `producto`, `reserva_item`). Generado a partir de `schema.sql`. 27

8. Diagrama de clases de la capa de entidades (modelo de dominio) con sus multiplicidades. Elaboración propia a partir de `modelos/__init__.py`. . 31

9. Diagrama de clases DAO (Data Access Object). Elaboración propia a partir de `datos/dao/__init__.py`. 32

10. Diagrama de clases de control con los seis patrones de diseño. Elaboración propia a partir del código. 34

11. Diagrama de estados de la clase `Reserva`. Elaboración propia a partir de `reserva_service.py`. 35

12. Diagrama de estados del acceso (intentos de inicio de sesión). Elaboración propia a partir de `limitador.py` y `auth_controller.py`. 36

13. Diagrama de estados del asistente de reserva (cinco pasos). Elaboración propia a partir de `reservar.js`. 37

14. Diagrama de colaboración: iniciar sesión. Elaboración propia a partir de `auth_controller.py`. 38

15. Diagrama de colaboración: reservar mesa. Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py` y `reserva_service.py`. 39

16. Diagrama de secuencia: iniciar sesión. Elaboración propia a partir de `auth_controller.py`. 40

17. Diagrama de secuencia: reservar mesa (1/2: del navegador al servicio). Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py`, `reserva_proxy.py`, `reserva_service.py` y `reserva_builder.py`. 41

18. Diagrama de secuencia: reservar mesa (2/2: validar y guardar). Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py`, `reserva_proxy.py`, `reserva_service.py` y `reserva_builder.py`. . 42

19. Diagrama de secuencia: cancelar una reserva (control de acceso del Proxy). Elaboración propia a partir de `reserva_proxy.py` y `reserva_service.py`. 43

20. Ensamblaje de DAO, servicios, Proxy y Facade en `dependencias.py` (fragmentos; los números de línea son los del archivo). 44

21. Singleton: clase `ConexionBD`. Elaboración propia a partir del código. . . . 46

22. Código de `ConexionBD` (`conexion.py`): doble verificación, `conectar` y `sesion`. 46

23. Prueba de concurrencia del Singleton (`test_patrones.py`). 47

24. Factory Method: creadores y productos de usuario. Elaboración propia a partir del código. 48

25. Código del Factory Method (`usuario_factory.py`). 49

26. Builder: constructor, director y producto. Elaboración propia a partir del código. 50

27. Código del Builder (1/2): estado acumulado y pasos de la interfaz fluida (`reserva_builder.py`). 51

28. Código del Builder (2/2): inicio de `build()` y la regla de no solapamiento. 51

29. Facade: `ReservaFacade` y los servicios que orquesta. Elaboración propia a partir del código. 52

30. Código del Facade (`reserva_facade.py`): constructor, `reservar` y `reservar_con_pre_pedido`. 53

31. Proxy de protección: interfaz común, servicio real y proxy. Elaboración propia a partir del código. 54

32. Código del Proxy (`reserva_proxy.py`): registro, `crear_reserva` y `cancelar_reserva`. 55

33. Servicio real (*RealSubject*, `reserva_service.py`): usa el Builder, congela el cobro y guarda con `insertar_si_libre`. 55

34. Prueba del Proxy sobre una reserva ajena (`test_patrones.py`). 56

35. Decorator: cadena de cobro. Elaboración propia a partir del código. . . . 57

36. Código del Decorator (1/2): regla de redondeo, interfaz `Cobro` y clase base `CobroDecorador` (`cobro_decorator.py`). 58

37. Código del Decorator (2/2): un decorador concreto y la construcción de la cadena. 58

38. Instantánea del cobro que se guarda con la reserva (cobro_decorator.py). 59
39. Prueba del desglose de ejemplo (test_fase4.py). 59
40. Controlador de POST /reservar (cliente_controller.py). 60
41. Controlador de inicio de sesión (auth_controller.py). 60
42. Hash y verificación de contraseñas con scrypt (seguridad.py). 60
43. Token CSRF por sesión (comun.py). 61
44. Tablas reserva y reserva_item (schema.sql). 61
45. BaseDAO: CRUD genérico (__init__.py). 62
46. ReservaDAO: columnas, inserción atómica con comprobación de solapamiento (insertar_si_libre) y cancelación. 62
47. Transacción exclusiva de ConexionBD (conexion.py). 63
48. Prueba de concurrencia: ocho conexiones compiten por la misma mesa (test_fase5.py). 63
49. Límite de intentos fallidos (limitador.py). 64
50. Ejecución completa de la batería de pruebas: 154 pruebas superadas. . . . 65
51. Salida detallada de test_patrones.py (Singleton, Factory Method, Builder, Facade y Proxy). 66
52. Salida detallada de test_fase4.py (Decorator y total a pagar). 67
53. Salida detallada de test_reglas_negocio.py (reglas de reserva). . . . 68
54. Salida detallada de test_fase5.py (concurrencia, seguridad, comprobante, CSV y consultas por lote). 69
55. Salida detallada de test_cobro_guardado.py (instantánea del cobro). . 70
56. Pantalla de inicio de sesión (login). 72
57. Inicio de sesión con credenciales incorrectas: mensaje genérico, sin revelar si falló el correo o la contraseña. 72
58. Registro de un cliente (registro). 73
59. Página de inicio. 73
60. Listado de restaurantes con atajos por tipo de cocina y filtros (restaurantes). 74
61. Carta de un restaurante, por categorías y con buscador (carta); se muestra el inicio de la página. 75
62. Carta con el cajón del pre-pedido abierto. 76
63. Formulario de reserva, paso 1: aviso del restaurante elegido, fecha y número de personas. 76

64. Paso 2 sin hora elegida: el formulario bloquea el avance y lo indica. . . . 77
65. Paso 3: plano de mesas por zonas (libres, ocupadas, seleccionada y no disponible por capacidad). 78
66. Paso 4: pre-pedido opcional, con el subtotal. 79
67. Paso 5: desglose del total a pagar calculado con el Decorator (sin propina). 79
68. Paso 5 con la propina sugerida incluida (el total cambia en el servidor). . 80
69. Mis reservas: tarjetas con código, estado, pre-pedido y total, con filtros. . 81
70. Comprobante imprimible de una reserva (con descarga del archivo `.ics`). 82
71. Intento de reservar una mesa ya ocupada en ese horario: la regla RN-3 rechaza la reserva y conserva los datos escritos. 83
72. Un cliente que intenta entrar a `admin` recibe 403 (control de acceso por rol). 84
73. Panel del administrador: indicadores, dona de estados, mapa de calor de ocupación, reservas de los últimos 14 días y productos más pedidos. . . . 85
74. Gestión de mesas (`mesas`). 85
75. Gestión del horario semanal (`horarios`). 86
76. Gestión de la carta: categorías y productos (`carta`); se muestra el inicio de la página. 86
77. Reservas del restaurante, con búsqueda por cliente, filtros por fecha y estado, exportación a CSV y paginación. 87
78. Confirmación antes de cancelar una reserva (cuadro de diálogo propio). . 88
79. Reserva confirmada por el administrador (aviso verde y estado actualizado). 89

Índice de tablas

1. Entregables del parcial y su ubicación en el documento. Elaboración propia a partir del enunciado. 9

2. Roles, casos de uso y rutas. Elaboración propia a partir de los controladores del proyecto. 10

3. Especificación del caso de uso extendido ECU-01: iniciar sesión. Elaboración propia a partir de `auth_controller.py` y `auth_service.py`. . . . 13

4. Especificación del caso de uso extendido ECU-02: reservar mesa. Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py` y `reserva_service.py`. 14

5. Formato resumido del caso de uso “Reservar mesa” (acción del actor y respuesta del sistema). Elaboración propia. 15

6. Especificación del caso de uso extendido ECU-03: gestionar las reservas del restaurante. Elaboración propia a partir de `admin_controller.py` y `reserva_proxy.py`. 16

7. Requerimientos funcionales y su trazabilidad hacia la implementación y las pruebas (CU: caso de uso de la Figura 1). Elaboración propia a partir del código del proyecto. 21

8. Correspondencia entre capas, carpetas y MVC. Elaboración propia. 25

9. Diccionario de datos del esquema relacional (PK: clave primaria; FK: clave foránea; NN: no nulo). Elaboración propia a partir de `schema.sql`. 30

10. Patrones de diseño aplicados (rutas relativas a `app/negocio/`, salvo `conexion.py`, relativa a `app/`). Elaboración propia. 44

11. Trazabilidad entre patrones, requerimientos y pruebas. Elaboración propia a partir del código y de las pruebas. 45

12. Pruebas automáticas por archivo (cifras calculadas con `pytest --collect-only`). Elaboración propia. 65

13. Usuarios de demostración creados por el seed (datos ficticios). Fuente: `README.md` del proyecto. 71

14. Créditos de las fotografías de las cartas. Fuente: `CREDITOS.json` del proyecto. 95

1 Introducción

El parcial CR-2 plantea una aplicación de reservas para restaurantes en la que un cliente reserva mesa y un administrador gestiona mesas, horarios y reservas, con un diseño orientado a objetos en tres capas (modelo–vista–controlador) y el uso de patrones de diseño: al menos dos creacionales y dos estructurales. Este documento recoge el análisis y el diseño de la solución desarrollada, **MIDNIGHT DINER**, así como la evidencia de su implementación y de su funcionamiento.

La aplicación está escrita en Python con FastAPI y plantillas Jinja2, y guarda sus datos en SQLite mediante el módulo `sqlite3` de la biblioteca estándar, sin ORM, de modo que la capa de datos se construye a mano con el patrón DAO. Sobre las reglas de reserva se aplican seis patrones: tres creacionales (Singleton, Factory Method y Builder) y tres estructurales (Facade, Proxy de protección y Decorator), uno más de cada tipo que el mínimo exigido.

1.1 Objetivos

Objetivo general. Diseñar e implementar una aplicación web de reservas de restaurantes, organizada en tres capas y apoyada en patrones de diseño creacionales y estructurales.

Objetivos específicos.

- Levantar los requerimientos funcionales y no funcionales y las reglas de negocio de una reserva.
- Modelar el sistema con los diagramas del curso: casos de uso de alto nivel y extendidos, clases (entidades, DAO y control), estados, colaboración y secuencia.
- Diseñar el modelo relacional y la capa de acceso a datos (DAO).
- Aplicar y justificar los patrones Singleton, Factory Method, Builder, Facade, Proxy y Decorator.
- Demostrar el funcionamiento con 154 pruebas automáticas y con la aplicación en ejecución.

1.2 Metodología

El trabajo siguió un proceso iterativo en el espíritu de RUP, visto en clase: primero se identificaron los actores y los casos de uso (alto nivel y extendidos), luego se modeló el dominio y la capa de datos, después se implementó cada capa con sus pruebas y, al final, se refinó la interfaz y la robustez. Los diagramas de este documento siguen las convenciones de las entregas anteriores del curso: UML en blanco y negro, con la terminología del docente (“casos de uso de alto nivel”, “caso de uso extendido” con `«include»` y `«extend»`, “clases DAO”, “diagrama de estados”, “diagrama de colaboración” y “diagrama de secuencia por proceso”), escritos como texto en PlantUML para que puedan regenerarse y versionarse.

1.3 Alcance y entregables

El alcance obligatorio del enunciado es el inicio de sesión y el formulario de reserva para cliente y administrador, junto con la gestión del administrador sobre mesas, horarios y reservas y la búsqueda de clientes. Se añadió, como ampliación, la carta de cada restaurante con pre-pedido y el cálculo del total a pagar, que es el caso de uso del patrón Decorator, además de varias mejoras de robustez descritas en la Sección 6. La Tabla 1 relaciona cada entregable con el lugar de este documento donde se atiende.

Entregable del enunciado	**Dónde se atiende**
Diseño orientado a objetos en 3 capas / MVC	Sección 3.1
Modelo relacional	Sección 3.2
Diagrama de clases de los DAO	Sección 3.4
Diagrama de clases de control (patrones)	Secciones 3.5 y 4
Documento con análisis y diseño (APA)	Este documento
Capturas del código fuente	Secciones 4, 5 y 6 (renderizadas desde los archivos reales)
Capturas del programa en ejecución	Sección 8
Código fuente	Repositorio de GitHub: `https://github.com/CamiloDG07/Parcial-2-Patrones-de-Dise-o`
Video explicativo	Enlace de Google Drive indicado en la entrega de la plataforma

Tabla 1. Entregables del parcial y su ubicación en el documento. Elaboración propia a partir del enunciado.

2 Análisis

2.1 Descripción del sistema

MIDNIGHT DINER es una marca ficticia creada para el parcial. El sistema administra 5 restaurantes de demostración; cada uno tiene mesas, un horario semanal y su propia carta (hay 5 cartas publicadas entre los 5 restaurantes). Hay dos roles: el **cliente**, que se registra, consulta restaurantes y cartas, reserva una mesa (con pre-pedido opcional) y gestiona sus propias reservas; y el **administrador**, que gestiona un único restaurante: sus mesas, horarios, carta y reservas (confirmar, cancelar, filtrar, buscar por cliente y exportar). Además de lo exigido, el sistema ofrece un plano de mesas por zonas en el paso de selección, tarjetas de reserva con filtros, comprobante imprimible y archivo de calendario (`.ics`), y un panel con gráficos (dona de estados y mapa de calor de ocupación).

2.2 Actores y roles

La Tabla 2 resume qué puede hacer cada rol y con qué casos de uso y rutas se corresponde. El control se aplica en dos niveles: por rol en los controladores y por propiedad del recurso en el Proxy (Sección 4).

Rol	Casos de uso	Rutas principales	Restricción
Visitante	CU-01, CU-02, CU-03, CU-04	`/registro`, `/login`, `/restaurantes`, `/carta`	Sin sesión no ve reservas ni panel.
Cliente	CU-05 a CU-08, CU-14	`/reservar`, `/api/carrito/*`, `/api/cobro`, `/mis-reservas*`	Solo ve y cancela sus reservas; no entra a `/admin`.
Administrador	CU-09 a CU-13, CU-15	`/admin`, `/admin/mesas`, `/admin/horarios`, `/admin/reservas*`, `/admin/carta`	Solo su restaurante; no reserva.

Tabla 2. Roles, casos de uso y rutas. Elaboración propia a partir de los controladores del proyecto.

Cómo se obtiene el rol de administrador. El registro público (`/registro`) *siempre* crea un cliente: el rol no viaja en el formulario, lo fija el servidor. Es una decisión de seguridad deliberada, porque si cualquier visitante pudiera registrarse como administrador podría adueñarse del panel de un restaurante, ver sus reservas y modificar su carta. Por eso los administradores solo existen de dos formas: los cinco que crea el *seed* (uno por restaurante, `admin@demo.com` a `admin5@demo.com`) o los que se agregan después con un *script* de alta (Sección 8). La relación es *un administrador, un restaurante* (`restaurante.admin_id` no admite nulos): cada administrador solo ve y modifica lo suyo, y si altera un identificador en la dirección para mirar otro restaurante, el Proxy lo rechaza y deja la línea `DENEGADO` en el registro. El administrador no usa el formulario de reserva (`/reservar` responde 403): su función es recibir, confirmar o cancelar las reservas de los clientes y configurar mesas, horarios y carta.

2.3 Casos de uso de alto nivel

La Figura 1 presenta el modelo de casos de uso de alto nivel por rol: quince casos de uso agrupados por actor. CU-02 es compartido por cliente y administrador.

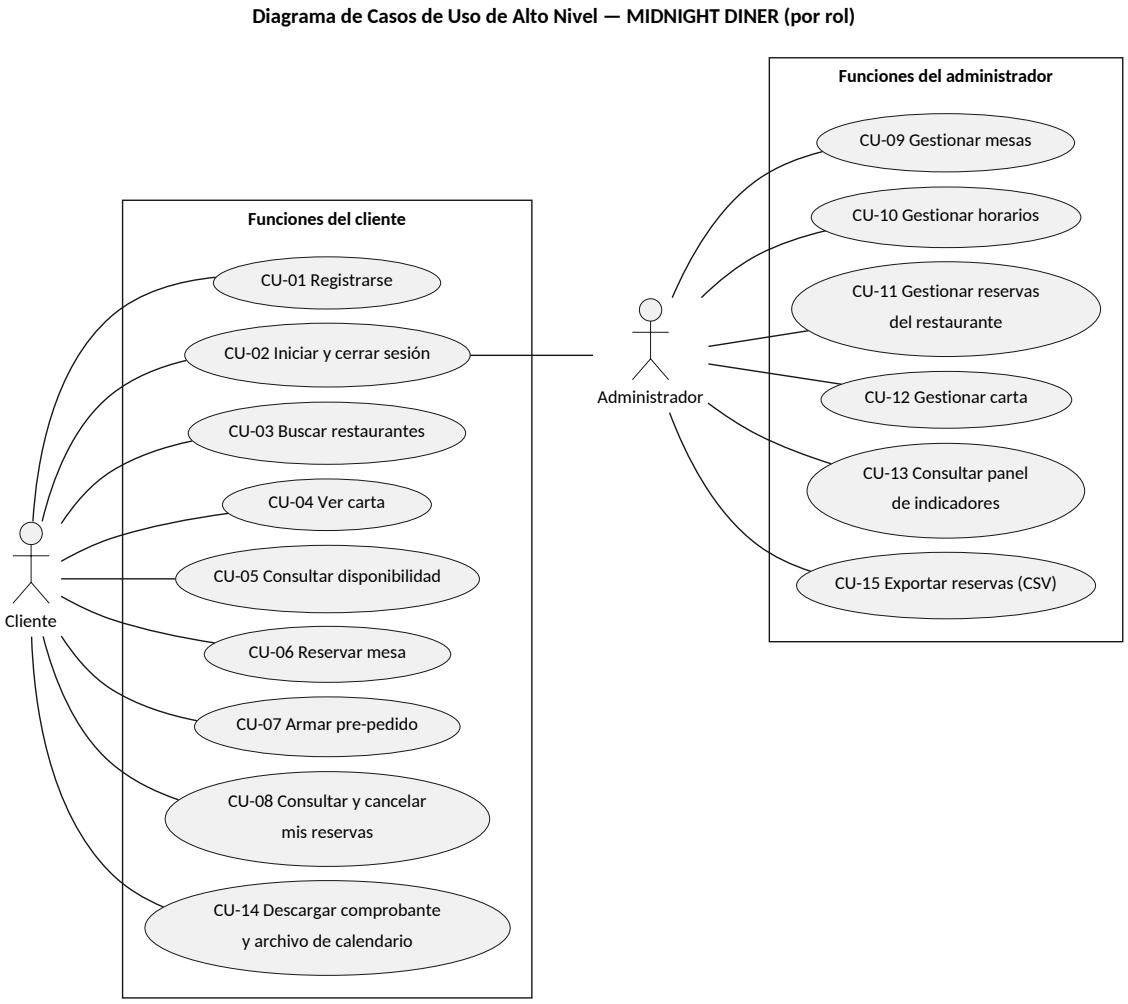


Figura 1. Diagrama de casos de uso de alto nivel (CU-01 a CU-15), por rol. Elaboración propia.

2.4 Casos de uso extendidos y su especificación

Se desarrollan tres procesos con el diagrama de caso de uso extendido (relaciones «include» y «extend») y su especificación completa: iniciar sesión (CUE-01), reservar mesa (CUE-02) y gestionar las reservas del restaurante (CUE-03).

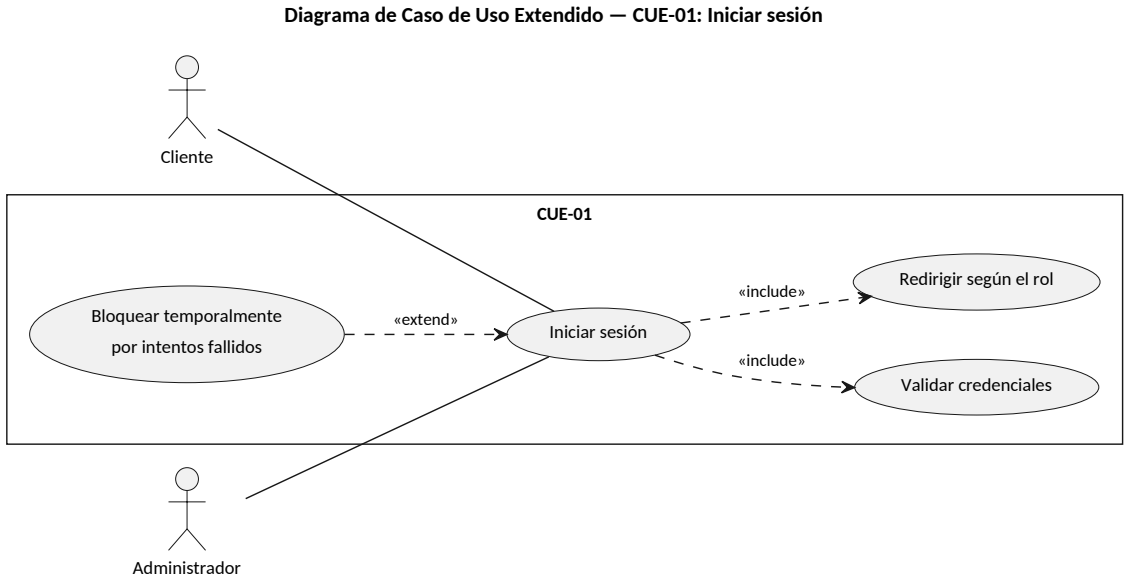


Figura 2. Caso de uso extendido CUE-01: iniciar sesión. Elaboración propia.

Campo	Contenido
ID	ECU-01
Nombre	Iniciar sesión
Actor(es)	Cliente y Administrador (principales).
Descripción	Autentica a un usuario registrado con su correo y contraseña y lo lleva a la pantalla que corresponde a su rol, limitando los intentos fallidos.
Precondiciones	El usuario existe (se registró o viene del seed) y su combinación IP + correo no está bloqueada.
Postcondiciones	Hay una sesión firmada con el identificador del usuario; la sesión anterior se descartó; el cliente queda en `/restaurantes` y el administrador en `/admin`.
Flujo principal	1. El usuario abre `/login` y recibe un token CSRF. 2. Escribe correo y contraseña y envía el formulario. 3. El controlador verifica el token CSRF y pregunta al limitador si la IP y el correo están bloqueados («include» Validar credenciales). 4. `AuthService.autenticar` busca al usuario por correo (`UsuarioDAO` crea un `Cliente` o un `Administrador` con `UsuarioFactory`) y compara la contraseña (scrypt, tiempo constante). 5. Se borran los fallos de esa clave, se limpia la sesión y se guarda el usuario. 6. El sistema redirige (303) a la ruta de inicio del rol («include» Redirigir según el rol).

continúa en la página siguiente

Campo	Contenido
Flujos alternativos	**A1 (credenciales incorrectas):** en el paso 4 se registra el fallo y se responde 401 con "Correo o contraseña incorrectos", sin revelar cuál de los dos falló. **A2 (bloqueo temporal, `«extend»`):** en el paso 3, si hay 5 fallos con la misma IP y correo, o 20 de la misma IP, en 5 minutos, se responde 429 con la cabecera `Retry-After` durante 5 minutos. **A3 (CSRF inválido):** se responde 403 sin procesar el formulario.
Reglas de negocio	Límite de intentos (5 por IP + correo, 20 por IP, ventana y bloqueo de 5 minutos); el rol decide el destino (Factory Method).
Frecuencia de uso	Alta: una vez por sesión de cada usuario.

Tabla 3. Especificación del caso de uso extendido ECU-01: iniciar sesión. Elaboración propia a partir de `auth_controller.py` y `auth_service.py`.

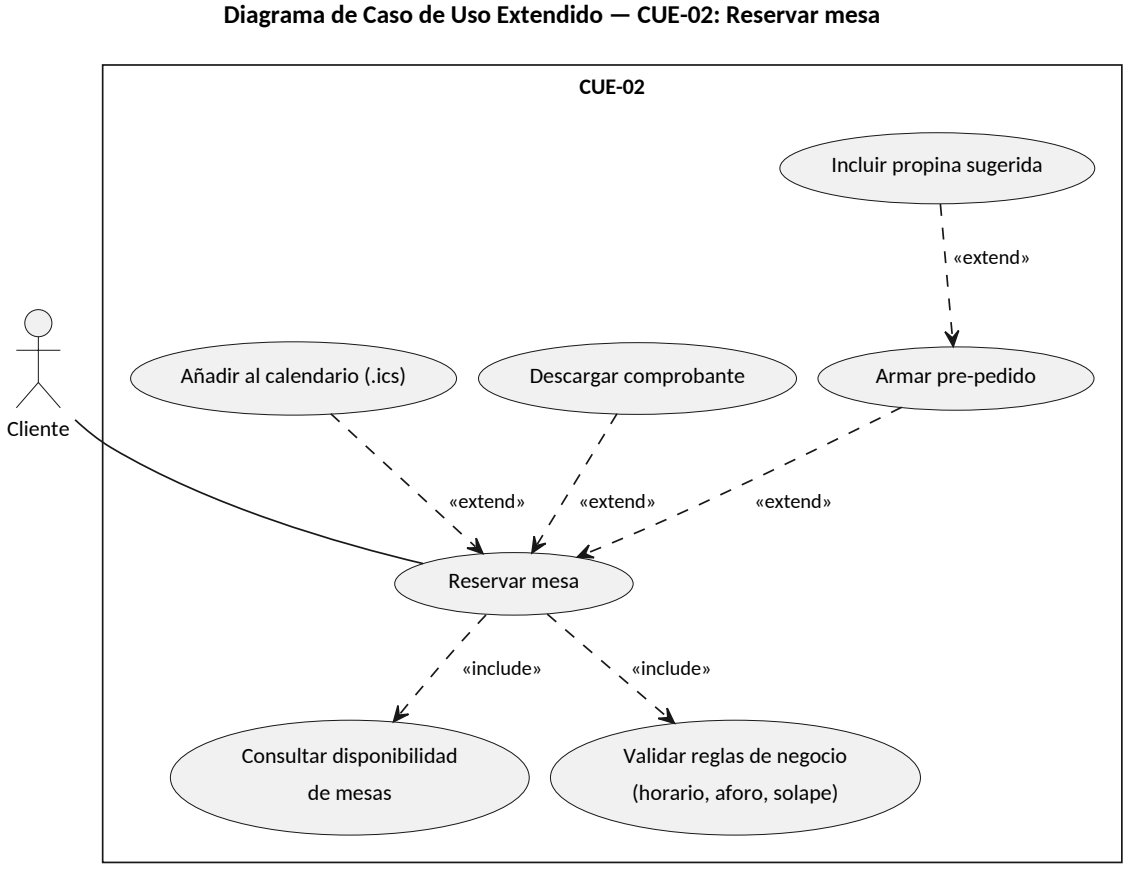


Figura 3. Caso de uso extendido CUE-02: reservar mesa. Elaboración propia.

Campo	Contenido
ID	ECU-02
Nombre	Reservar mesa

continúa en la página siguiente

Campo	Contenido
Actor(es)	Cliente (principal).
Descripción	Permite reservar una mesa de un restaurante para una fecha, una hora y un número de personas, con un pre-pedido opcional y el total a pagar calculado en el servidor.
Precondiciones	El cliente tiene sesión; el restaurante tiene horario ese día y mesas activas.
Postcondiciones	Existe una reserva `PENDIENTE` con su mesa; si hubo pre-pedido, se guardan sus ítems con el precio congelado y la instantánea del cobro (subtotal, descuento, impuesto y total); el carrito queda vacío; la mesa queda ocupada 120 minutos desde la hora elegida.
Flujo principal	1. El cliente elige el restaurante; la página muestra "Estás reservando en . . . ". 2. Paso 1: fecha y número de personas. 3. Paso 2: el sistema consulta la disponibilidad y ofrece las franjas libres cada 30 minutos («`include`» Consultar disponibilidad). 4. Paso 3: el cliente elige una mesa en el plano o deja que el sistema asigne la más ajustada. 5. Paso 4: arma un pre-pedido con platos de la carta de ese restaurante (opcional, «`extend`» Armar pre-pedido). 6. Paso 5: revisa el desglose (descuento, impuesto al consumo y propina sugerida opcional) y confirma. 7. El servidor valida las reglas con el Builder («`include`» Validar reglas de negocio), guarda la reserva en una transacción exclusiva y redirige a `/mis-reservas` con un aviso.
Flujos alternativos	**A1 (sin pre-pedido):** el paso 4 se omite y la reserva queda sin total a pagar. **A2 (regla incumplida):** en el paso 7 la hora está fuera del horario, la mesa no cabe o está ocupada: se vuelve al formulario con el mensaje y los datos ya escritos. **A3 (propina):** si el cliente la incluye en el paso 5, el total mostrado cambia; la propina no se guarda («`extend`» Incluir propina sugerida). **A4 (carrera):** si dos clientes confirman la misma mesa a la vez, gana quien toma primero el bloqueo de escritura; el otro recibe "La mesa ya está reservada en ese horario". **A5 (después de reservar):** el cliente puede descargar el comprobante y el archivo `.ics` («`extend`»).
Reglas de negocio	RN-1 a RN-3 y RN-6 (Sección 2.5); tasas del cobro configurables en `app/config.py`.
Frecuencia de uso	Media a alta: una vez por reserva, con picos en las horas previas a la cena.

Tabla 4. Especificación del caso de uso extendido ECU-02: reservar mesa. Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py` y `reserva_service.py`.

Para este mismo caso de uso se incluye además el formato resumido de dos columnas (acción del actor y respuesta del sistema), que suele usarse como paso previo a los diagramas de interacción.

Paso	Acción del actor (cliente)	Respuesta del sistema
1	Elige un restaurante.	Muestra el aviso “Estás reservando en …” y el asistente de cinco pasos.
2	Indica fecha y número de personas.	Consulta las franjas con mesas libres y las ofrece cada 30 minutos.
3	Elige una franja.	Dibuja el plano de mesas: libres, ocupadas y no disponibles por capacidad.
4	Elige una mesa (o ninguna).	Guarda la elección; si no hay mesa, asignará la más ajustada.
5	Agrega platos al pre-pedido (opcional).	Mantiene el carrito en la sesión y muestra el subtotal.
6	Revisa el desglose y confirma.	Calcula descuento, impuesto y propina con la cadena Decorator.
7	—	Valida con el Builder, guarda con `BEGIN IMMEDIATE`, vacía el carrito y redirige a `/mis-reservas`.

Tabla 5. Formato resumido del caso de uso “Reservar mesa” (acción del actor y respuesta del sistema). Elaboración propia.

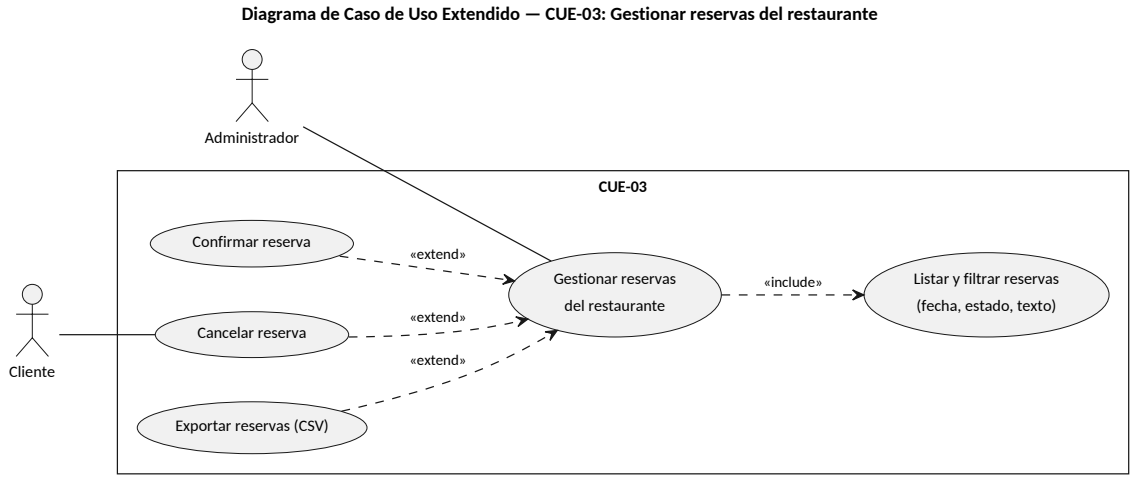


Figura 4. Caso de uso extendido CUE-03: gestionar las reservas del restaurante. Elaboración propia.

Campo	Contenido
ID	ECU-03
Nombre	Gestionar las reservas del restaurante
Actor(es)	Administrador (principal); Cliente (secundario: cancela solo las suyas desde `/mis-reservas`).

continúa en la página siguiente

Campo	Contenido
Descripción	Permite al administrador listar y filtrar las reservas de su restaurante, confirmarlas o cancelarlas y exportarlas.
Precondiciones	El administrador tiene sesión y un restaurante asignado.
Postcondiciones	El estado de la reserva cambia conforme a las transiciones permitidas; al cancelar se elimina su pre-pedido y la mesa queda libre; cada operación queda registrada como `PERMITIDO` o `DENEGADO`.
Flujo principal	1. Abre `/admin/reservas`. 2. Filtra por fecha, estado o texto (cliente, correo, nota o número) y pagina («`include`» Listar y filtrar). 3. Elige una reserva `PENDIENTE` y pulsa Confirmar («`extend`»). 4. El Proxy comprueba el rol y que la reserva sea de su restaurante. 5. El servicio valida la transición y actualiza el estado. 6. El sistema muestra el aviso y la lista actualizada.
Flujos alternativos	**A1 (cancelar):** en el paso 3 elige Cancelar; un cuadro de confirmación pide la decisión antes de enviar. **A2 (exportar):** descarga un CSV solo con las reservas de su restaurante. **A3 (reserva ajena):** el Proxy lanza `NoAutorizadoError`; no se toca el servicio real. **A4 (transición inválida):** por ejemplo reabrir una cancelada: "No se puede pasar de CANCELADA a CONFIRMADA".
Reglas de negocio	RN-4 y RN-5 (Sección 2.5).
Frecuencia de uso	Diaria, varias veces por turno.

Tabla 6. Especificación del caso de uso extendido ECU-03: gestionar las reservas del restaurante. Elaboración propia a partir de `admin_controller.py` y `reserva_proxy.py`.

2.5 Reglas de negocio

Las reglas se aplican en el servidor, dentro de la capa de negocio, y no dependen de lo que valide el navegador:

RN-1. La fecha y la hora deben ser futuras y caer dentro del horario del restaurante ese día. Como cada reserva dura 120 minutos, la última hora de inicio válida es la hora de cierre menos 120 minutos.

RN-2. La mesa debe pertenecer al restaurante, estar activa y tener capacidad mayor o igual al número de personas.

RN-3. No puede haber solapamiento en una mesa: dos reservas se cruzan si sus intervalos de 120 minutos se intersectan (las canceladas no cuentan, y dos reservas contiguas no se solapan).

RN-4. El cliente solo ve y cancela sus propias reservas; el administrador solo gestiona las de su restaurante.

RN-5. Los estados son `PENDIENTE`, `CONFIRMADA` y `CANCELADA`. Toda reserva nace pendiente; el administrador la confirma o la cancela, y una confirmada aún puede cancelarse. Una cancelada no se reabre.

RN-6. El precio de un plato del pre-pedido se lee de la base de datos y queda congelado en la reserva; el navegador nunca envía precios ni montos. El total a pagar (subtotal, descuento, impuesto y total) también se guarda al reservar, de modo que un cambio posterior de tasas no altera reservas ya hechas.

2.6 Requerimientos funcionales

La Tabla 7 lista los requerimientos funcionales con su rastro hacia la implementación (ruta HTTP y clases) y hacia las pruebas automáticas que los ejercitan.

Cód.	Requerimiento	Rol	Implementación	Pruebas
RF-01	Registrar un cliente con nombre, correo y contraseña confirmada (CU-01). El rol siempre es cliente.	Visitante	`GET/POST /registro`; `AuthService.registrar`; `FabricaCliente`	`test_registro_crea_cliente_y_hace_login`; `test_registro_valida`; `test_registro_web_ignora_rol_enviado`
RF-02	Iniciar sesión con correo y contraseña; el rol decide la pantalla de destino; se bloquean los intentos repetidos (CU-02).	Cliente, administrador	`GET/POST /login`; `AuthService.autenticar`; `LimitadorIntentos`; `Usuario.ruta_inicio`	`test_login_redirige_segun_rol`; `test_login_incorrecto`; `test_login_sin_csrf_se_rechaza`; `test_login_se_bloquea_tras_cinco_fallos`; `test_un_login_correcto_reinicia_el_contador`
RF-03	Cerrar sesión (CU-02).	Cliente, administrador	`POST /logout`	Sin prueba automática específica; se recorre en la demostración
RF-04	Consultar restaurantes y filtrarlos por nombre, tipo de cocina y ciudad, con atajos por tipo de cocina (CU-03).	Público	`GET /restaurantes`; `ReservaFacade.listar_restaurantes`	`test_paginas_publicas`; `test_chips_de_cocina_en_directorio`
RF-05	Consultar la carta propia de un restaurante, por categorías, con búsqueda (CU-04).	Público	`GET /carta`; `ReservaFacade.carta_de`; `cartas_demo/<id>.json`	`test_seed_carta_demo`; `test_cada_plato_tiene_foto_y_credito`; `test_el_seed_carga_la_carta_de_cada_restaurante_y_no_las_mezcla`; `test_ninguna_foto_se_repite_entre_restaurantes`
RF-06	Consultar las franjas y mesas libres para una fecha y un número de personas (CU-05).	Cliente	`GET /api/disponibilidad`; `ReservaFacade.buscar_disponibilidad`	`test_api_disponibilidad`; `test_facade_disponibilidad_y_reserva_con_mesa_automatica`

continúa en la página siguiente

Cód.	Requerimiento	Rol	Implementación	Pruebas
RF-07	Reservar una mesa en cinco pasos (fecha y personas, hora, mesa, pre-pedido y confirmación), con asignación automática de mesa si no se elige una (CU-06).	Cliente	`GET/POST /reservar`; `ReservaFacade.reservar_con_pre_pedido`; `ReservaBuilder`; `ReservaDAO.insertar_si_libre`	`test_flujo_completo_reserva_confirmacion_y_doble_reserva`; `test_reserva_con_mesa_automatica`; `test_errores_de_validacion_en_servidor`; `test_aviso_del_restaurante_elegido_al_reservar`
RF-08	Armar un pre-pedido (agregar, cambiar y vaciar platos) que se confirma junto con la reserva (CU-07).	Cliente	`/api/carrito/*`; `CarritoService`; `ReservaBuilder.con_items`	`test_carrito_total_correcto`; `test_carrito_rechaza_otro_restaurante_y_no_disponible`; `test_reserva_con_items_guarda_total_y_precio_congelado`
RF-09	Mostrar el total a pagar del pre-pedido con descuento, impuesto al consumo y propina sugerida opcional, y guardarlo al reservar (CU-07).	Cliente	`GET /api/cobro`; `armar_cadena` (Decorator); `instantanea`; `desglose_guardado`	`test_composicion_apilada_con_montos_exactos`; `test_desglose_suma_exactamente_el_total`; `test_el_cliente_no_envia_montos_y_la_base_no_cambia`; `test_la_reserva_guarda_la_instantanea_del_cobro`; `test_cambiar_las_tasas_no_altera_reservas_anteriores`
RF-10	Consultar las propias reservas (próximas, pasadas y canceladas, con código `MD-nnnn`), su estado y su pre-pedido (CU-08).	Cliente	`GET /mis-reservas`; `ReservaFacade.mis_reservas`; `clasificar_reservas`	`test_facade_mis_reservas_y_cancelar`; `test_clasificar_reservas_orden_y_grupos`; `test_mis_reservas_filtros_y_codigo`

continúa en la página siguiente

Cód.	Requerimiento	Rol	Implementación	Pruebas
RF-11	Cancelar una reserva propia; sus platos se eliminan y la mesa queda libre (CU-08).	Cliente	`POST /mis-reservas/{id}/cancelar`; `ReservaServiceProxy.cancelar_reserva`	`test_proxy_rechaza_cancelar_reserva_ajena`; `test_cancelar_borra_los_items`; `test_cancelada_libera_la_mesa`; `test_cancelar_libera_la_mesa_para_insertar_si_libre`
RF-12	Ver un panel con reservas del día, próximas, ocupación e ingresos estimados, una dona de estados y un mapa de calor de ocupación (CU-13).	Administrador	`GET /admin`; `EstadisticasService`	`test_estadisticas_kpis`; `test_ocupacion_cero_si_cerrado`; `test_estadisticas_por_estado_y_mapa_de_calor`; `test_panel_muestra_dona_y_mapa_de_calor`
RF-13	Crear, editar y eliminar mesas (número, capacidad, ubicación, activa) (CU-09).	Administrador	`/admin/mesas*`; `MesaService`; `MesaDAO`	`test_admin_crud_mesas_y_horarios`
RF-14	Definir y eliminar el horario de cada día de la semana (CU-10).	Administrador	`/admin/horarios*`; `HorarioService`; `HorarioDAO`	`test_admin_crud_mesas_y_horarios`; `test_rechaza_dia_cerrado`
RF-15	Listar las reservas del restaurante con filtros por fecha y estado, búsqueda por cliente, correo, nota o número, y paginación; confirmarlas o cancelarlas (CU-11).	Administrador	`/admin/reservas*`; `ReservaServiceProxy.confirmar_reserva`	`test_admin_reservas_busqueda_y_paginacion`; `test_confirmar_cambia_estado`; `test_admin_de_otro_restaurante_no_gestiona_ajenos`
RF-16	Gestionar categorías y productos de la carta, con imagen, y marcar un producto como agotado (CU-12).	Administrador	`/admin/carta/*`; `CartaService`; `guardar_imagen`	`test_admin_crud_carta_con_imagen`; `test_guardar_imagen_valida_tipo_tamano_y_nombre`; `test_admin_ajeno_no_toca_la_carta`

continúa en la página siguiente

Cód.	Requerimiento	Rol	Implementación	Pruebas
RF-17	Descargar el comprobante imprimible y el archivo de calendario (`.ics`) de una reserva propia (CU-14).	Cliente	`GET /mis-reservas/{id}/comprobante`; `GET /mis-reservas/{id}/calendario.ics`; `evento_ics`	`test_comprobante_y_calendario_solo_para_el_dueno`; `test_evento_ics_valido_y_escapado`; `test_doblar_lineas_largas_sin_cortar_caracteres`; `test_ics_de_reserva_cancelada`
RF-18	Exportar las reservas del restaurante a CSV (CU-15).	Administrador	`GET /admin/reservas.csv`	`test_exportar_csv`; `test_csv_solo_del_restaurante_del_admin_y_solo_admin`

Tabla 7. Requerimientos funcionales y su trazabilidad hacia la implementación y las pruebas (CU: caso de uso de la Figura 1). Elaboración propia a partir del código del proyecto.

2.7 Requerimientos no funcionales

Los requerimientos no funcionales se verifican en el código y en las pruebas; los que no están cubiertos se declaran en la Sección 10.

- **Seguridad de credenciales.** Las contraseñas se almacenan con scrypt (`hashlib.scrypt`, con sal aleatoria de 16 bytes) y se comparan en tiempo constante con `hmac.compare_digest`.
- **Fuerza bruta.** El inicio de sesión se bloquea 5 minutos tras 5 fallos con la misma IP y correo, o 20 de la misma IP, y responde 429 con `Retry-After` (`test_login_se_bloquea_tras_cinco_fallos`).
- **Protección de formularios.** Todo `POST` exige un token CSRF por sesión (campo `csrf_token` o cabecera `X-CSRF-Token`); sin él la respuesta es 403.
- **Sesiones.** Cookie firmada (`SessionMiddleware`) con `SameSite=Lax`, vigencia de 8 horas, y la sesión se limpia al iniciar sesión para evitar la fijación de sesión. Con la variable `HTTPS_ONLY=1` la cookie pasa a `Secure` y se envía HSTS.
- **Cabeceras de seguridad.** Política de contenido (CSP) restrictiva sin JavaScript en línea, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` y `Permissions-Policy` en todas las respuestas (`test_cabeceras_de_seguridad`).
- **Autorización.** Control de acceso por rol en los controladores (`requiere_cliente`, `requiere_admin`) y por propiedad del recurso en el Proxy.
- **Inyección SQL.** Todas las consultas usan parámetros `?`; los nombres de tabla y columna son constantes de clase, nunca datos del usuario.
- **Subida de imágenes.** Solo JPG, PNG o WebP detectados por su cabecera (no por la extensión), máximo 2 MB y nombre aleatorio en disco.
- **Integridad de datos.** Claves foráneas activadas en cada conexión (`PRAGMA foreign_keys = ON`), restricciones `CHECK` y `UNIQUE` en el esquema (incluida la coherencia del total guardado de una reserva) y escritura de reserva y pre-pedido en una sola transacción.
- **Concurrencia.** La comprobación de solapamiento y la inserción de la reserva ocurren dentro de una transacción `BEGIN IMMEDIATE` (con el modo WAL de SQLite), de modo que dos conexiones simultáneas no pueden reservar la misma mesa (`test_dos_conexiones_a_la_vez_solo_una_gana_la_mesa`).
- **Exactitud monetaria.** Los valores se manejan en pesos colombianos enteros, con una única regla de redondeo (mitad hacia arriba, con aritmética entera).
- **Rendimiento.** Las listas de reservas cargan sus pre-pedidos con una sola consulta en lugar de una por plato (Sección 6); los archivos estáticos se sirven con caché y las fotos con versión de 400 px (`srcset`).

- **Mantenibilidad.** Tres capas con dependencias en un solo sentido, DAO sin lógica de negocio, ensamblaje en un único módulo (`dependencias.py`) y 154 pruebas automáticas.
- **Portabilidad y despliegue.** Solo requiere Python y las dependencias de `requirements.txt` (`requirements-prod.txt` para producción); la base SQLite es un archivo y no hay servidor de base de datos que instalar. La configuración se toma de variables de entorno (`SECRET_KEY`, `HTTPS_ONLY`, `RESERVAS_DB`, `AUTOSEMBRAR`).
- **Usabilidad y accesibilidad.** Asistente de reserva por pasos, diseño adaptable (se generan capturas a 1440 y a 375 píxeles de ancho), etiquetas asociadas a los campos, mensajes de error comprensibles y respeto de la preferencia del sistema `prefers-reduced-motion`.

3 Diseño

3.1 Arquitectura en tres capas y MVC

La aplicación se divide en tres capas con dependencia unidireccional: presentación → negocio → datos (Figura 5). La Tabla 8 muestra cómo se corresponden con el modelo–vista–controlador.

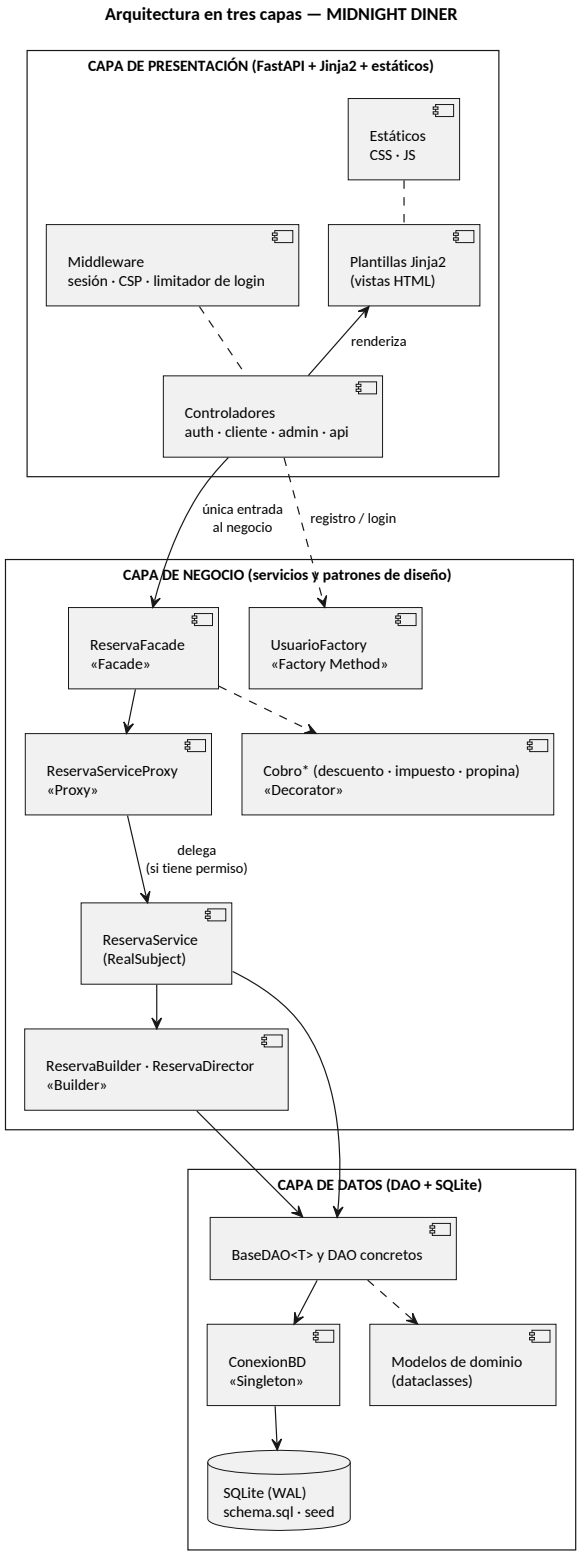


Figura 5. Arquitectura en tres capas y ubicación de los patrones. Elaboración propia a partir de la estructura del proyecto.

Capa	Carpeta	Rol en MVC
Presentación	`app/presentacion/`	**Controlador**: `auth_controller`, `cliente_controller`, `admin_controller` y `api_controller`; **Vista**: plantillas Jinja2, CSS y JavaScript estáticos. `comun.py` concentra sesión, CSRF y guardas de rol; `limitador.py`, el límite de intentos de login.
Negocio	`app/negocio/`	Servicios, reglas de negocio, excepciones y los seis patrones (`patrones/creacionales`, `patrones/estructurales`).
Datos	`app/datos/`	**Modelo**: entidades (`modelos`), DAO, `ConexionBD` (Singleton), `schema.sql` y `seed.py`.
Raíz de composición	`app/dependencias.py`	Construye y conecta DAO, servicios, Proxy y Facade (Sección 3.9).

Tabla 8. Correspondencia entre capas, carpetas y MVC. Elaboración propia.

Los controladores del cliente solo conocen a `ReservaFacade`; el administrador usa el Proxy para las reservas y los servicios directamente para mesas, horarios y carta. Un DAO nunca importa lógica de negocio: `UsuarioDAO` recibe la fábrica de usuarios por inyección.

3.2 Modelo relacional

El esquema (`app/datos/schema.sql`) tiene ocho tablas. Las Figuras 6 y 7 muestran el modelo en dos vistas que comparten las tablas `restaurante` y `reserva`; la Tabla 9 es el diccionario de datos.

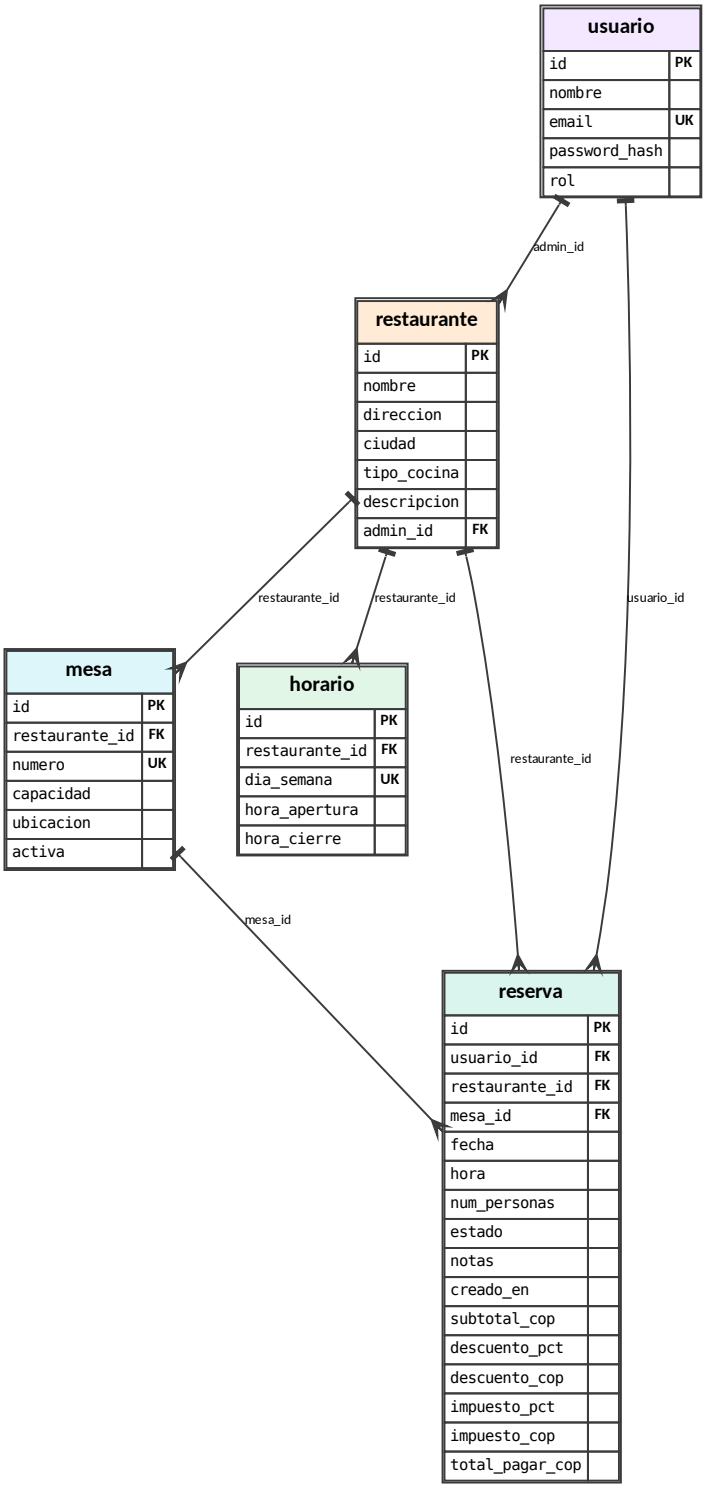


Figura 6. Modelo relacional, vista 1: usuarios, restaurantes, mesas, horarios y reservas. Generado a partir de `schema.sql`.

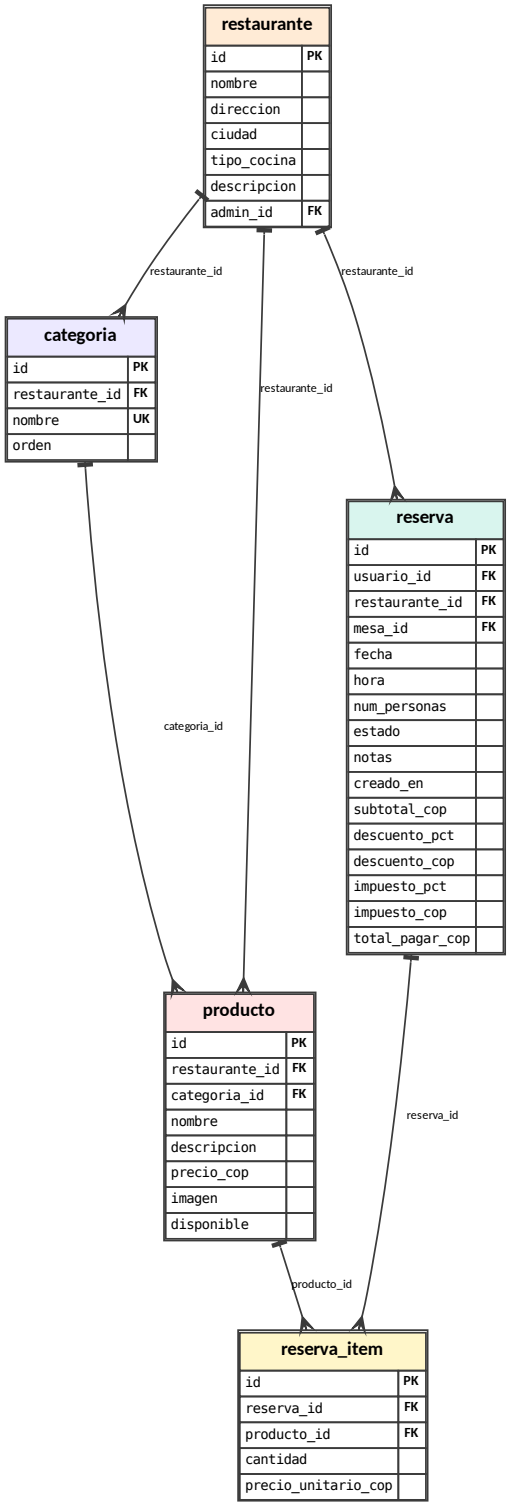


Figura 7. Modelo relacional, vista 2: carta y pre-pedido (`categoria`, `producto`, `reserva_item`). Generado a partir de `schema.sql`.

Decisiones de modelado:

- Las fechas y horas se guardan como texto ISO (`AAAA-MM-DD` y `HH:MM`), porque SQLite no tiene tipo de fecha y así se ordenan y comparan correctamente.
- `reserva_item.precio_unitario_cop` se congela a propósito: si el administrador cambia después el precio de un plato, las reservas ya hechas conservan el precio acordado (RN-6).
- La reserva guarda una *instantánea del cobro* (`subtotal_cop`, `descuento_pct`, `descuento_cop`, `impuesto_pct`, `impuesto_cop` y `total_pagar_cop`). Una restricción `CHECK` exige que el total sea subtotal menos descuento más impuesto, y un cambio posterior de las tasas en `config.py` no altera lo cobrado (`test_cambiar_las_tasas_no_altera_reservas_anteriores`). La propina es voluntaria y no se guarda. Las bases creadas antes de este cambio se migran al arrancar, calculando la instantánea de las reservas activas con las tasas vigentes (aproximación declarada, `test_migracion_agrega_columnas_y_calcula_reservas_anteriores`).
- `reserva.restaurante_id` es redundante con `mesa.restaurante_id`. Se mantiene para consultar por restaurante y fecha sin unir tablas, y su coherencia la garantiza el Builder, que rechaza una mesa de otro restaurante (prueba `test_rechaza_mesa_de_otro_restaurante`).
- Borrar un restaurante elimina en cascada sus mesas, horarios, categorías y productos; borrar una reserva elimina sus ítems.
- Se crean índices para las consultas más frecuentes: `idx_reserva_mesa_fecha`, `idx_reserva_usuario`, `idx_reserva_restaurante_fecha`, `idx_producto_restaurante` e `idx_item_reserva`.

Tabla	Columna	Tipo	Restricciones y descripción
`usuario`	`id`	INTEGER	PK, autoincremental
	`nombre`	TEXT	NN
	`email`	TEXT	NN, único; se guarda en minúsculas
	`password_hash`	TEXT	NN; hash scrypt con sal
	`rol`	TEXT	NN; `cliente` o `administrador`
`restaurante`	`id`	INTEGER	PK
	`nombre`, `direccion`, `ciudad`, `tipo_cocina`	TEXT	NN
	`descripcion`	TEXT	NN, por defecto vacío
	`admin_id`	INTEGER	FK → `usuario`; administrador del restaurante
`mesa`	`id`	INTEGER	PK
	`restaurante_id`	INTEGER	FK → `restaurante`, eliminación en cascada
	`numero`	INTEGER	NN, mayor que 0; único por restaurante

continúa en la página siguiente

Tabla	Columna	Tipo	Restricciones y descripción
	capacidad	INTEGER	NN, mayor que 0
	ubicacion	TEXT	NN, por defecto vacío; define la zona del plano
	activa	INTEGER	NN, 0 o 1; por defecto 1
horario	id	INTEGER	PK
	restaurante_id	INTEGER	FK → restaurante, en cascada
	dia_semana	INTEGER	NN, de 0 (lunes) a 6 (domingo); único por restaurante
	hora_apertura	TEXT	NN, HH:MM
	hora_cierre	TEXT	NN, HH:MM; mayor que la apertura
reserva	id	INTEGER	PK
	usuario_id	INTEGER	FK → usuario; cliente que reserva
	restaurante_id	INTEGER	FK → restaurante
	mesa_id	INTEGER	FK → mesa
	fecha, hora	TEXT	NN, ISO
	num_personas	INTEGER	NN, mayor que 0
	estado	TEXT	NN; PENDIENTE, CONFIRMADA o CANCELADA; por defecto PENDIENTE
	notas	TEXT	NN, por defecto vacío (máx. 300 caracteres)
	creado_en	TEXT	NN, marca de tiempo ISO
	subtotal_cop	INTEGER	NN, mayor o igual a 0, por defecto 0; subtotal del pre-pedido
	descuento_pct, impuesto_pct	INTEGER	NN, de 0 a 100; tasas aplicadas al reservar
	descuento_cop, impuesto_cop	INTEGER	NN, mayor o igual a 0; montos aplicados
	total_pagar_cop	INTEGER	NN; CHECK: igual a subtotal menos descuento más impuesto
categoria	id	INTEGER	PK
	restaurante_id	INTEGER	FK → restaurante, en cascada
	nombre	TEXT	NN; único por restaurante
	orden	INTEGER	NN, por defecto 0
producto	id	INTEGER	PK
	restaurante_id	INTEGER	FK → restaurante, en cascada
	categoria_id	INTEGER	FK → categoria
	nombre, descripcion	TEXT	nombre NN; descripcion por defecto vacío
	precio_cop	INTEGER	NN, mayor o igual a 0; pesos enteros
	imagen	TEXT	NN, nombre de archivo; por defecto vacío
	disponible	INTEGER	NN, 0 o 1; por defecto 1
reserva_item	id	INTEGER	PK

continúa en la página siguiente

Tabla	Columna	Tipo	Restricciones y descripción
	`reserva_id`	INTEGER	FK → `reserva`, en cascada
	`producto_id`	INTEGER	FK → `producto`
	`cantidad`	INTEGER	NN, mayor que 0
	`precio_unitario_cop`	INTEGER	NN, mayor o igual a 0; precio congelado al reservar

Tabla 9. Diccionario de datos del esquema relacional (PK: clave primaria; FK: clave foránea; NN: no nulo). Elaboración propia a partir de `schema.sql`.

3.3 Diagrama de clases de la capa de entidades

Las entidades del dominio son clases de datos sin lógica de persistencia. `Usuario` es la clase base de `Cliente` y `Administrador` (Factory Method), y cada entidad se corresponde con una tabla. La Figura 8 muestra los atributos, los métodos y las multiplicidades.

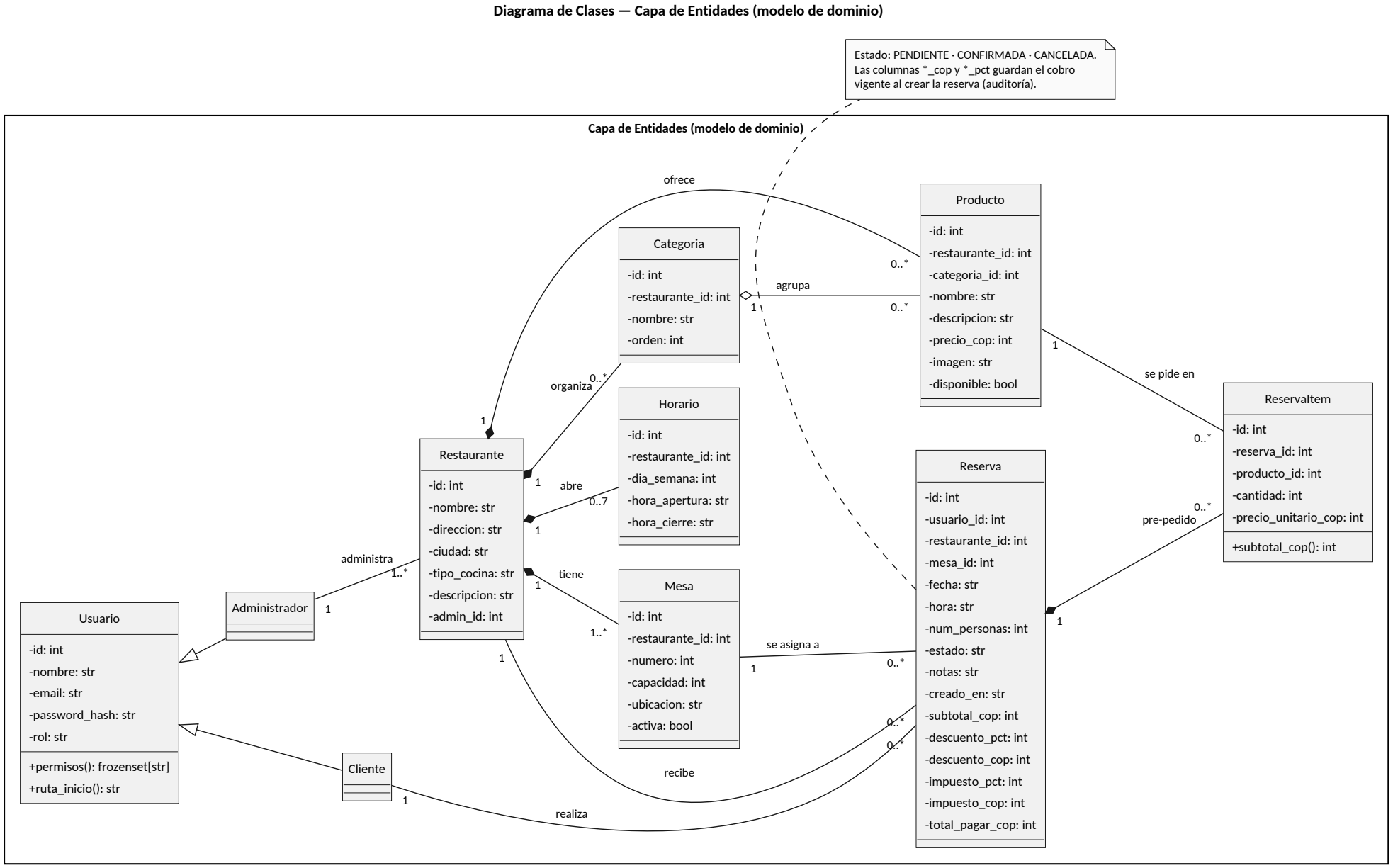


Figura 8. Diagrama de clases de la capa de entidades (modelo de dominio) con sus multiplicidades. Elaboración propia a partir de `modelos/__init__.py`.

3.4 Diagrama de clases de la capa de datos (DAO)

La capa de datos usa el patrón DAO (*Data Access Object*; Alur et al., 2003): cada tabla tiene un objeto que encapsula el SQL y devuelve entidades del dominio, de modo que ninguna otra capa escribe consultas. `BaseDAO<T>` ofrece el CRUD común (`obtener_por_id`, `listar`, `insertar`, `actualizar`, `eliminar`) a partir de dos constantes de clase, `tabla` y `columnas`, y obliga a cada subclase a implementar `_desde_fila`, que convierte una fila en entidad. Todos los DAO obtienen sus conexiones de `ConexionBD` (Singleton), que además ofrece `sesion_inmediata()` para las escrituras que deben comprobar y guardar de forma atómica. La Figura 9 muestra los ocho DAO.

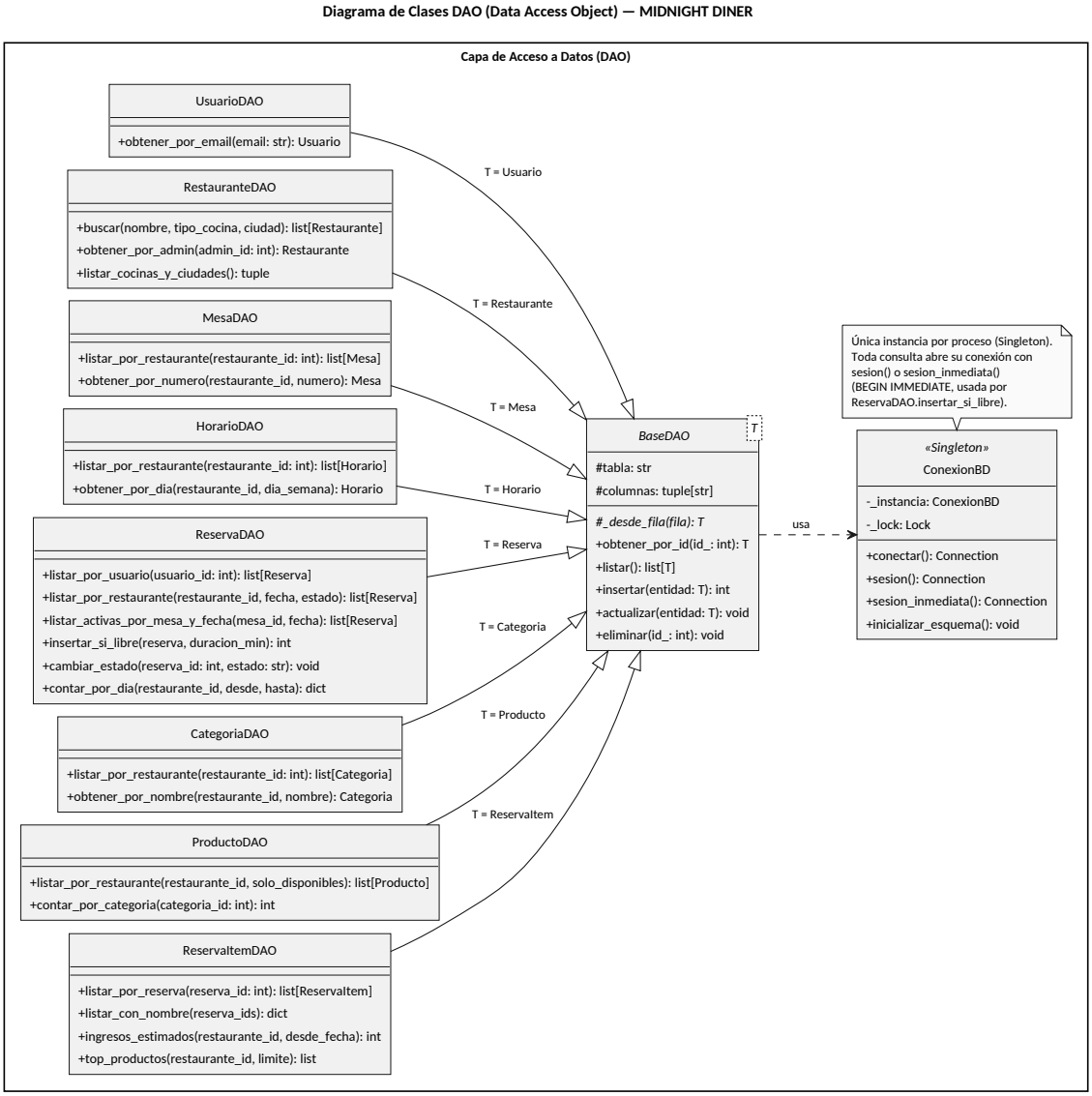


Figura 9. Diagrama de clases DAO (Data Access Object). Elaboración propia a partir de `datos/dao/__init__.py`.

3.5 Diagrama de clases de control (patrones)

La Figura 10 presenta el diagrama de clases de control con los seis patrones, sus servicios y los DAO que usan; el detalle de cada patrón, con su código y sus pruebas, está en la Sección 4.

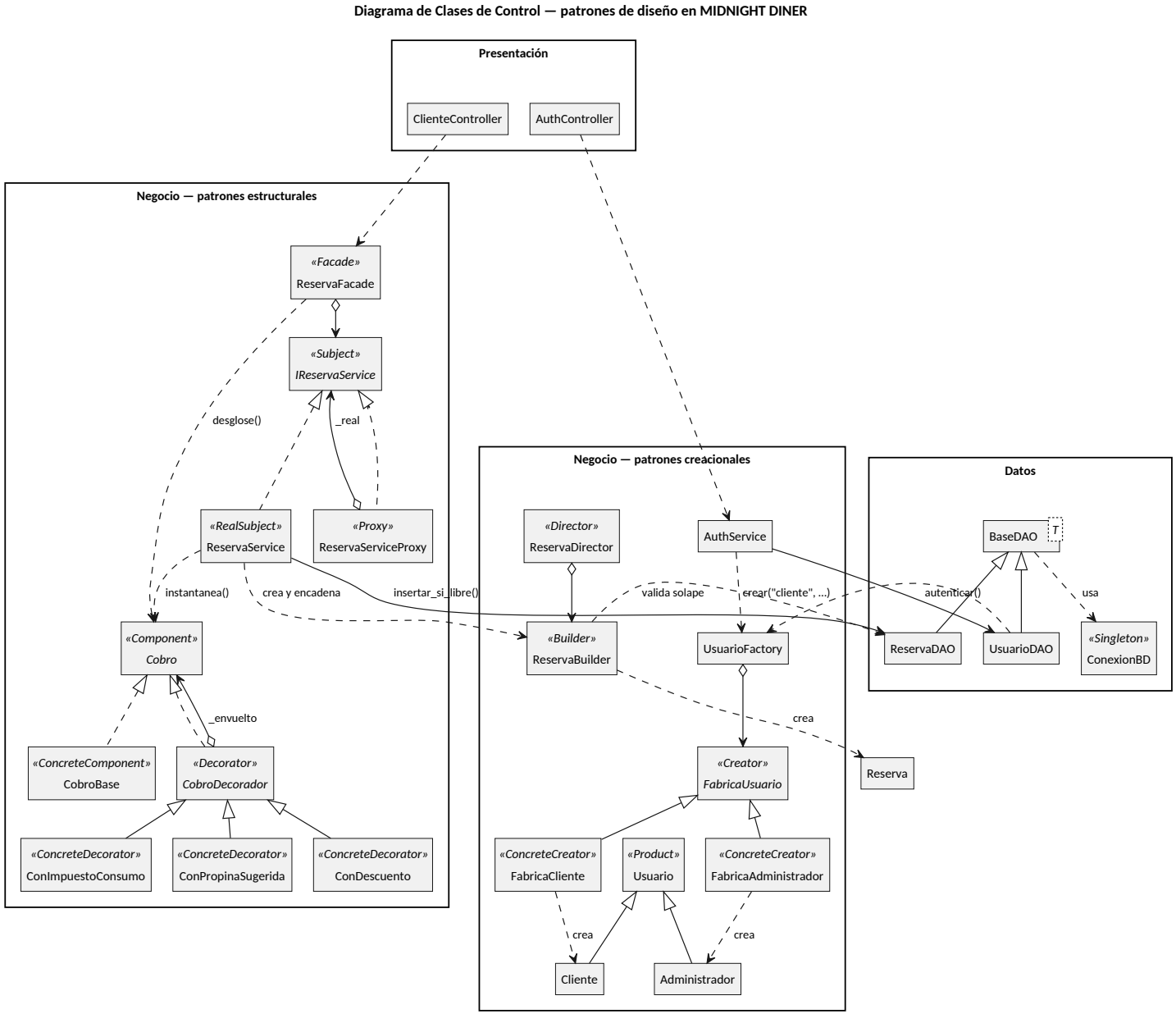


Figura 10. Diagrama de clases de control con los seis patrones de diseño. Elaboración propia a partir del código.

3.6 Diagramas de estados

Se modelan tres clases con comportamiento por estados: la reserva, el acceso (intentos de inicio de sesión) y el asistente de reserva de la interfaz.

La reserva (Figura 11) nace `PENDIENTE`; el administrador puede confirmarla y tanto el dueño como el administrador pueden cancelarla, pero una cancelada no vuelve atrás. Las transiciones válidas están en el diccionario `TRANSICIONES` de `ReservaService`.

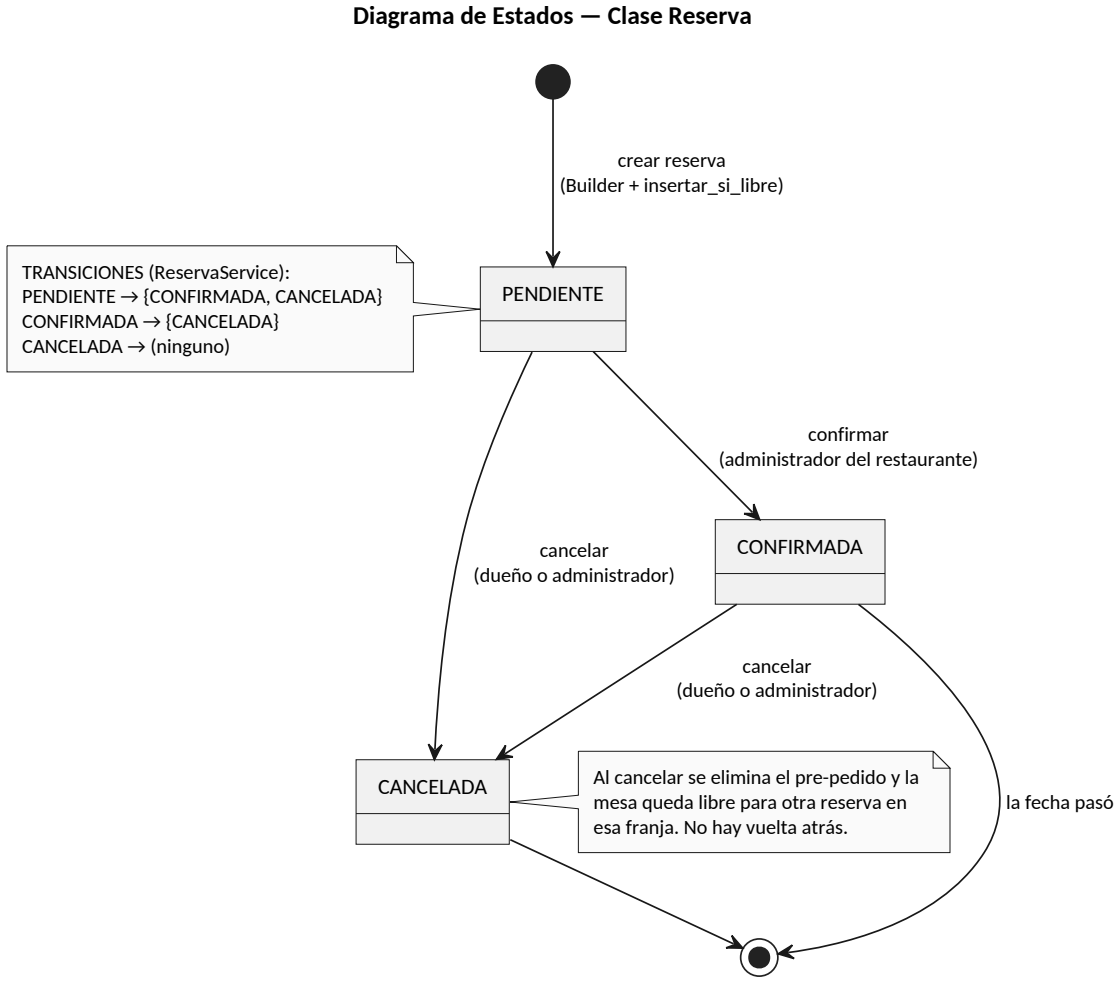


Figura 11. Diagrama de estados de la clase `Reserva`. Elaboración propia a partir de `reserva_service.py`.

El acceso (Figura 12) refleja el limitador de intentos: los fallos se acumulan y, al llegar al umbral, el acceso queda bloqueado hasta que pasan 5 minutos.

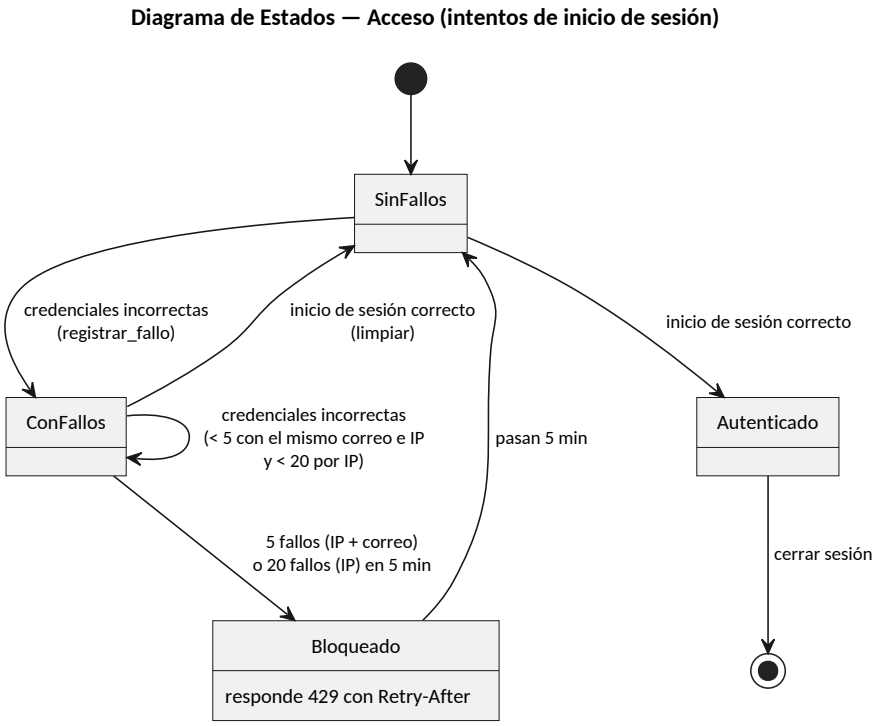


Figura 12. Diagrama de estados del acceso (intentos de inicio de sesión). Elaboración propia a partir de `limitador.py` y `auth_controller.py`.

El asistente de reserva (Figura 13) recorre cinco pasos y puede retroceder; si el servidor rechaza la reserva (mesa ocupada o regla incumplida), vuelve al paso de la mesa con los datos conservados.

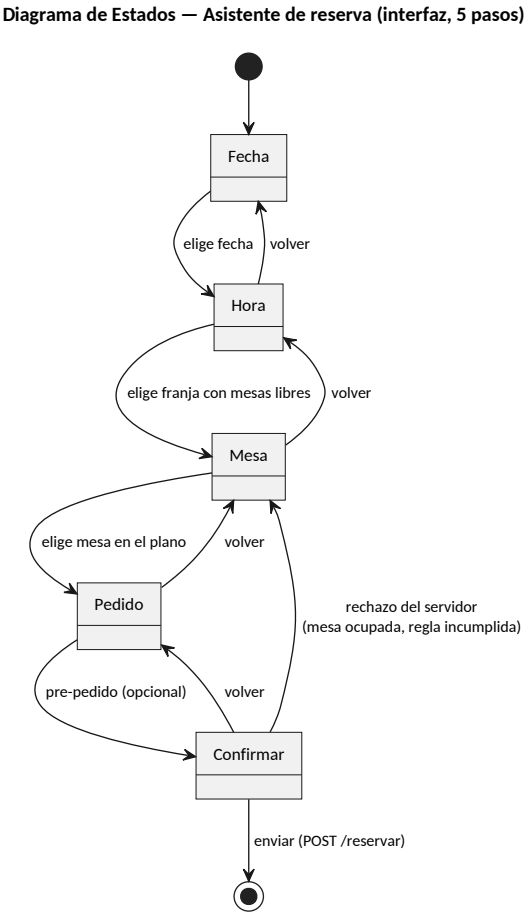


Figura 13. Diagrama de estados del asistente de reserva (cinco pasos). Elaboración propia a partir de `reservar.js`.

3.7 Diagramas de colaboración

Los diagramas de colaboración muestran los objetos que participan en un proceso y los mensajes numerados que se envían. Se presentan los dos procesos principales: iniciar sesión (Figura 14) y reservar mesa (Figura 15).

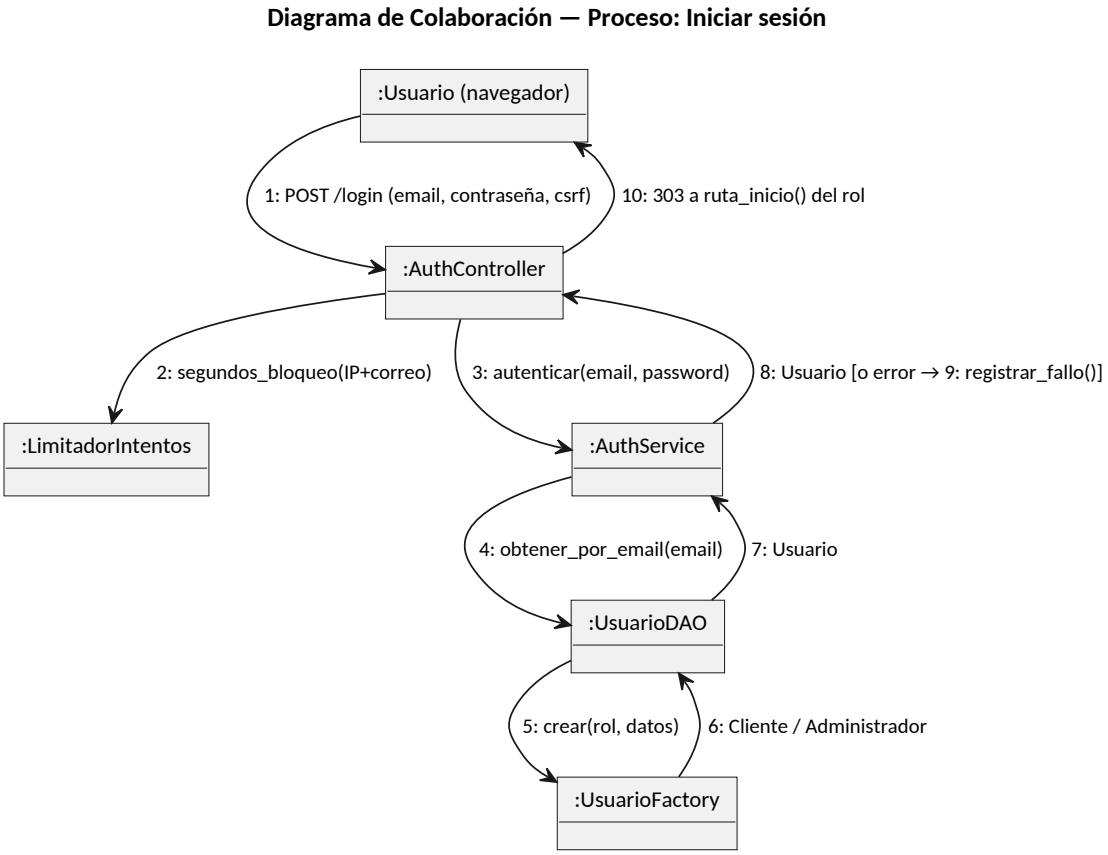


Figura 14. Diagrama de colaboración: iniciar sesión. Elaboración propia a partir de `auth_controller.py`.

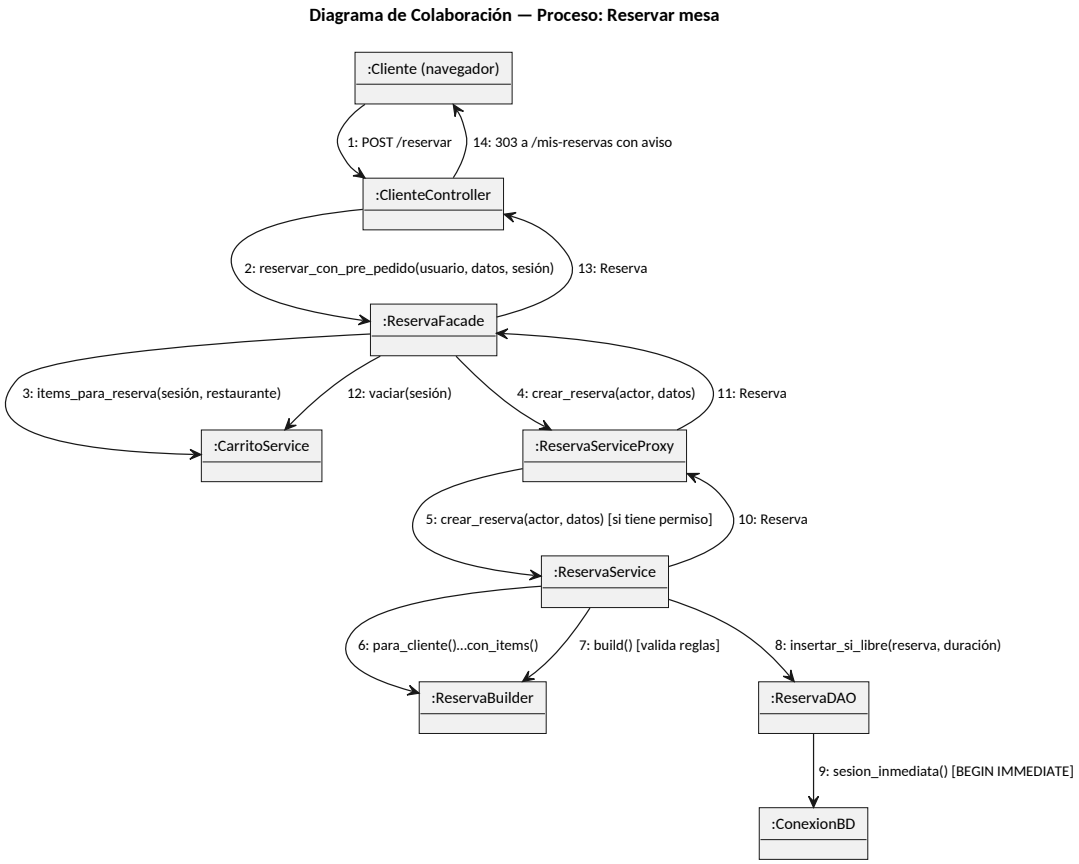


Figura 15. Diagrama de colaboración: reservar mesa. Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py` y `reserva_service.py`.

3.8 Diagramas de secuencia

Los diagramas de secuencia detallan, en el tiempo, los mismos procesos. La Figura 16 sigue `POST /login`: el controlador valida CSRF, consulta el limitador, delega la autenticación en el servicio y el DAO (que crea el usuario con la fábrica) y responde según el resultado. Las Figuras 17 y 18 siguen `POST /reservar` de punta a punta, divididas en dos para conservar la legibilidad: la fachada completa los datos, el Proxy autoriza y registra, el servicio construye la reserva con el Builder, congela el cobro y el DAO la guarda junto con su pre-pedido en una transacción exclusiva. La Figura 19 muestra el control de acceso del Proxy al cancelar.

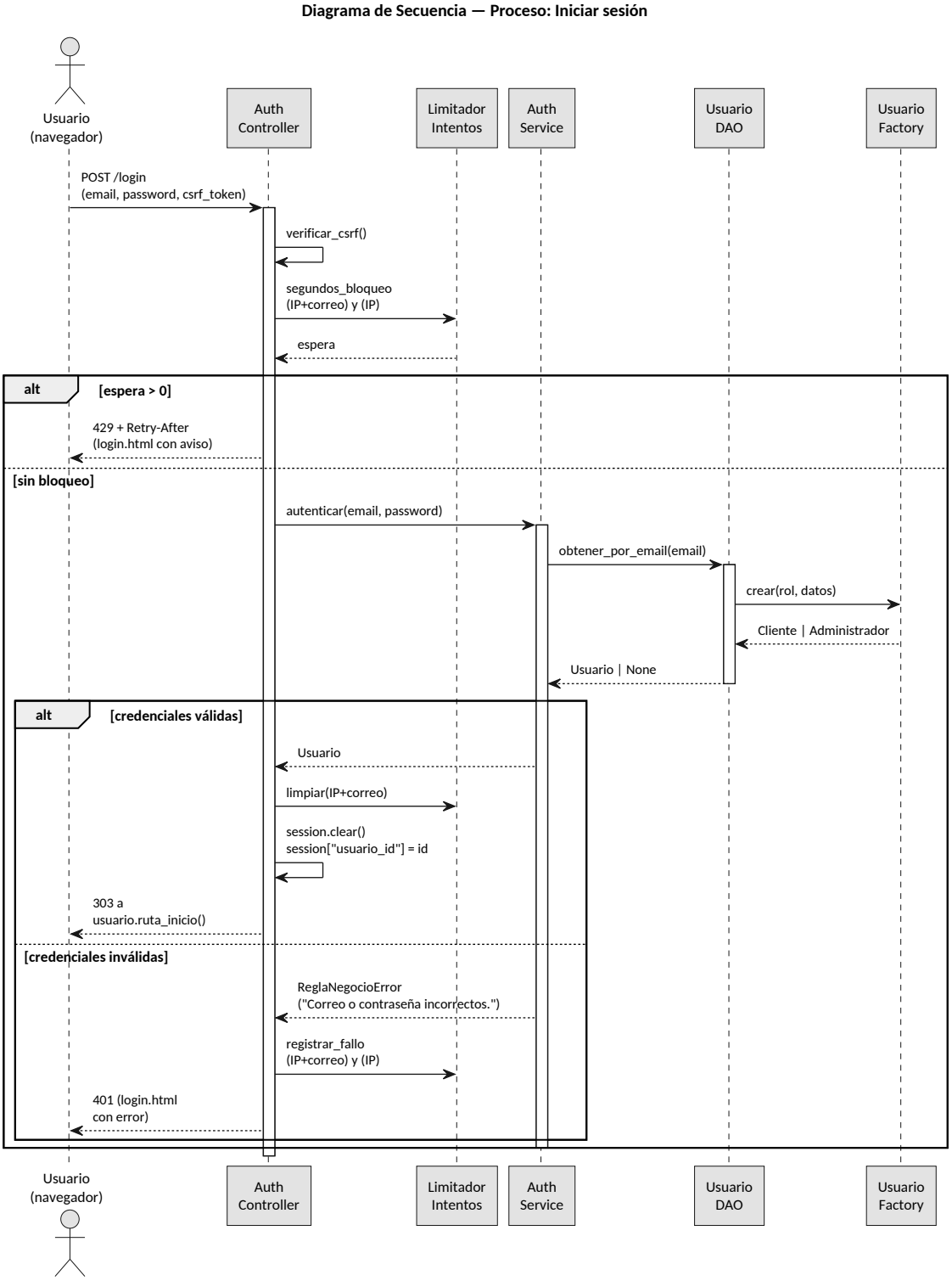


Figura 16. Diagrama de secuencia: iniciar sesión. Elaboración propia a partir de `auth_controller.py`.

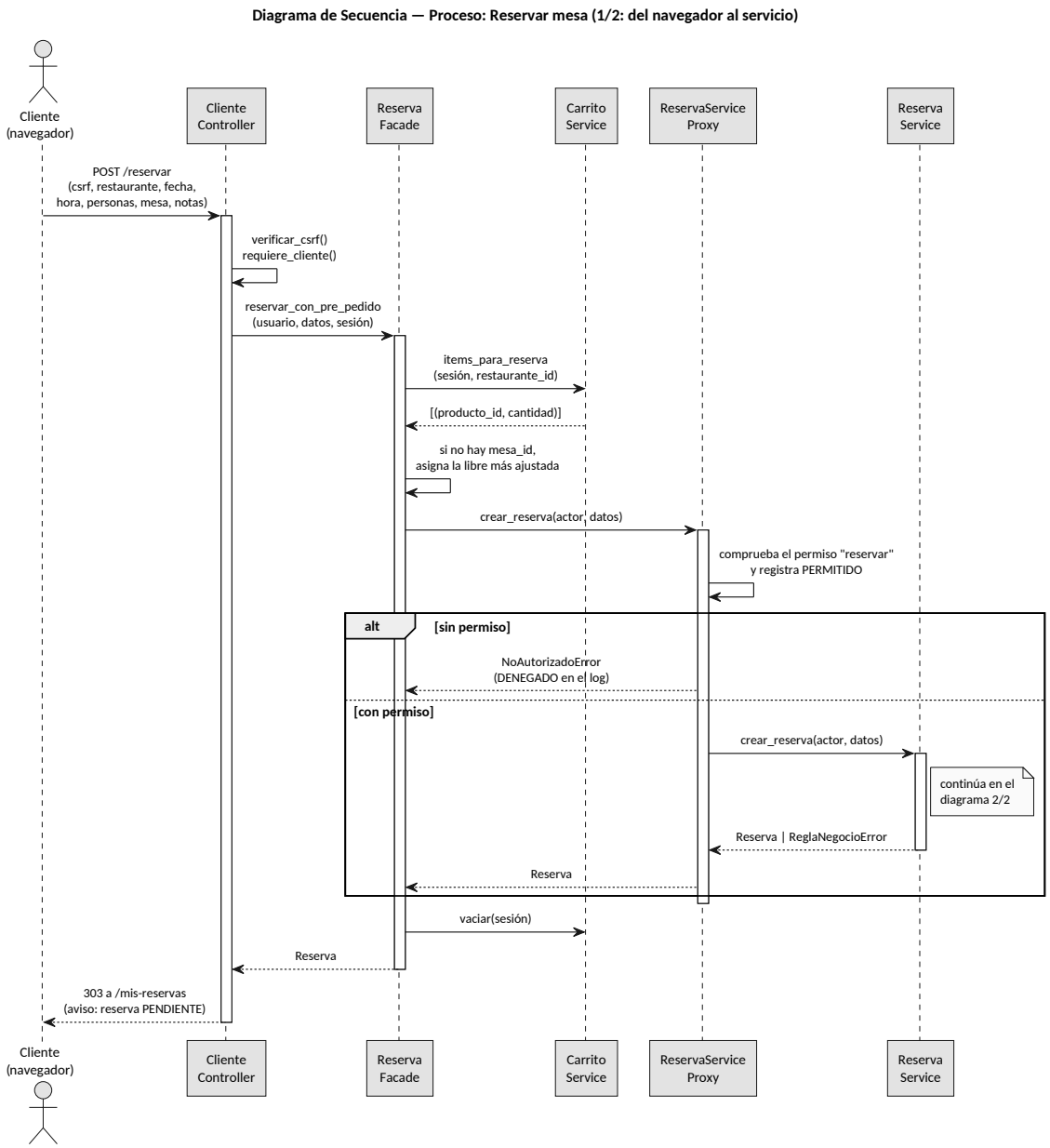


Figura 17. Diagrama de secuencia: reservar mesa (1/2: del navegador al servicio). Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py`, `reserva_proxy.py`, `reserva_service.py` y `reserva_builder.py`.

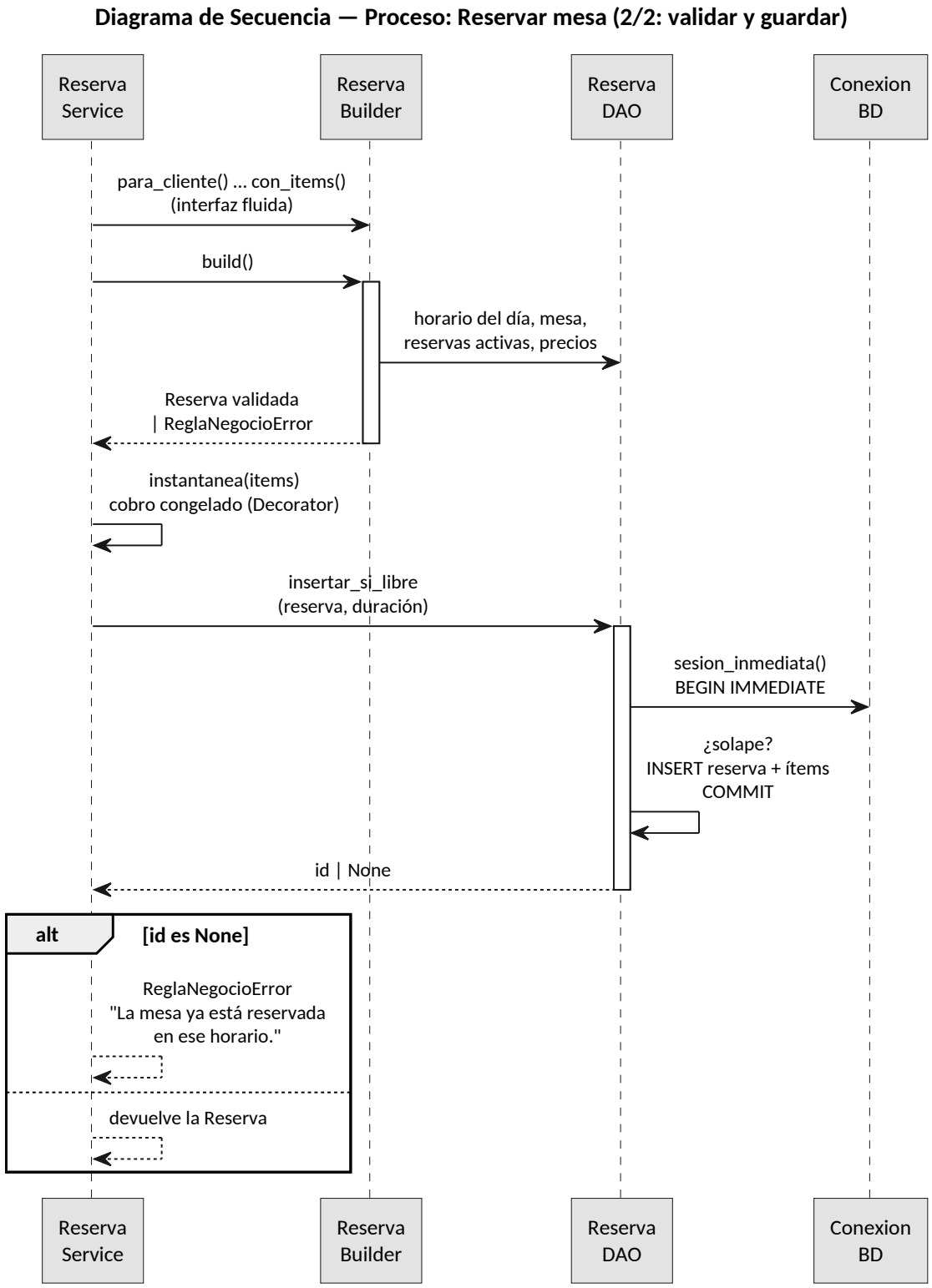


Figura 18. Diagrama de secuencia: reservar mesa (2/2: validar y guardar). Elaboración propia a partir de `cliente_controller.py`, `reserva_facade.py`, `reserva_proxy.py`, `reserva_service.py` y `reserva_builder.py`.

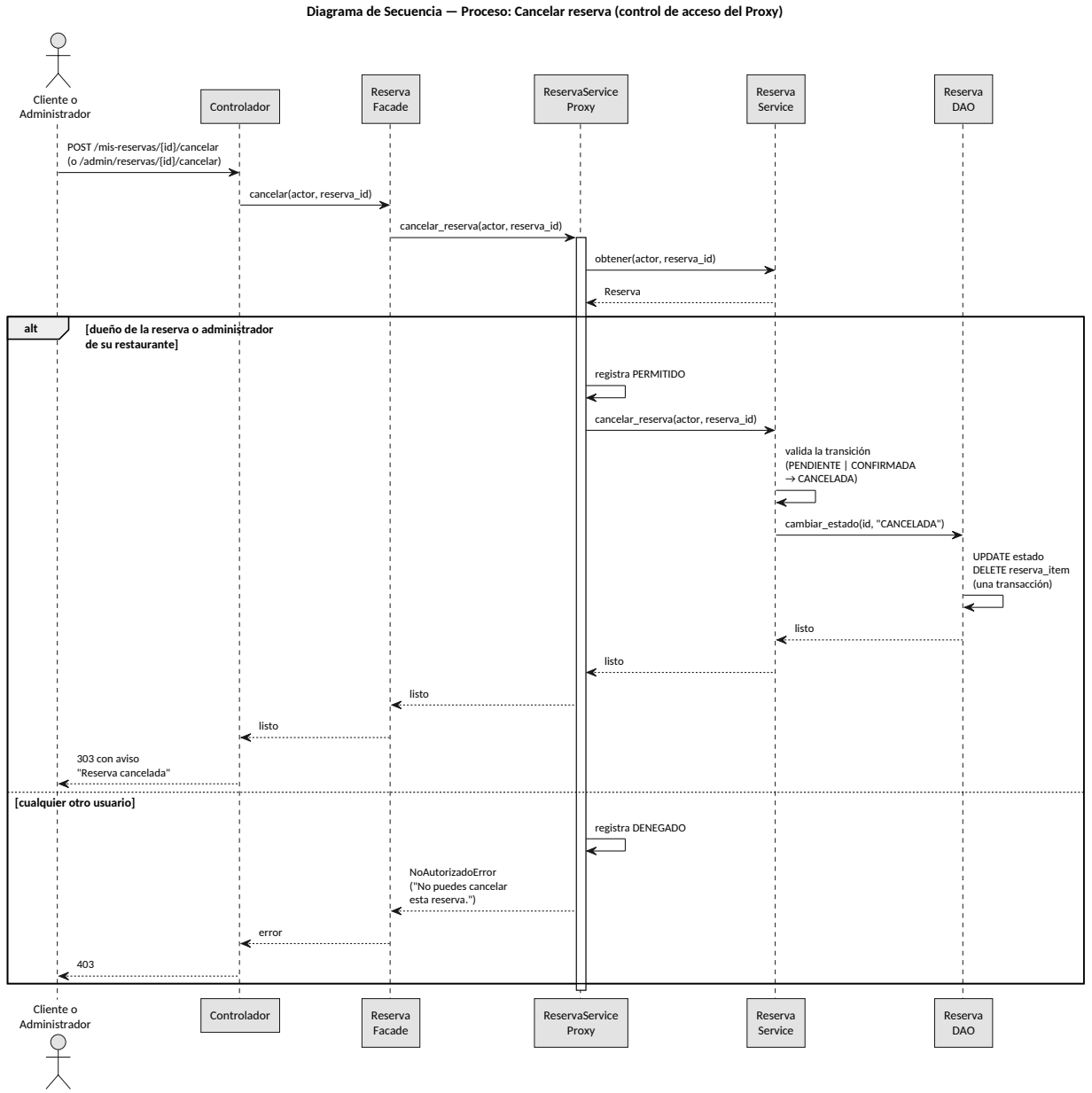


Figura 19. Diagrama de secuencia: cancelar una reserva (control de acceso del Proxy). Elaboración propia a partir de `reserva_proxy.py` y `reserva_service.py`.

3.9 Raíz de composición

Todas las dependencias se ensamblan en `app/dependencias.py` (Figura 20). Allí se ve la regla central del diseño: los controladores reciben el *Proxy*, nunca el servicio real, y la fábrica de usuarios se inyecta en el DAO.

```
16  usuario_dao = UsuarioDAO(UsuarioFactory.crear)      # Factory Method inyectado en el DAO
    …
30  reserva_service_real = ReservaService(
31      reserva_dao, mesa_service,
32      builder_factory=lambda: ReservaBuilder(mesa_dao, horario_dao, reserva_dao, producto_dao=producto_dao))
33  reserva_service = ReservaServiceProxy(reserva_service_real, restaurante_service)   # los controladores solo ven el proxy
34
35  carta_service = CartaService(categoria_dao, producto_dao, item_dao, restaurante_service)
36  carrito_service = CarritoService(producto_dao)
37  estadisticas_service = EstadisticasService(reserva_dao, mesa_dao, horario_dao, item_dao)
38
39  fachada = ReservaFacade(reserva_service, restaurante_service, mesa_service, horario_service, auth_service,
40                          carta_service, carrito_service)
```

Figura 20. Ensamblaje de DAO, servicios, Proxy y Facade en `dependencias.py` (fragmentos; los números de línea son los del archivo).

4 Patrones de diseño aplicados

La Tabla 10 resume los patrones, el problema concreto que resuelve cada uno y dónde están. Las definiciones y los roles siguen a Gamma et al. (1994). En las imágenes de código, los números de línea son los del archivo; las imágenes se generaron a partir de los archivos reales del proyecto, no se escribieron aparte.

Patrón	Clase(s)	Problema que resuelve aquí	Archivo
Singleton (creacional)	`ConexionBD`	Un único punto que conoce la base y activa las claves foráneas en cada conexión.	`datos/conexion.py`
Factory Method (creacional)	`FabricaUsuario`, `UsuarioFactory`	Crear `Cliente` o `Administrador` según el rol sin condicionales dispersos.	`patrones/creacionales/usuario_factory.py`
Builder (creacional)	`ReservaBuilder`, `ReservaDirector`	Construir una reserva con muchos datos y validar todas las reglas en un solo paso.	`patrones/creacionales/reserva_builder.py`
Facade (estructural)	`ReservaFacade`	Dar al controlador del cliente una interfaz simple sobre siete servicios.	`patrones/estructurales/reserva_facade.py`
Proxy de protección (estructural)	`ReservaServiceProxy`	Comprobar rol y propiedad, y registrar cada operación, sin tocar el servicio real.	`patrones/estructurales/reserva_proxy.py`
Decorator (estructural)	`Cobro`, `ConDescuento`, `ConImpuestoConsumo`, `ConPropinaSugerida`	Sumar o restar conceptos al total del pre-pedido combinándolos en cadena.	`patrones/estructurales/cobro_decorator.py`

Tabla 10. Patrones de diseño aplicados (rutas relativas a `app/negocio/`, salvo `conexion.py`, relativa a `app/`). Elaboración propia.

La Tabla 11 relaciona cada patrón con los requerimientos y reglas que sirve y con las pruebas que lo ejercitan, para que se pueda seguir el rastro desde la necesidad hasta la evidencia.

Patrón	Requerimientos y reglas	Pruebas
Singleton	Integridad de datos, concurrencia (RNF)	`test_singleton_misma_instancia`; `test_singleton_seguro_con_hilos`; `test_singleton_activa_claves_foraneas`; `test_sqlite_en_modo_wal`
Factory Method	RF-01, RF-02	`test_factory_crea_subclase_segun_rol`; `test_factory_rechaza_rol_desconocido`; `test_dao_y_login_usan_la_factory`
Builder	RF-07, RF-08; RN-1, RN-2, RN-3, RN-6	`test_builder_falla_sin_mesa`; `test_builder_interfaz_fluida_construye_reserva`; `test_builder_valida_en_build`; `test_director_reserva_rapida`; las 17 de `test_reglas_negocio.py`
Facade	RF-04 a RF-11	`test_facade_usa_el_proxy_y_no_el_servicio_real`; `test_facade_mis_reservas_y_cancelar`; `test_fachada_pre_pedido_vacia_el_carrito`; `test_facade_disponibilidad_y_reserva_con_mesa_automatica`
Proxy	RN-4; RF-11, RF-15	`test_proxy_rechaza_cancelar_reserva_ajena`; `test_proxy_rechaza_ver_reserva_ajena`; `test_proxy_admin_solo_gestiona_su_restaurante`; `test_proxy_cliente_no_confirma_y_admin_no_reserva`; `test_proxy_registra_operaciones`
Decorator	RF-09; RN-6	32 pruebas de `test_fase4.py` y 9 de `test_cobro_guardado.py`

Tabla 11. Trazabilidad entre patrones, requerimientos y pruebas. Elaboración propia a partir del código y de las pruebas.

4.1 Patrones creacionales

4.1.1 Singleton: `ConexionBD`

Problema. Los DAO necesitan la misma configuración de base de datos y, en SQLite, las claves foráneas vienen desactivadas por defecto: si una conexión olvida activarlas, se pierde la integridad referencial. **Solución.** `ConexionBD` garantiza una sola instancia (con bloqueo y doble verificación para que sea segura entre hilos) que conoce la ruta de la base y entrega cada conexión ya configurada: `PRAGMA foreign_keys = ON`, modo WAL y filas accesibles por nombre. El método `sesion()` es un administrador de contexto que confirma, revierte si hay una excepción y cierra siempre; `sesion_inmediata()` hace lo mismo abriendo la transacción con `BEGIN IMMEDIATE` (Sección 6). **Precisión.** Lo único que el Singleton comparte es la configuración: cada `conectar()` abre una conexión nueva, porque las conexiones de `sqlite3` no deben compartirse entre hilos. `reiniciar()` permite a las pruebas apuntar a otra base.

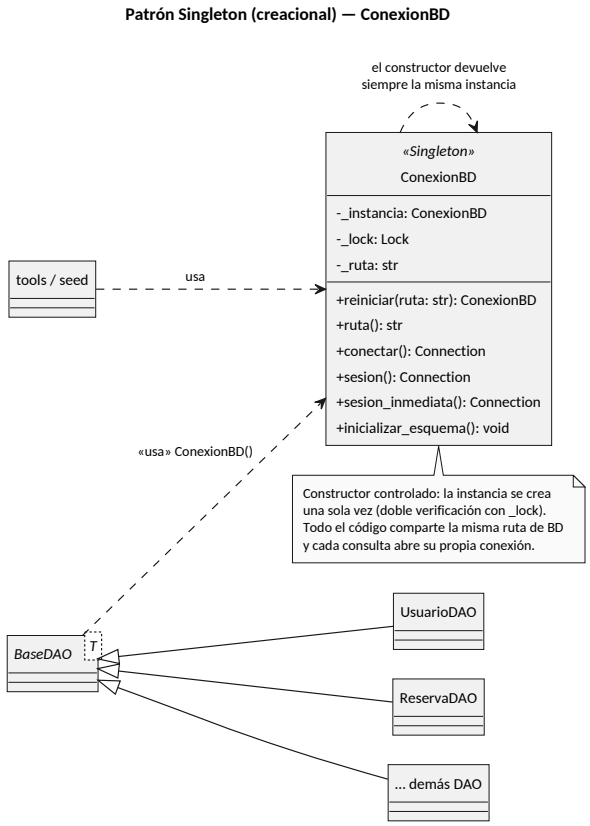


Figura 21. Singleton: clase `ConexionBD`. Elaboración propia a partir del código.

```python
class ConexionBD:
    """PATRÓN CREACIONAL · Singleton (rol: Singleton).

    Garantiza una única instancia que conoce la ruta de la base de datos y
    entrega conexiones SQLite con las claves foráneas activadas. La creación es
    segura entre hilos: bloqueo + doble verificación.
    """

    _instancia: "ConexionBD | None" = None
    _lock = threading.Lock()

    def __new__(cls, ruta: str | None = None) -> "ConexionBD":
        if cls._instancia is None:                # 1.ª verificación (sin bloqueo)
            with cls._lock:
                if cls._instancia is None:        # 2.ª verificación (con bloqueo)
                    instancia = super().__new__(cls)
                    instancia._ruta = str(ruta or config.DB_PATH)
                    cls._instancia = instancia
        return cls._instancia
    …
    def conectar(self) -> sqlite3.Connection:
        """Abre una conexión nueva con FK activadas y filas accesibles por nombre."""
        conexion = sqlite3.connect(self._ruta, timeout=10)      # espera hasta 10 s si otra conexión escribe
        conexion.row_factory = sqlite3.Row
        conexion.execute("PRAGMA foreign_keys = ON")
        conexion.execute("PRAGMA journal_mode = WAL")           # lectores y escritor no se bloquean entre sí
        conexion.execute("PRAGMA synchronous = NORMAL")         # seguro con WAL y bastante más rápido
        return conexion

    @contextmanager
    def sesion(self) -> Iterator[sqlite3.Connection]:
        """Conexión con commit/rollback automático y cierre garantizado."""
        conexion = self.conectar()
        try:
            yield conexion
            conexion.commit()
        except Exception:
            conexion.rollback()
            raise
        finally:
            conexion.close()
```

Figura 22. Código de `ConexionBD` (`conexion.py`): doble verificación, `conectar` y `sesion`.

Pruebas. `test_singleton_misma_instancia`, `test_singleton_seguro_con_`

`hilos` (20 hilos obtienen la misma instancia) y `test_singleton_activa_claves_foraneas`.

```
45 def test_singleton_seguro_con_hilos(bd):
46     ConexionBD.reiniciar()
47     instancias = []
48     hilos = [threading.Thread(target=lambda: instancias.append(ConexionBD())) for _ in range(20)]
49     [h.start() for h in hilos]
50     [h.join() for h in hilos]
51     assert len({id(i) for i in instancias}) == 1
```

Figura 23. Prueba de concurrencia del Singleton (`test_patrones.py`).

4.1.2 Factory Method: `FabricaUsuario`

Problema. El sistema tiene dos tipos de usuario con permisos y pantalla de inicio distintos. Decidirlo con `if rol == ...` en cada lugar (DAO, login, registro) duplica la lógica y obliga a editar muchos sitios al agregar un rol. **Solución.** `FabricaUsuario` (*Creator*) declara el método fábrica `_instanciar`; `FabricaCliente` y `FabricaAdministrador` (*ConcreteCreator*) deciden la subclase de `Usuario` (*Product*) que se crea. El método `crear` de la fábrica base normaliza el correo (recorta y pasa a minúsculas) para todos los productos. `UsuarioFactory` es el punto de entrada: elige el creador por el nombre del rol y rechaza roles desconocidos. Cada producto implementa `permisos()` y `ruta_inicio()`, de modo que el login redirige al cliente a `/restaurantes` y al administrador a `/admin` sin condicionales. La fábrica la usan `UsuarioDAO` (inyectada) y `AuthService`.

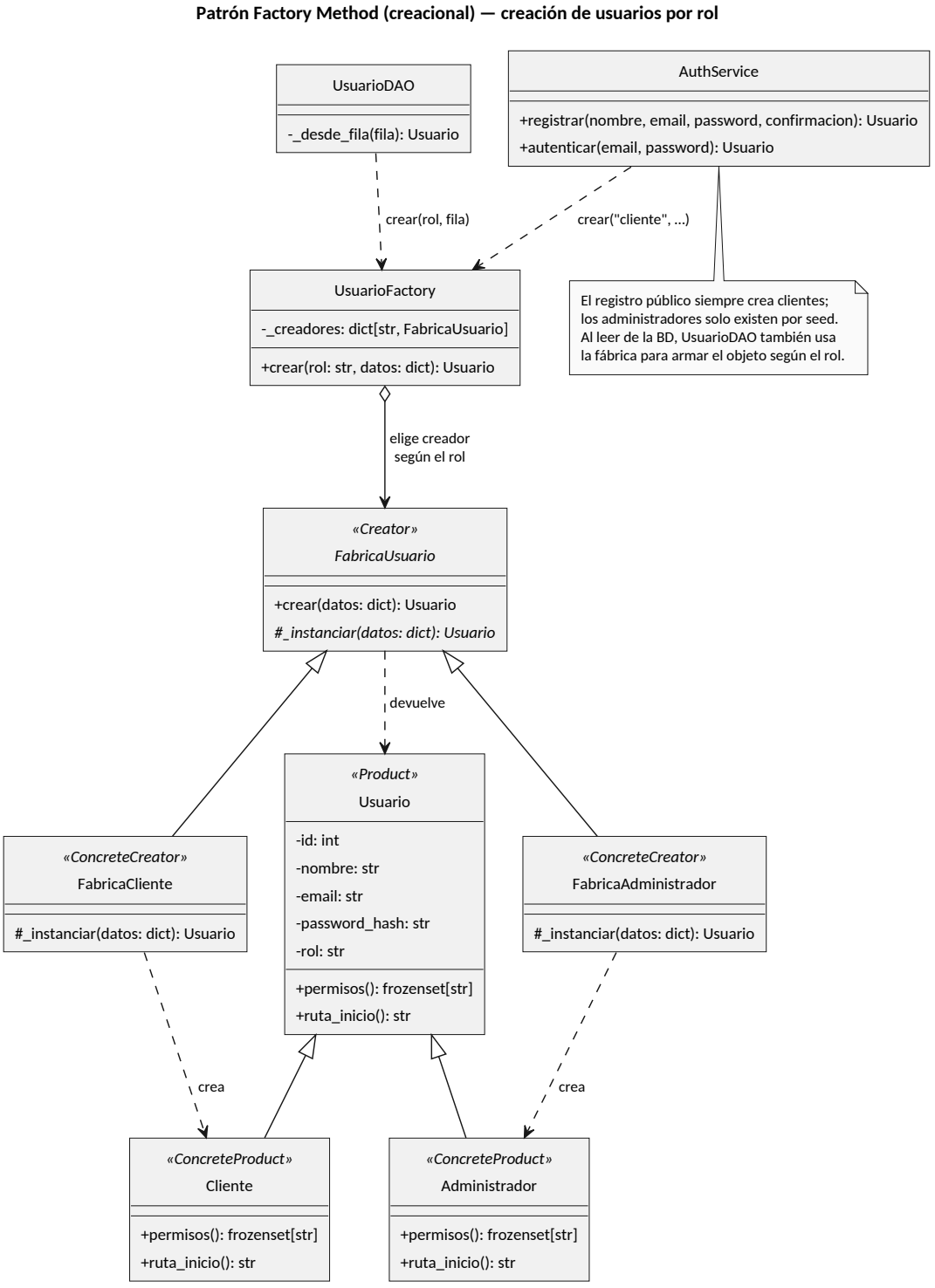


Figura 24. Factory Method: creadores y productos de usuario. Elaboración propia a partir del código.

```python
class FabricaUsuario(ABC):
    """PATRÓN CREACIONAL · Factory Method (rol: Creator).

    Declara el método fábrica `_instanciar`; `crear` lo usa y normaliza los datos.
    """

    def crear(self, datos: dict) -> Usuario:
        usuario = self._instanciar(datos)
        usuario.email = usuario.email.strip().lower()
        return usuario

    @abstractmethod
    def _instanciar(self, datos: dict) -> Usuario:
        """Método fábrica: cada creador concreto decide la subclase."""


class FabricaCliente(FabricaUsuario):
    """Factory Method · rol: ConcreteCreator (produce `Cliente`)."""

    def _instanciar(self, datos: dict) -> Usuario:
        return Cliente(id=datos.get("id"), nombre=datos["nombre"], email=datos["email"],
                       password_hash=datos.get("password_hash", ""))


class UsuarioFactory:
    """Punto de entrada: elige el creador concreto según el rol.

    Lo usan `UsuarioDAO` (al reconstruir filas) y `AuthService` (tras el login/registro).
    """

    _creadores: dict[str, FabricaUsuario] = {
        "cliente": FabricaCliente(),
        "administrador": FabricaAdministrador(),
    }

    @classmethod
    def crear(cls, rol: str, datos: dict) -> Usuario:
        creador = cls._creadores.get(rol)
        if creador is None:
            raise ReglaNegocioError(f"Rol desconocido: {rol!r}")
        return creador.crear(datos)
```

Figura 25. Código del Factory Method (`usuario_factory.py`).

Pruebas. `test_factory_crea_subclase_segun_rol`, `test_factory_rechaza_rol_desconocido` y `test_dao_y_login_usan_la_factory`.

4.1.3 Builder: `ReservaBuilder`

Problema. Una reserva reúne cliente, restaurante, fecha, hora, personas, mesa, notas y pre-pedido, y debe cumplir varias reglas a la vez. Un constructor con ocho parámetros es ilegible y permite crear objetos inválidos. **Solución.** `ReservaBuilder` (*Builder* y *ConcreteBuilder*) acumula los datos con una interfaz fluida (`para_cliente`, `en_restaurante`, `el_dia`, `a_las`, `para_personas`, `en_mesa`, `con_notas`, `con_items`) y `build()` valida todas las reglas de negocio (futura y dentro del horario, mesa activa y con capacidad, sin solapamiento y pre-pedido válido) antes de devolver el producto `Reserva`; si falta algún dato, lo dice por su nombre. Los precios del pre-pedido se leen de la base en `build()`, nunca del navegador. `ReservaDirector.reserva_rapida` (*Director*) encapsula una receta predefinida: mesa para dos personas, sin notas. El reloj es inyectable (`ahora`), lo que permite probar las reglas de fecha sin esperar.

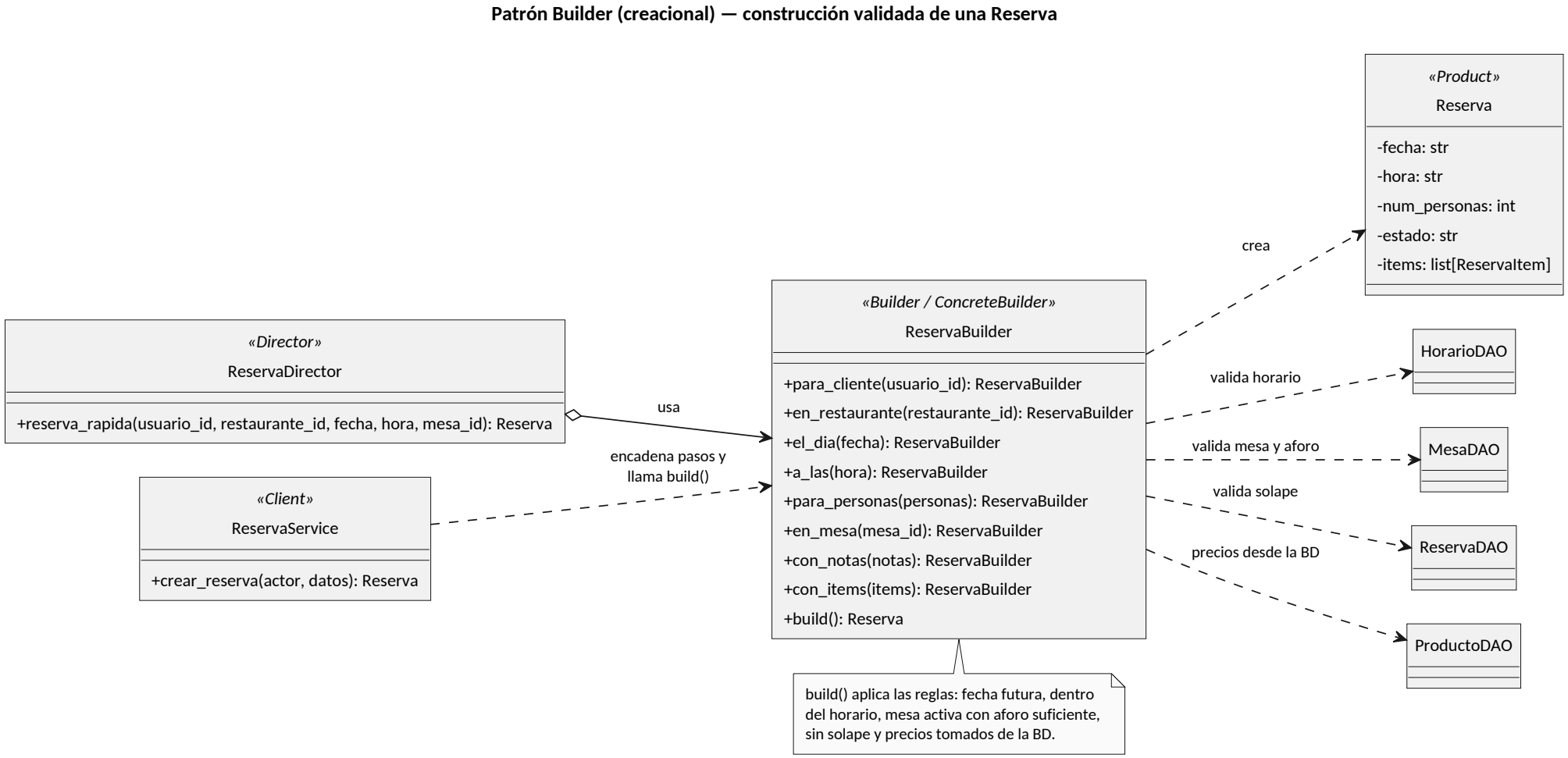


Figura 26. Builder: constructor, director y producto. Elaboración propia a partir del código.

```python
class ReservaBuilder:
    """PATRÓN CREACIONAL · Builder (rol: Builder + ConcreteBuilder).

    Interfaz fluida que acumula los datos de la reserva; `build()` (el paso final)
    valida TODAS las reglas de negocio y devuelve el producto `Reserva`.
    """

    def __init__(self, mesa_dao: MesaDAO, horario_dao: HorarioDAO, reserva_dao: ReservaDAO,
                 ahora: Callable[[], datetime] = datetime.now, producto_dao: ProductoDAO | None = None) -> None:
        self._mesa_dao, self._horario_dao, self._reserva_dao, self._ahora = mesa_dao, horario_dao, reserva_dao, ahora
        self._producto_dao = producto_dao
        self._pedido: list[tuple[int, int]] = []
        self._usuario_id: int | None = None
        self._restaurante_id: int | None = None
        self._fecha: date | str | None = None
        self._hora: str | None = None
        self._personas: int | None = None
        self._mesa_id: int | None = None
        self._notas = ""
    ...
    def para_cliente(self, usuario_id: int) -> "ReservaBuilder":
        self._usuario_id = usuario_id
        return self
    ...
    def con_items(self, items: list[tuple[int, int]] | None) -> "ReservaBuilder":
        """Pre-pedido: lista de (producto_id, cantidad). Los precios se leen de la BD en `build()`."""
        self._pedido = list(items or [])
        return self
```

Figura 27. Código del Builder (1/2): estado acumulado y pasos de la interfaz fluida (`reserva_builder.py`).

```python
    def build(self) -> Reserva:
        faltan = [n for n, v in (("cliente", self._usuario_id), ("restaurante", self._restaurante_id),
                                 ("fecha", self._fecha), ("hora", self._hora),
                                 ("número de personas", self._personas), ("mesa", self._mesa_id)) if v in (None, "")]
        if faltan:
            raise ReglaNegocioError("Falta indicar: " + ", ".join(faltan) + ".")
    ...
        # Regla 3: sin solapamientos (las canceladas se ignoran en el DAO)
        for existente in self._reserva_dao.listar_activas_por_mesa_y_fecha(mesa.id, fecha.isoformat()):
            if hay_solape(existente.hora, hora):
                raise ReglaNegocioError("La mesa ya está reservada en ese horario.")

        # Pre-pedido: productos del mismo restaurante, disponibles; el precio sale de la BD, nunca del navegador
        items = self._construir_items()

        return Reserva(id=None, usuario_id=self._usuario_id, restaurante_id=self._restaurante_id,
                       mesa_id=mesa.id, fecha=fecha.isoformat(), hora=hora, num_personas=self._personas,
                       estado="PENDIENTE", notas=self._notas,
                       creado_en=self._ahora().isoformat(timespec="seconds"), items=items)
```

Figura 28. Código del Builder (2/2): inicio de `build()` y la regla de no solapamiento.

Pruebas. `test_builder_falla_sin_mesa`, `test_builder_interfaz_fluida_construye_reserva`, `test_builder_valida_en_build`, `test_director_reserva_rapida` y, para las reglas, las 17 de `tests/test_reglas_negocio.py` (por ejemplo `test_rechaza_solapamiento` y `test_reservas_contiguas_no_se_solapan`).

4.2 Patrones estructurales

4.2.1 Facade: `ReservaFacade`

Problema. Reservar involucra restaurantes, mesas, horarios, usuarios, carta, carrito y reservas. Si el controlador los llamara uno por uno, conocería toda la capa de negocio.

Solución. `ReservaFacade` orquesta esos servicios tras operaciones simples (`buscar_disponibilidad`, `reservar`, `cancelar`, `mis_reservas`, `reservar_con_pre_pedido`, `cobro_pre_pedido`). Además decide detalles que el controlador no debe conocer, como asignar la mesa libre más ajustada a la capacidad cuando el cliente no elige una, y adjunta a cada reserva los datos que la vista necesita (restaurante, mesa, cliente y pre-pedido) con un número constante de consultas. La fachada recibe el servicio de reservas como `IReservaService`; en producción es el Proxy.

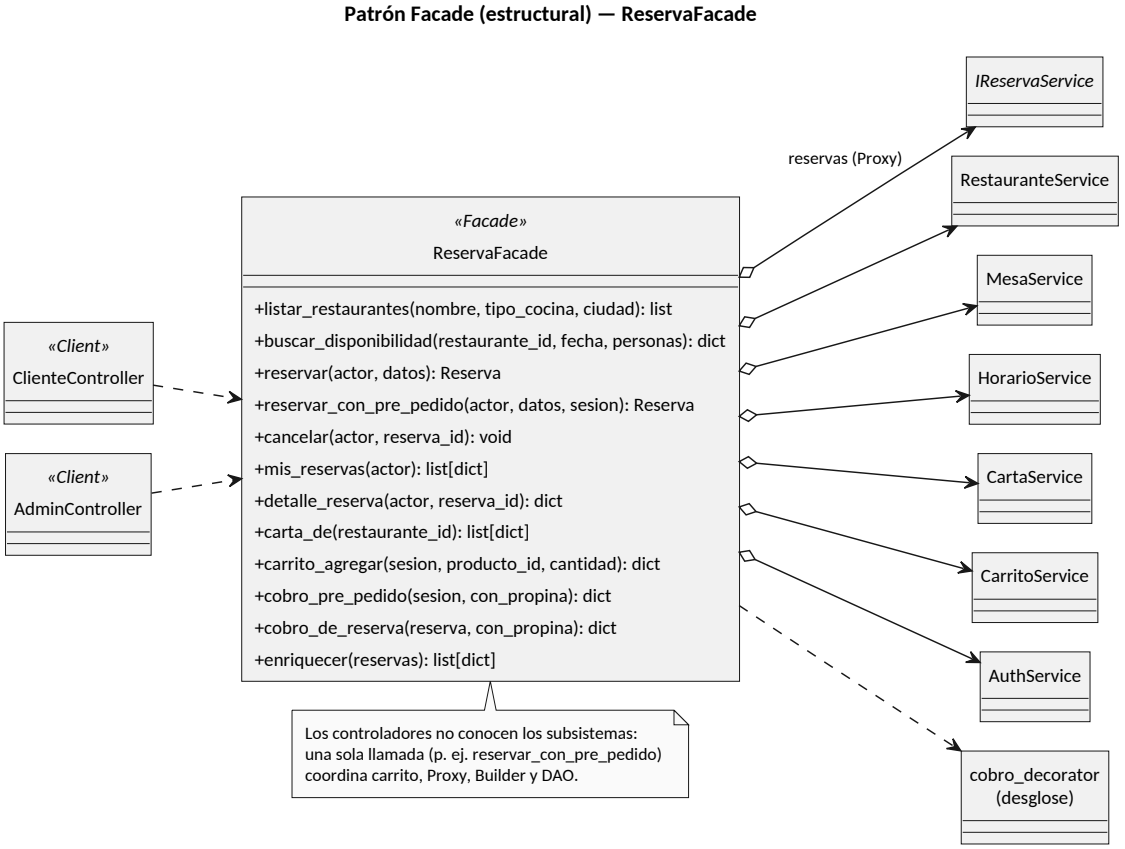


Figura 29. Facade: `ReservaFacade` y los servicios que orquesta. Elaboración propia a partir del código.

```python
class ReservaFacade:
    """PATRÓN ESTRUCTURAL · Facade (rol: Facade).

    Orquesta `RestauranteService`, `MesaService`, `HorarioService` y el servicio de reservas
    (recibido como `IReservaService`: en producción es el `ReservaServiceProxy`, nunca el real)
    detrás de cuatro operaciones simples: buscar_disponibilidad, reservar, cancelar y mis_reservas.
    """

    def __init__(self, reservas: IReservaService, restaurantes: RestauranteService,
                 mesas: MesaService, horarios: HorarioService, usuarios: AuthService,
                 carta: CartaService, carrito: CarritoService) -> None:
        self._reservas, self._restaurantes = reservas, restaurantes
        self._mesas, self._horarios, self._usuarios, self._carta = mesas, horarios, usuarios, carta
        self._carrito = carrito

    ...

    def reservar(self, actor: Usuario, datos: dict) -> Reserva:
        """Crea la reserva; si no se eligió mesa, asigna la libre más ajustada a la capacidad."""
        datos = dict(datos)
        if not datos.get("mesa_id"):
            datos["mesa_id"] = self._mesa_automatica(datos)
        return self._reservas.crear_reserva(actor, datos)

    ...

    def reservar_con_pre_pedido(self, actor: Usuario, datos: dict, sesion) -> Reserva:
        """Reserva usando el carrito de la sesión (debe ser del mismo restaurante); lo vacía al confirmar."""
        datos = dict(datos)
        datos["items"] = self._carrito.items_para_reserva(sesion, int(datos["restaurante_id"]))
        reserva = self.reservar(actor, datos)
        self._carrito.vaciar(sesion)
        return reserva
```

Figura 30. Código del Facade (`reserva_facade.py`): constructor, `reservar` y `reservar_con_pre_pedido`.

Pruebas. `test_facade_usa_el_proxy_y_no_el_servicio_real`, `test_facade_mis_reservas_y_cancelar`, `test_fachada_pre_pedido_vacia_el_carrito` y `test_facade_disponibilidad_y_reserva_con_mesa_automatica`.

4.2.2 Proxy de protección: `ReservaServiceProxy`

Problema. Las reglas de autorización (el cliente solo toca lo suyo, el administrador solo su restaurante) no deben mezclarse con la lógica de reservas ni repetirse en cada controlador. **Solución.** `ReservaServiceProxy` implementa la misma interfaz `IReservaService` que el servicio real (*RealSubject*), comprueba rol y propiedad del recurso y registra cada operación como `PERMITIDO` o `DENEGADO` antes de delegar. Si no hay permiso lanza `NoAutorizadoError` y el servicio real ni se entera. Como el resto del sistema solo ve la interfaz, el Proxy puede quitarse o sustituirse sin tocar a los clientes.

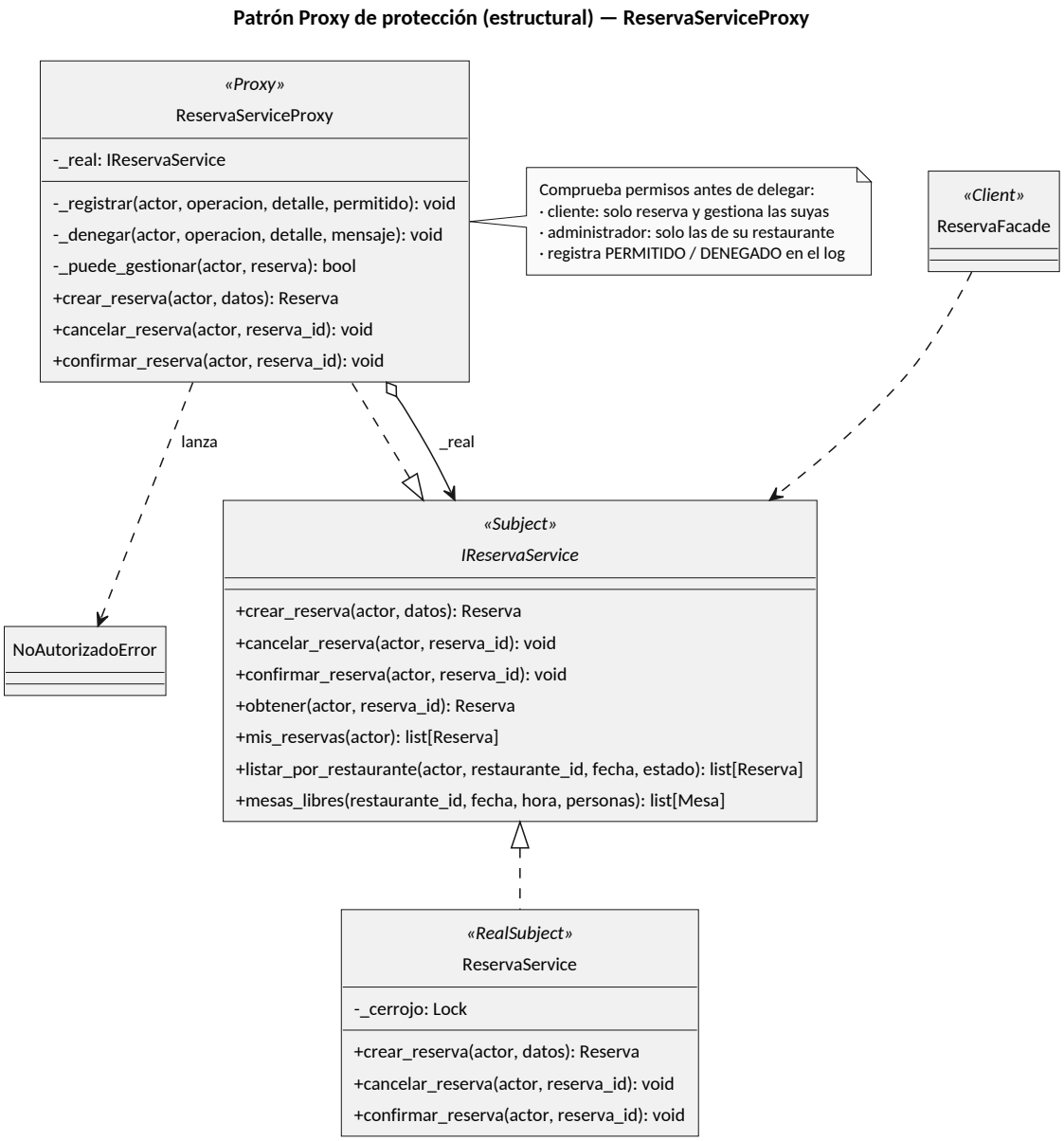


Figura 31. Proxy de protección: interfaz común, servicio real y proxy. Elaboración propia a partir del código.

```python
class ReservaServiceProxy(IReservaService):
    """PATRÓN ESTRUCTURAL · Proxy de protección (rol: Proxy).

    Implementa la misma interfaz `IReservaService` que el servicio real (RealSubject),
    comprueba rol y propiedad del recurso (regla 4) y registra cada operación antes de delegar.
    """

    def __init__(self, real: IReservaService, restaurantes: RestauranteService) -> None:
        self._real, self._restaurantes = real, restaurantes
    …
    def _registrar(self, actor: Usuario | None, operacion: str, detalle: str = "", permitido: bool = True) -> None:
        quien = f"{actor.rol}#{actor.id}" if actor else "anonimo"
        nivel = logging.INFO if permitido else logging.WARNING
        log.log(nivel, "%s %s %s %s", "PERMITIDO" if permitido else "DENEGADO", quien, operacion, detalle)
    …
    def crear_reserva(self, actor: Usuario, datos: dict) -> Reserva:
        if "reservar" not in actor.permisos():
            self._denegar(actor, "crear_reserva", str(datos.get("restaurante_id")),
                          "Solo los clientes pueden crear reservas.")
        self._registrar(actor, "crear_reserva", f"restaurante={datos.get('restaurante_id')}")
        return self._real.crear_reserva(actor, datos)
    …
    def cancelar_reserva(self, actor: Usuario, reserva_id: int) -> None:
        reserva = self._real.obtener(actor, reserva_id)
        if not self._puede_gestionar(actor, reserva):
            self._denegar(actor, "cancelar_reserva", f"reserva={reserva_id}",
                          "No puedes cancelar esta reserva.")
        self._registrar(actor, "cancelar_reserva", f"reserva={reserva_id}")
        self._real.cancelar_reserva(actor, reserva_id)
```

Figura 32. Código del Proxy (`reserva_proxy.py`): registro, `crear_reserva` y `cancelar_reserva`.

```python
class ReservaService(IReservaService):
    """PATRÓN ESTRUCTURAL · Proxy (rol: RealSubject). Contiene la lógica; NO comprueba permisos
    (eso lo hace `ReservaServiceProxy`). Construye cada reserva con `ReservaBuilder`."""

    # Dos defensas contra la doble reserva: el cerrojo (hilos de un proceso) y, sobre todo, la transacción
    # BEGIN IMMEDIATE de `insertar_si_libre` (varios procesos o conexiones).
    _cerrojo = threading.Lock()
    …
    def crear_reserva(self, actor: Usuario, datos: dict) -> Reserva:
        constructor: ReservaBuilder = (self._nuevo_builder()
                                       .para_cliente(actor.id)
                                       .en_restaurante(datos.get("restaurante_id"))
                                       .el_dia(datos.get("fecha"))
                                       .a_las(datos.get("hora"))
                                       .para_personas(datos.get("personas"))
                                       .en_mesa(datos.get("mesa_id"))
                                       .con_notas(datos.get("notas", ""))
                                       .con_items(datos.get("items")))
        with self._cerrojo:
            reserva = constructor.build()
            for campo, valor in instantanea(reserva.items).items():      # cobro congelado al crear (auditable)
                setattr(reserva, campo, valor)
            nuevo_id = self._dao.insertar_si_libre(reserva, config.DURACION_RESERVA_MIN)
        if nuevo_id is None:                          # otra reserva ganó la mesa entre la validación y el guardado
            raise ReglaNegocioError("La mesa ya está reservada en ese horario.")
        reserva.id = nuevo_id
        return reserva
```

Figura 33. Servicio real (*RealSubject*, `reserva_service.py`): usa el Builder, congela el cobro y guarda con `insertar_si_libre`.

Pruebas. `test_proxy_rechaza_cancelar_reserva_ajena`, `test_proxy_rechaza_ver_reserva_ajena`, `test_proxy_admin_solo_gestiona_su_restaurante`, `test_proxy_cliente_no_confirma_y_admin_no_reserva` y `test_proxy_registra_operaciones`.

```python
def test_proxy_rechaza_cancelar_reserva_ajena(cliente, otro_cliente):
    r = reserva_service.crear_reserva(cliente, datos())
    with pytest.raises(NoAutorizadoError):
        reserva_service.cancelar_reserva(otro_cliente, r.id)
    assert reserva_service.obtener(cliente, r.id).estado == "PENDIENTE"
```

Figura 34. Prueba del Proxy sobre una reserva ajena (`test_patrones.py`).

4.2.3 Decorator: `Cobro`

Problema. El total a pagar del pre-pedido parte de un subtotal al que se le aplican conceptos opcionales (descuento por pedido grande, impuesto al consumo, propina sugerida). Resolverlo con una clase por cada combinación genera una explosión de clases. **Solución.** `Cobro` (*Component*) define `monto_cop()` y `descripcion()`; `CobroBase` (*ConcreteComponent*) suma `cantidad` por `precio_unitario_cop`; `CobroDecorador` (*Decorator*) envuelve a otro `Cobro`; y `ConDescuento`, `ConImpuestoConsumo` y `ConPropinaSugerida` (*ConcreteDecorator*) aplican su porcentaje sobre el monto del componente envuelto. La función `armar_cadena` arma la cadena en un orden fijo: base, descuento (solo si el subtotal llega al mínimo), impuesto al consumo y propina (solo si el cliente la pide). Todo está en pesos enteros y `porcentaje_de` es la única regla de redondeo (mitad hacia arriba, aritmética entera), de modo que la suma de las líneas del desglose es exactamente el total. Al crear la reserva, `instantanea` toma el resultado de la cadena (sin propina) y lo congela en la propia reserva; `desglose_guardado` lo muestra después sin recalcular las tasas.

Las tasas (impuesto al consumo del 8 %, propina sugerida del 10 %, descuento del 10 % desde $150.000) son **supuestos de demostración** configurables en `app/config.py`; no se contrastaron con la normativa vigente. Con un subtotal de $170.000 el desglose es: descuento −$17.000, impuesto al consumo +$12.240 y total de $165.240; con propina (+$16.524), $181.764. El cálculo es solo del servidor y la elección de propina no se guarda.

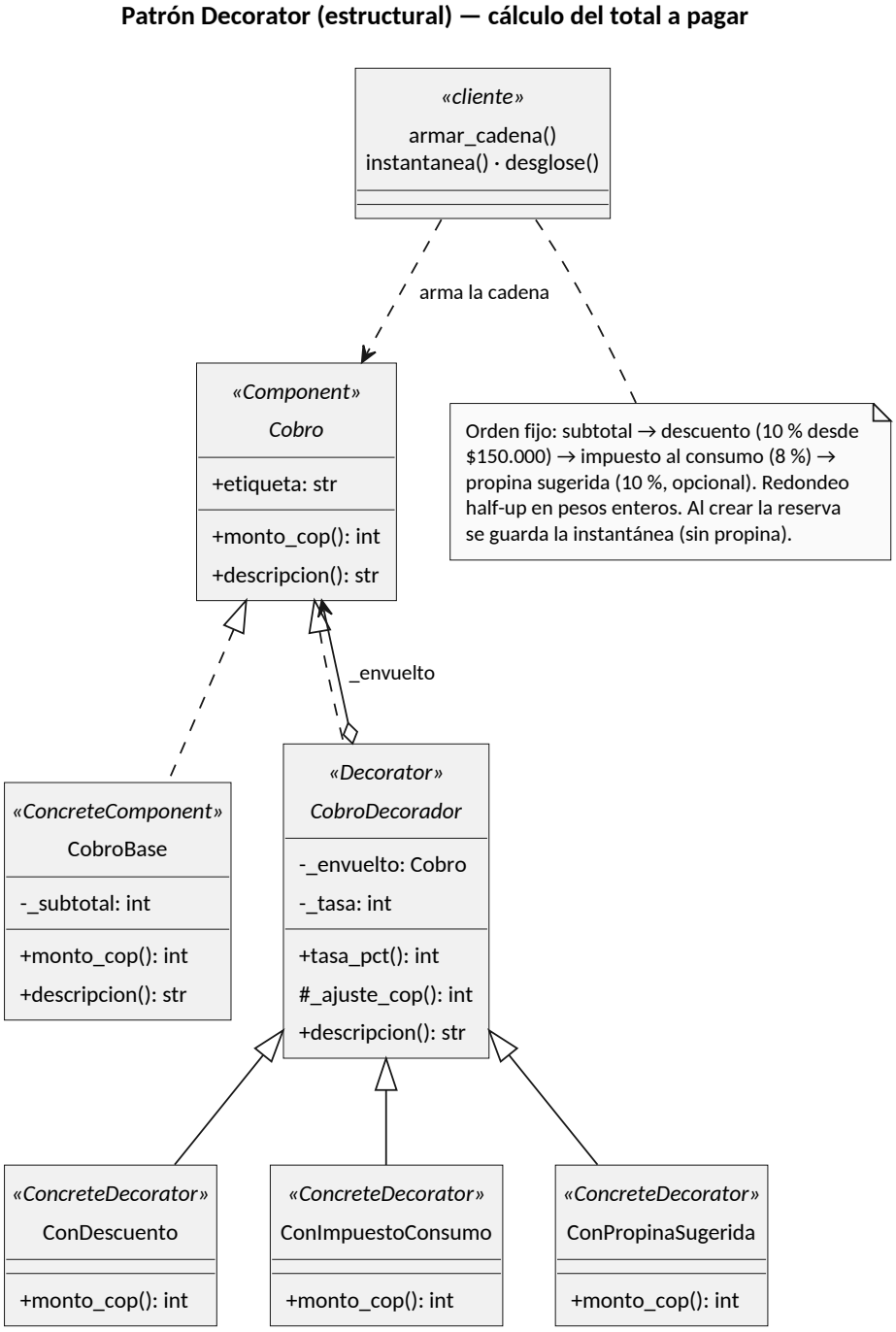


Figura 35. Decorator: cadena de cobro. Elaboración propia a partir del código.

```python
def porcentaje_de(monto_cop: int, tasa_pct: int) -> int:
    """Porcentaje entero de un monto entero, redondeado half-up al peso (regla única de redondeo)."""
    if monto_cop < 0 or tasa_pct < 0:
        raise ValueError("El monto y la tasa deben ser no negativos.")
    return (monto_cop * tasa_pct + 50) // 100


class Cobro(ABC):
    """PATRÓN ESTRUCTURAL · Decorator (rol: Component). Interfaz común de la base y de los decoradores."""

    etiqueta: str          # nombre corto de ESTE componente (para el desglose)

    @abstractmethod
    def monto_cop(self) -> int:
        """Monto acumulado en pesos enteros hasta este componente."""

    @abstractmethod
    def descripcion(self) -> str:
        """Descripción acumulada de la cadena, p. ej. 'Subtotal → Descuento (10 %) → Impuesto al consumo (8 %)'."""


class CobroDecorador(Cobro, ABC):
    """Decorator · rol: Decorator (clase base). Envuelve a otro `Cobro` y delega en él."""

    def __init__(self, envuelto: Cobro, tasa_pct: int) -> None:
        if not isinstance(tasa_pct, int) or not 0 <= tasa_pct <= 100:
            raise ValueError("La tasa debe ser un entero entre 0 y 100.")
        self._envuelto, self._tasa = envuelto, tasa_pct

    @property
    def tasa_pct(self) -> int:
        return self._tasa

    def _ajuste_cop(self) -> int:
        """Ajuste (siempre >= 0) = porcentaje del monto del componente envuelto, con la regla única de redondeo."""
        return porcentaje_de(self._envuelto.monto_cop(), self._tasa)

    def descripcion(self) -> str:
        return f"{self._envuelto.descripcion()} → {self.etiqueta}"
```

Figura 36. Código del Decorator (1/2): regla de redondeo, interfaz `Cobro` y clase base `CobroDecorador` (`cobro_decorator.py`).

```python
class ConDescuento(CobroDecorador):
    """Decorator · rol: ConcreteDecorator. Resta `tasa_pct` % del monto envuelto."""

    def __init__(self, envuelto: Cobro, tasa_pct: int | None = None) -> None:
        tasa_pct = config.DESCUENTO_PEDIDO_GRANDE_PCT if tasa_pct is None else tasa_pct     # se lee al construir (configurable)
        super().__init__(envuelto, tasa_pct)
        self.etiqueta = f"Descuento por pedido grande ({tasa_pct} %)"

    def monto_cop(self) -> int:
        return self._envuelto.monto_cop() - self._ajuste_cop()


def armar_cadena(items: Iterable[_Linea], con_propina: bool = False) -> list[Cobro]:
    """Cadena de producción, de la base al componente más externo (cada elemento envuelve al anterior).

    Orden fijo: Base → [Descuento si el subtotal llega al mínimo] → Impuesto al consumo → [Propina si se pide].
    """
    etapas: list[Cobro] = [CobroBase(list(items))]
    if etapas[0].monto_cop() >= config.DESCUENTO_PEDIDO_GRANDE_MINIMO_COP > 0:
        etapas.append(ConDescuento(etapas[-1]))
    etapas.append(ConImpuestoConsumo(etapas[-1]))
    if con_propina:
        etapas.append(ConPropinaSugerida(etapas[-1]))
    return etapas
```

Figura 37. Código del Decorator (2/2): un decorador concreto y la construcción de la cadena.

```python
def instantanea(items: Iterable[_Linea]) -> dict:
    """Valores a congelar en la reserva al crearla (con las tasas vigentes en ese momento, sin propina)."""
    etapas = armar_cadena(items, False)
    subtotal, total = etapas[0].monto_cop(), etapas[-1].monto_cop()
    descuento = next((e for e in etapas if isinstance(e, ConDescuento)), None)
    impuesto = next(e for e in etapas if isinstance(e, ConImpuestoConsumo))
    desc_cop = subtotal - descuento.monto_cop() if descuento else 0
    return {"subtotal_cop": subtotal,
            "descuento_pct": descuento.tasa_pct if descuento else 0, "descuento_cop": desc_cop,
            "impuesto_pct": impuesto.tasa_pct, "impuesto_cop": total - (subtotal - desc_cop),
            "total_pagar_cop": total}
```

Figura 38. Instantánea del cobro que se guarda con la reserva (`cobro_decorator.py`).

Pruebas. La regla de redondeo (`test_porcentaje_de_redondea_half_up_con_enteros`), la composición apilada con montos exactos (`test_composicion_apilada_con_montos_exactos`), que la suma de las líneas sea el total (`test_desglose_suma_exactamente_el_total`), el mínimo del descuento (`test_descuento_solo_desde_el_minimo`), la propina opcional (`test_propina_es_opcional`), el caso de $162.000 (`test_desglose_de_ejemplo`) y la instantánea guardada (`test_la_reserva_guarda_la_instantanea_del_cobro`).

```python
def test_desglose_de_ejemplo():
    d = desglose(items((9, 18000)), con_propina=True)       # 162000
    assert [(l["etiqueta"], l["monto_cop"]) for l in d["lineas"]] == [
        ("Subtotal del pre-pedido", 162000), ("Descuento por pedido grande (10 %)", -16200),
        ("Impuesto al consumo (8 %)", 11664), ("Propina sugerida (10 %)", 15746)]
    assert d["total_cop"] == 173210 and d["propina_sugerida_cop"] == 15746 and d["incluye_propina"]
```

Figura 39. Prueba del desglose de ejemplo (`test_fase4.py`).

5 Implementación de la presentación y la seguridad

5.1 Controladores

Los controladores son delgados: validan la entrada, llaman a la fachada o al servicio y responden. El de reserva (Figura 40) pasa por la guarda de CSRF y por la de rol, delega todo en `reservar_con_pre_pedido` y, si la capa de negocio lanza una regla violada, vuelve al formulario con el mensaje y los datos ya escritos. El inicio de sesión (Figura 41) consulta el limitador, regenera la sesión y deja que el producto de la fábrica decida el destino.

```python
@router.post("/reservar", dependencies=[Depends(verificar_csrf)])
def crear_reserva(request: Request, restaurante_id: str = Form(""), fecha: str = Form(""), hora: str = Form(""),
                  personas: str = Form(""), mesa_id: str = Form(""), notas: str = Form(""),
                  propina: str = Form(""), usuario: Usuario = Depends(requiere_cliente)):
    try:
        reserva = fachada.reservar_con_pre_pedido(usuario, {
            "restaurante_id": entero(restaurante_id, "restaurante"), "fecha": fecha, "hora": hora,
            "personas": entero(personas, "personas"), "mesa_id": entero(mesa_id, "mesa") if mesa_id else None,
            "notas": notas}, request.session)
    except (ReglaNegocioError, NoEncontradoError) as error:
        avisar(request, str(error), "error")
        consulta = urlencode({"restaurante_id": restaurante_id, "fecha": fecha, "hora": hora,
                              "personas": personas, "mesa_id": mesa_id, "notas": notas[:300], "propina": propina})
        return redirigir(f"/reservar?{consulta}")
    extra = ""
    if reserva.items:
        cobro = fachada.cobro_de_reserva(reserva, bool(propina))     # calculado en el servidor con la cadena de decoradores
        extra = (f" Pre-pedido: {len(reserva.items)} plato(s). Total a pagar estimado: {_cop(cobro['total_cop'])}"
                 f"{' (propina incluida)' if propina else ''}.")
    avisar(request, f"¡Reserva #{reserva.id} creada! Queda PENDIENTE hasta que el restaurante la confirme.{extra}")
    return redirigir("/mis-reservas")
```

Figura 40. Controlador de `POST /reservar` (`cliente_controller.py`).

```python
@router.post("/login", dependencies=[Depends(verificar_csrf)])
def iniciar_sesion(request: Request, email: str = Form(""), password: str = Form("")):
    ip = request.client.host if request.client else "desconocida"
    clave_cuenta = (ip, (email or "").strip().lower())
    espera = max(limitador_cuenta.segundos_bloqueo(clave_cuenta), limitador_ip.segundos_bloqueo(ip))
    if espera:
        respuesta = renderizar(request, "login.html", estado=429, email=email,
                               error=f"Demasiados intentos fallidos. Inténtalo de nuevo en {-(-espera // 60)} min.")
        respuesta.headers["Retry-After"] = str(espera)
        return respuesta
    try:
        usuario = auth_service.autenticar(email, password)
    except ReglaNegocioError as error:
        limitador_cuenta.registrar_fallo(clave_cuenta)
        limitador_ip.registrar_fallo(ip)
        return renderizar(request, "login.html", estado=401, error=str(error), email=email)
    limitador_cuenta.limpiar(clave_cuenta)
    request.session.clear()                        # evita fijación de sesión
    request.session["usuario_id"] = usuario.id
    return redirigir(usuario.ruta_inicio())        # el rol decide el destino (Factory Method: ruta_inicio)
```

Figura 41. Controlador de inicio de sesión (`auth_controller.py`).

5.2 Seguridad

```python
def hashear_password(password: str) -> str:
    sal = secrets.token_bytes(16)
    h = hashlib.scrypt(password.encode(), salt=sal, n=_N, r=_R, p=_P, dklen=32)
    return f"scrypt${_N}${_R}${_P}${sal.hex()}${h.hex()}"


def verificar_password(password: str, almacenado: str) -> bool:
    try:
        _, n, r, p, sal, esperado = almacenado.split("$")
        h = hashlib.scrypt(password.encode(), salt=bytes.fromhex(sal), n=int(n), r=int(r), p=int(p), dklen=32)
        return hmac.compare_digest(h.hex(), esperado)
```

Figura 42. Hash y verificación de contraseñas con scrypt (`seguridad.py`).

```python
def token_csrf(request: Request) -> str:
    if "csrf" not in request.session:
        request.session["csrf"] = secrets.token_urlsafe(32)
    return request.session["csrf"]


async def verificar_csrf(request: Request) -> None:
    """Protección CSRF por token de sesión: campo `csrf_token` del formulario o cabecera X-CSRF-Token."""
    esperado = request.session.get("csrf")
    recibido = request.headers.get("x-csrf-token")
    if recibido is None:
        formulario = await request.form()
        recibido = formulario.get("csrf_token")
    if not esperado or not isinstance(recibido, str) or not secrets.compare_digest(esperado, recibido):
        raise HTTPException(status_code=403, detail="Token CSRF inválido. Recarga la página e inténtalo de nuevo.")
```

Figura 43. Token CSRF por sesión (`comun.py`).

5.3 Esquema de la reserva en SQL

La reserva y su pre-pedido, con la instantánea del cobro y sus restricciones, se definen así (Figura 44).

```sql
CREATE TABLE IF NOT EXISTS reserva (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario_id     INTEGER NOT NULL REFERENCES usuario(id),
    restaurante_id INTEGER NOT NULL REFERENCES restaurante(id),
    mesa_id        INTEGER NOT NULL REFERENCES mesa(id),
    fecha          TEXT NOT NULL,
    hora           TEXT NOT NULL,
    num_personas   INTEGER NOT NULL CHECK (num_personas > 0),
    estado         TEXT NOT NULL DEFAULT 'PENDIENTE' CHECK (estado IN ('PENDIENTE', 'CONFIRMADA', 'CANCELADA')),
    notas          TEXT NOT NULL DEFAULT '',
    creado_en      TEXT NOT NULL,
    -- Instantánea del cobro al crear la reserva (COP enteros): un cambio posterior de tasas en config.py
    -- no altera lo que se le cobró a cada cliente. La propina es voluntaria y no se guarda.
    subtotal_cop    INTEGER NOT NULL DEFAULT 0 CHECK (subtotal_cop >= 0),
    descuento_pct   INTEGER NOT NULL DEFAULT 0 CHECK (descuento_pct BETWEEN 0 AND 100),
    descuento_cop   INTEGER NOT NULL DEFAULT 0 CHECK (descuento_cop >= 0),
    impuesto_pct    INTEGER NOT NULL DEFAULT 0 CHECK (impuesto_pct BETWEEN 0 AND 100),
    impuesto_cop    INTEGER NOT NULL DEFAULT 0 CHECK (impuesto_cop >= 0),
    total_pagar_cop INTEGER NOT NULL DEFAULT 0 CHECK (total_pagar_cop = subtotal_cop - descuento_cop + impuesto_cop)
);


CREATE TABLE IF NOT EXISTS reserva_item (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    reserva_id          INTEGER NOT NULL REFERENCES reserva(id) ON DELETE CASCADE,
    producto_id         INTEGER NOT NULL REFERENCES producto(id),
    cantidad            INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unitario_cop INTEGER NOT NULL CHECK (precio_unitario_cop >= 0)
);
```

Figura 44. Tablas `reserva` y `reserva_item` (`schema.sql`).

5.4 Acceso a datos

```python
class BaseDAO(ABC, Generic[T]):
    """DAO genérico: CRUD común. Las subclases definen tabla, columnas y el mapeo fila→modelo.

    Los nombres de tabla y columnas son constantes de clase (nunca datos del usuario);
    todos los valores viajan como parámetros `?`.
    """

    tabla: str
    columnas: tuple[str, ...]          # columnas sin `id`

    @abstractmethod
    def _desde_fila(self, fila: sqlite3.Row) -> T: ...

    def _valores(self, entidad: T) -> tuple[Any, ...]:
        return tuple(getattr(entidad, c) for c in self.columnas)

    def obtener_por_id(self, id_: int) -> T | None:
        with ConexionBD().sesion() as con:
            fila = con.execute(f"SELECT * FROM {self.tabla} WHERE id = ?", (id_,)).fetchone()
        return self._desde_fila(fila) if fila else None
```

Figura 45. `BaseDAO`: CRUD genérico (`__init__.py`).

```python
class ReservaDAO(BaseDAO[Reserva]):
    tabla = "reserva"
    columnas = ("usuario_id", "restaurante_id", "mesa_id", "fecha", "hora",
                "num_personas", "estado", "notas", "creado_en",
                "subtotal_cop", "descuento_pct", "descuento_cop", "impuesto_pct", "impuesto_cop", "total_pagar_cop")

    def _desde_fila(self, fila: sqlite3.Row) -> Reserva:
        return Reserva(**dict(fila))

    def insertar_si_libre(self, reserva: Reserva, duracion_min: int) -> int | None:
        """Comprueba el solape y guarda la reserva con su pre-pedido en UNA transacción exclusiva (BEGIN IMMEDIATE).

        Devuelve el id nuevo, o None si la mesa ya está ocupada en esa franja. Como el bloqueo de escritura se toma
        antes de comprobar, dos procesos (o dos conexiones) no pueden reservar la misma mesa a la vez.
        Las horas son «HH:MM»: se pasan a minutos en SQL y dos reservas chocan si distan menos de `duracion_min`."""
        h, m = reserva.hora.split(":")[:2]
        minutos = int(h) * 60 + int(m)
        marcas = ", ".join("?" for _ in self.columnas)
        with ConexionBD().sesion_inmediata() as con:
            choque = con.execute(
                "SELECT 1 FROM reserva WHERE mesa_id = ? AND fecha = ? AND estado <> 'CANCELADA' AND "
                "ABS((CAST(substr(hora, 1, 2) AS INTEGER) * 60 + CAST(substr(hora, 4, 2) AS INTEGER)) - ?) < ? LIMIT 1",
                (reserva.mesa_id, reserva.fecha, minutos, duracion_min)).fetchone()
            if choque:

                cur = con.execute(
                    f"INSERT INTO reserva ({', '.join(self.columnas)}) VALUES ({marcas})", self._valores(reserva))
                nuevo_id = int(cur.lastrowid)
                con.executemany(
                    "INSERT INTO reserva_item (reserva_id, producto_id, cantidad, precio_unitario_cop) VALUES (?,?,?,?)",
                    [(nuevo_id, i.producto_id, i.cantidad, i.precio_unitario_cop) for i in reserva.items])
            return nuevo_id

    def cambiar_estado(self, reserva_id: int, estado: str) -> None:
        """Al cancelar, el pre-pedido de la reserva se elimina en la misma transacción."""
        with ConexionBD().sesion() as con:
            con.execute("UPDATE reserva SET estado = ? WHERE id = ?", (estado, reserva_id))
            if estado == "CANCELADA":
                con.execute("DELETE FROM reserva_item WHERE reserva_id = ?", (reserva_id,))
```

Figura 46. `ReservaDAO`: columnas, inserción atómica con comprobación de solapamiento (`insertar_si_libre`) y cancelación.

6 Robustez y calidad

Además de cumplir las reglas de negocio, se reforzaron cuatro aspectos que una aplicación de reservas real necesita. Cada uno tiene su evidencia en el código y en las pruebas.

6.1 Doble reserva bajo concurrencia

Comprobar que la mesa está libre y después insertar son dos pasos; si dos solicitudes los intercalan, ambas pueden ganar. La primera defensa es un cerrojo del servicio, que solo protege dentro de un proceso. La definitiva está en la base: `insertar_si_libre` abre la transacción con `BEGIN IMMEDIATE` (Figura 47), que toma el bloqueo de escritura de SQLite antes de comprobar el solapamiento, de modo que la comprobación y la inserción son atómicas para cualquier conexión. La base funciona en modo WAL, de modo que las lecturas no esperan a la escritura. La prueba de la Figura 48 lanza 8 hilos, cada uno con su propia conexión y sin pasar por el cerrojo, que intentan reservar la misma mesa y hora a la vez: exactamente una reserva se guarda y las otras siete reciben `None`.

```
66      def sesion_inmediata(self) -> Iterator[sqlite3.Connection]:
67          """Como `sesion`, pero abre la transacción con BEGIN IMMEDIATE: toma el bloqueo de escritura desde el
68          principio, así que «comprobar e insertar» es atómico incluso entre varios procesos (no solo hilos)."""
69          conexion = self.conectar()
70          try:
71              conexion.execute("BEGIN IMMEDIATE")
72              yield conexion
73              conexion.commit()
74          except Exception:
75              conexion.rollback()
76              raise
77          finally:
78              conexion.close()
```

Figura 47. Transacción exclusiva de `ConexionBD` (`conexion.py`).

```
203 def test_dos_conexiones_a_la_vez_solo_una_gana_la_mesa(bd):
204     """Sin pasar por el cerrojo del servicio: la transacción BEGIN IMMEDIATE decide quién se queda con la mesa."""
205     from app.dependencias import reserva_dao
206     hilos, resultados, barrera = 8, [], threading.Barrier(8)
207
208     def intentar(i):
209         barrera.wait()
210         resultados.append(reserva_dao.insertar_si_libre(_reserva(3, "19:00", usuario=1), 120))
211
212     ts = [threading.Thread(target=intentar, args=(i,)) for i in range(hilos)]
213     [t.start() for t in ts]
214     [t.join() for t in ts]
215     assert sum(r is not None for r in resultados) == 1 and sum(r is None for r in resultados) == hilos - 1
```

Figura 48. Prueba de concurrencia: ocho conexiones compiten por la misma mesa (`test_fase5.py`).

6.2 Límite de intentos de inicio de sesión

El limitador (Figura 49) cuenta los fallos por clave dentro de una ventana de tiempo y bloquea la clave al llegar al umbral. Se usan dos instancias: una por IP y correo (5 fallos) y

otra por IP (20 fallos), ambas con ventana y bloqueo de 5 minutos; un inicio correcto borra los fallos de esa clave. El controlador responde 429 con la cabecera `Retry-After`. Es un mecanismo en memoria de un solo proceso: no se comparte entre procesos ni sobrevive a un reinicio (Sección 10).

```
class LimitadorIntentos:
    """Bloquea una clave (p. ej. IP + correo) tras `max_fallos` fallos dentro de `ventana_s` segundos.

    El bloqueo dura `bloqueo_s`. Un inicio de sesión correcto borra los fallos de esa clave.
    Es una defensa sencilla contra la fuerza bruta: no se comparte entre procesos ni sobrevive a un reinicio.
    """

    def __init__(self, max_fallos: int = 5, ventana_s: int = 300, bloqueo_s: int = 300,
                 reloj: Callable[[], float] = time.monotonic) -> None:
        self._max, self._ventana, self._bloqueo, self._reloj = max_fallos, ventana_s, bloqueo_s, reloj
        self._fallos: dict[Hashable, list[float]] = {}
        self._bloqueados: dict[Hashable, float] = {}
        self._cerrojo = threading.Lock()
…
    def registrar_fallo(self, clave: Hashable) -> None:
        with self._cerrojo:
            ahora = self._reloj()
            recientes = [t for t in self._fallos.get(clave, []) if ahora - t < self._ventana] + [ahora]
            self._fallos[clave] = recientes
            if len(recientes) >= self._max:
                self._bloqueados[clave] = ahora + self._bloqueo
```

Figura 49. Límite de intentos fallidos (`limitador.py`).

6.3 Consultas por lote en las listas de reservas

Al mostrar una lista de reservas, el código original consultaba el pre-pedido de cada reserva y el nombre de cada plato por separado (el problema de las “N+1 consultas”). `CartaService.detalles_de` ahora trae los pre-pedidos de todas las reservas en una sola consulta con unión. Se midió el método `ReservaFacade.enriquecer` sobre todas las reservas del restaurante 1 con el seed de cada versión y sobre una base recién creada (mediana de 15 ejecuciones en el equipo de desarrollo, Linux, Python 3.13): la versión anterior ejecutaba 208 sentencias SQL con 59 reservas y tardaba 36,3 ms; la actual ejecuta 11 sentencias con 61 reservas y tarda 5,6 ms. La prueba `test_enriquecer_usa_un_numero_constante_de_consultas` fija el límite para que no regrese el problema. Son mediciones locales de una base pequeña: sirven para comparar las dos versiones, no como cifra de rendimiento en producción.

6.4 Cabeceras de seguridad y configuración

Toda respuesta lleva la política de contenido (CSP), `X-Frame-Options: DENY`, `X-Content-Type-Options`, `Referrer-Policy` y `Permissions-Policy`; los archivos estáticos se sirven con `Cache-Control`. Con `HTTPS_ONLY=1` la cookie de sesión pasa a `Secure` y se envía HSTS (`test_hsts_y_cookie_segura_solo_con_https_only`). Como la CSP prohíbe el JavaScript en línea, todo el código del navegador está en archivos propios. El script de capturas verifica en un navegador real que ningún recurso propio sea bloqueado por la política.

7 Pruebas automáticas

El proyecto tiene 154 pruebas con `pytest`, repartidas como muestra la Tabla 12. Las de la capa web recorren el flujo real (login, reserva, doble reserva, confirmación y permisos) con el cliente de pruebas de FastAPI.

Archivo	Pruebas	Qué cubre
`test_patrones.py`	19	Singleton, Factory Method, Builder, Facade y Proxy.
`test_reglas_negocio.py`	17	Reglas de reserva: horario, aforo, solapamiento, estados.
`test_fase2.py`	28	Carta, carrito, pre-pedido, registro, panel y administración de la carta.
`test_fase4.py`	32	Decorator y total a pagar.
`test_fase5.py`	27	Concurrencia, límite de login, cabeceras, comprobante y calendario, CSV, consultas por lote.
`test_cobro_guardado.py`	9	Instantánea del cobro y migración.
`test_cartas_demo.py`	5	Formato de las cartas, fotos y créditos.
`test_web.py`	17	Capa web de punta a punta.
Total	**154**	

Tabla 12. Pruebas automáticas por archivo (cifras calculadas con `pytest -collect-only`). Elaboración propia.

Las figuras siguientes son la salida real de `pytest` ejecutada para este documento (Linux, Python 3.13, pytest 9.1.1), con la opción `-p no:warnings`; solo se reemplazaron las rutas locales del equipo por marcadores genéricos.

```
$ pytest -q -p no:warnings
........................................................................ [ 46%]
........................................................................ [ 93%]
..........                                                             [100%]
154 passed in 10.97s
```

Figura 50. Ejecución completa de la batería de pruebas: 154 pruebas superadas.

```
$ pytest tests/test_patrones.py -v -p no:warnings
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.6.0 -- /tmp/claude-0/-home-claude/80842ca2-cce6-5273-980d-cd9eb1cff35d/scratchpad/venv_reservas/bin/python
cachedir: .pytest_cache
rootdir: /ruta/reservas
plugins: anyio-4.15.1
collecting ... collected 19 items

tests/test_patrones.py::test_singleton_misma_instancia PASSED            [  5%]
tests/test_patrones.py::test_singleton_seguro_con_hilos PASSED           [ 10%]
tests/test_patrones.py::test_singleton_activa_claves_foraneas PASSED     [ 15%]
tests/test_patrones.py::test_factory_crea_subclase_segun_rol PASSED      [ 21%]
tests/test_patrones.py::test_factory_rechaza_rol_desconocido PASSED      [ 26%]
tests/test_patrones.py::test_dao_y_login_usan_la_factory PASSED          [ 31%]
tests/test_patrones.py::test_builder_falla_sin_mesa PASSED               [ 36%]
tests/test_patrones.py::test_builder_interfaz_fluida_construye_reserva PASSED [ 42%]
tests/test_patrones.py::test_builder_valida_en_build PASSED              [ 47%]
tests/test_patrones.py::test_director_reserva_rapida PASSED              [ 52%]
tests/test_patrones.py::test_el_servicio_construye_por_el_builder PASSED [ 57%]
tests/test_patrones.py::test_facade_usa_el_proxy_y_no_el_servicio_real PASSED [ 63%]
tests/test_patrones.py::test_facade_disponibilidad_y_reserva_con_mesa_automatica PASSED [ 68%]
tests/test_patrones.py::test_facade_mis_reservas_y_cancelar PASSED       [ 73%]
tests/test_patrones.py::test_proxy_rechaza_cancelar_reserva_ajena PASSED [ 78%]
tests/test_patrones.py::test_proxy_rechaza_ver_reserva_ajena PASSED      [ 84%]
tests/test_patrones.py::test_proxy_admin_solo_gestiona_su_restaurante PASSED [ 89%]
tests/test_patrones.py::test_proxy_cliente_no_confirma_y_admin_no_reserva PASSED [ 94%]
tests/test_patrones.py::test_proxy_registra_operaciones PASSED           [100%]

============================== 19 passed in 1.36s ==============================
```

Figura 51. Salida detallada de `test_patrones.py` (Singleton, Factory Method, Builder, Facade y Proxy).

```
$ pytest tests/test_fase4.py -v -p no:warnings
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.6.0 -- /tmp/claude-0/-home-claude/80842ca2-cce6-5273-980d-cd9eb1cff35d/scratchpad/venv_reservas/bin/python
cachedir: .pytest_cache
rootdir: /ruta/reservas
plugins: anyio-4.15.1
collecting ... collected 32 items

tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[100-8-8] PASSED [  3%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[0-8-0] PASSED [  6%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[4-10-0] PASSED [  9%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[5-10-1] PASSED [ 12%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[15-10-2] PASSED [ 15%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[25-10-3] PASSED [ 18%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[1001-8-80] PASSED [ 21%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[1006-8-80] PASSED [ 25%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[1007-8-81] PASSED [ 28%]
tests/test_fase4.py::test_porcentaje_de_redondea_half_up_con_enteros[153000-8-12240] PASSED [ 31%]
tests/test_fase4.py::test_porcentaje_de_rechaza_negativos PASSED         [ 34%]
tests/test_fase4.py::test_cobro_base_suma_precios_congelados PASSED      [ 37%]
tests/test_fase4.py::test_pre_pedido_vacio PASSED                        [ 40%]
tests/test_fase4.py::test_con_impuesto_consumo PASSED                    [ 43%]
tests/test_fase4.py::test_con_propina_sugerida PASSED                    [ 46%]
tests/test_fase4.py::test_con_descuento PASSED                           [ 50%]
tests/test_fase4.py::test_tasa_propia_y_validacion PASSED                [ 53%]
tests/test_fase4.py::test_las_tasas_son_configurables PASSED             [ 56%]
tests/test_fase4.py::test_composicion_apilada_con_montos_exactos PASSED  [ 59%]
tests/test_fase4.py::test_el_orden_no_importa_con_montos_redondos PASSED [ 62%]
tests/test_fase4.py::test_el_orden_solo_cambia_por_redondeo PASSED       [ 65%]
tests/test_fase4.py::test_descuento_solo_desde_el_minimo PASSED          [ 68%]
tests/test_fase4.py::test_propina_es_opcional PASSED                     [ 71%]
tests/test_fase4.py::test_desglose_suma_exactamente_el_total PASSED      [ 75%]
tests/test_fase4.py::test_desglose_de_ejemplo PASSED                     [ 78%]
tests/test_fase4.py::test_total_de_la_pantalla_de_confirmacion_coincide_con_la_cadena PASSED [ 81%]
tests/test_fase4.py::test_el_cliente_no_envia_montos_y_la_base_no_cambia PASSED [ 84%]
tests/test_fase4.py::test_detalle_de_reserva_y_admin_muestran_el_total_a_pagar PASSED [ 87%]
tests/test_fase4.py::test_mensaje_de_confirmacion_usa_la_cadena PASSED   [ 90%]
tests/test_fase4.py::test_reserva_sin_pre_pedido_no_muestra_cobro PASSED [ 93%]
tests/test_fase4.py::test_api_cobro_exige_cliente PASSED                 [ 96%]
tests/test_fase4.py::test_enriquecer_usa_precios_congelados_aunque_cambie_el_precio PASSED [100%]

============================== 32 passed in 1.39s ==============================
```

Figura 52. Salida detallada de `test_fase4.py` (Decorator y total a pagar).

```
$ pytest tests/test_reglas_negocio.py -v -p no:warnings
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.6.0 -- /tmp/claude-0/-home-claude/80842ca2-cce6-5273-980d-cd9eb1cff35d/scratchpad/venv_reservas/bin/python
cachedir: .pytest_cache
rootdir: /ruta/reservas
plugins: anyio-4.15.1
collecting ... collected 17 items

tests/test_reglas_negocio.py::test_reserva_valida_queda_pendiente PASSED [  5%]
tests/test_reglas_negocio.py::test_rechaza_fecha_pasada PASSED           [ 11%]
tests/test_reglas_negocio.py::test_rechaza_fuera_de_horario[08:00] PASSED [ 17%]
tests/test_reglas_negocio.py::test_rechaza_fuera_de_horario[11:30] PASSED [ 23%]
tests/test_reglas_negocio.py::test_rechaza_fuera_de_horario[21:30] PASSED [ 29%]
tests/test_reglas_negocio.py::test_rechaza_fuera_de_horario[22:30] PASSED [ 35%]
tests/test_reglas_negocio.py::test_acepta_limites_del_horario PASSED     [ 41%]
tests/test_reglas_negocio.py::test_rechaza_dia_cerrado PASSED            [ 47%]
tests/test_reglas_negocio.py::test_rechaza_capacidad_insuficiente PASSED [ 52%]
tests/test_reglas_negocio.py::test_rechaza_mesa_inactiva PASSED          [ 58%]
tests/test_reglas_negocio.py::test_rechaza_mesa_de_otro_restaurante PASSED [ 64%]
tests/test_reglas_negocio.py::test_rechaza_solapamiento PASSED           [ 70%]
tests/test_reglas_negocio.py::test_reservas_contiguas_no_se_solapan PASSED [ 76%]
tests/test_reglas_negocio.py::test_misma_hora_en_otra_mesa_si_es_posible PASSED [ 82%]
tests/test_reglas_negocio.py::test_cancelada_libera_la_mesa PASSED       [ 88%]
tests/test_reglas_negocio.py::test_estados_no_permiten_reabrir PASSED    [ 94%]
tests/test_reglas_negocio.py::test_confirmar_cambia_estado PASSED        [100%]

============================== 17 passed in 1.53s ==============================
```

Figura 53. Salida detallada de `test_reglas_negocio.py` (reglas de reserva).

```
$ pytest tests/test_fase5.py -v -p no:warnings
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.6.0 -- /tmp/claude-0/-home-claude/80842ca2-cce6-5273-980d-cd9eb1cff35d/scratchpad/venv_reservas/bin/python
cachedir: .pytest_cache
rootdir: /ruta/reservas
plugins: anyio-4.15.1
collecting ... collected 27 items

tests/test_fase5.py::test_codigo_y_fecha_larga PASSED                    [  3%]
tests/test_fase5.py::test_srcset_solo_si_existe_la_foto_pequena PASSED   [  7%]
tests/test_fase5.py::test_clasificar_reservas_orden_y_grupos PASSED      [ 11%]
tests/test_fase5.py::test_mis_reservas_filtros_y_codigo PASSED           [ 14%]
tests/test_fase5.py::test_comprobante_y_calendario_solo_para_el_dueno PASSED [ 18%]
tests/test_fase5.py::test_paginas_con_portadas_noscript_y_srcset PASSED  [ 22%]
tests/test_fase5.py::test_evento_ics_valido_y_escapado PASSED            [ 25%]
tests/test_fase5.py::test_doblar_lineas_largas_sin_cortar_caracteres PASSED [ 29%]
tests/test_fase5.py::test_ics_de_reserva_cancelada PASSED                [ 33%]
tests/test_fase5.py::test_estadisticas_por_estado_y_mapa_de_calor PASSED [ 37%]
tests/test_fase5.py::test_panel_muestra_dona_y_mapa_de_calor PASSED      [ 40%]
tests/test_fase5.py::test_exportar_csv PASSED                            [ 44%]
tests/test_fase5.py::test_csv_solo_del_restaurante_del_admin_y_solo_admin PASSED [ 48%]
tests/test_fase5.py::test_insertar_si_libre_respeta_la_franja PASSED     [ 51%]
tests/test_fase5.py::test_cancelar_libera_la_mesa_para_insertar_si_libre PASSED [ 55%]
tests/test_fase5.py::test_dos_conexiones_a_la_vez_solo_una_gana_la_mesa PASSED [ 59%]
tests/test_fase5.py::test_el_servicio_rechaza_si_pierde_la_carrera PASSED [ 62%]
tests/test_fase5.py::test_sqlite_en_modo_wal PASSED                      [ 66%]
tests/test_fase5.py::test_seed_reset_borra_tambien_los_archivos_wal PASSED [ 70%]
tests/test_fase5.py::test_limitador_bloquea_y_se_libera PASSED           [ 74%]
tests/test_fase5.py::test_limitador_ventana_y_limpiar PASSED             [ 77%]
tests/test_fase5.py::test_login_se_bloquea_tras_cinco_fallos PASSED      [ 81%]
tests/test_fase5.py::test_un_login_correcto_reinicia_el_contador PASSED  [ 85%]
tests/test_fase5.py::test_cabeceras_de_seguridad PASSED                  [ 88%]
tests/test_fase5.py::test_hsts_y_cookie_segura_solo_con_https_only PASSED [ 92%]
tests/test_fase5.py::test_cache_en_archivos_estaticos PASSED             [ 96%]
tests/test_fase5.py::test_enriquecer_usa_un_numero_constante_de_consultas PASSED [100%]

============================== 27 passed in 3.56s ==============================
```

Figura 54. Salida detallada de `test_fase5.py` (concurrencia, seguridad, comprobante, CSV y consultas por lote).

```
$ pytest tests/test_cobro_guardado.py -v -p no:warnings
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.6.0 -- /tmp/claude-0/-home-claude/80842ca2-cce6-5273-980d-cd9eb1cff35d/scratchpad/venv_reservas/bin/python
cachedir: .pytest_cache
rootdir: /ruta/reservas
plugins: anyio-4.15.1
collecting ... collected 9 items

tests/test_cobro_guardado.py::test_la_reserva_guarda_la_instantanea_del_cobro PASSED [ 11%]
tests/test_cobro_guardado.py::test_sin_descuento_cuando_no_llega_al_minimo PASSED [ 22%]
tests/test_cobro_guardado.py::test_reserva_sin_pre_pedido_cobra_cero PASSED [ 33%]
tests/test_cobro_guardado.py::test_cambiar_las_tasas_no_altera_reservas_anteriores PASSED [ 44%]
tests/test_cobro_guardado.py::test_desglose_guardado_coincide_con_el_calculo_en_vivo_y_suma_las_lineas PASSED [ 55%]
tests/test_cobro_guardado.py::test_cancelar_borra_el_pre_pedido_y_el_cobro_visible_es_cero PASSED [ 66%]
tests/test_cobro_guardado.py::test_el_esquema_rechaza_totales_incoherentes PASSED [ 77%]
tests/test_cobro_guardado.py::test_instantanea_usa_las_mismas_reglas_de_redondeo PASSED [ 88%]
tests/test_cobro_guardado.py::test_migracion_agrega_columnas_y_calcula_reservas_anteriores PASSED [100%]

============================== 9 passed in 0.99s ===============================
```

Figura 55. Salida detallada de `test_cobro_guardado.py` (instantánea del cobro).

8 Programa en ejecución

8.1 Instalación y ejecución

Desde la carpeta del proyecto:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m app.datos.seed
uvicorn app.main:app --reload      # http://127.0.0.1:8000
```

El comando `python -m app.datos.seed` crea la base con datos de demostración: 9 usuarios, 5 restaurantes, 25 mesas, 32 horarios, 61 reservas, 18 categorías, 62 productos y 138 ítems de pre-pedido. Las credenciales de demostración están en la Tabla 13.

Rol	Correo	Contraseña	Restaurante
Cliente	`cliente@demo.com`	`Cliente123!`	—
Administrador	`admin@demo.com`	`Admin123!`	1
Administrador	`admin2@demo.com` a `admin5@demo.com`	`Admin123!`	2 a 5

Tabla 13. Usuarios de demostración creados por el seed (datos ficticios). Fuente: `README.md` del proyecto.

8.2 Flujo del cliente

Las capturas se generaron con el script `tools/e2e_capturas.py`, que maneja un navegador real sobre la aplicación en ejecución. Muestran la versión de escritorio (1440 píxeles de ancho); el proyecto incluye también las de 375 píxeles. El script comprueba además, durante el recorrido, que no haya errores en la consola del navegador ni atributos HTML pegados, y que el cobro coincida con el cálculo esperado. El nombre que aparece en las pantallas es el de los restaurantes ficticios de los datos de demostración.

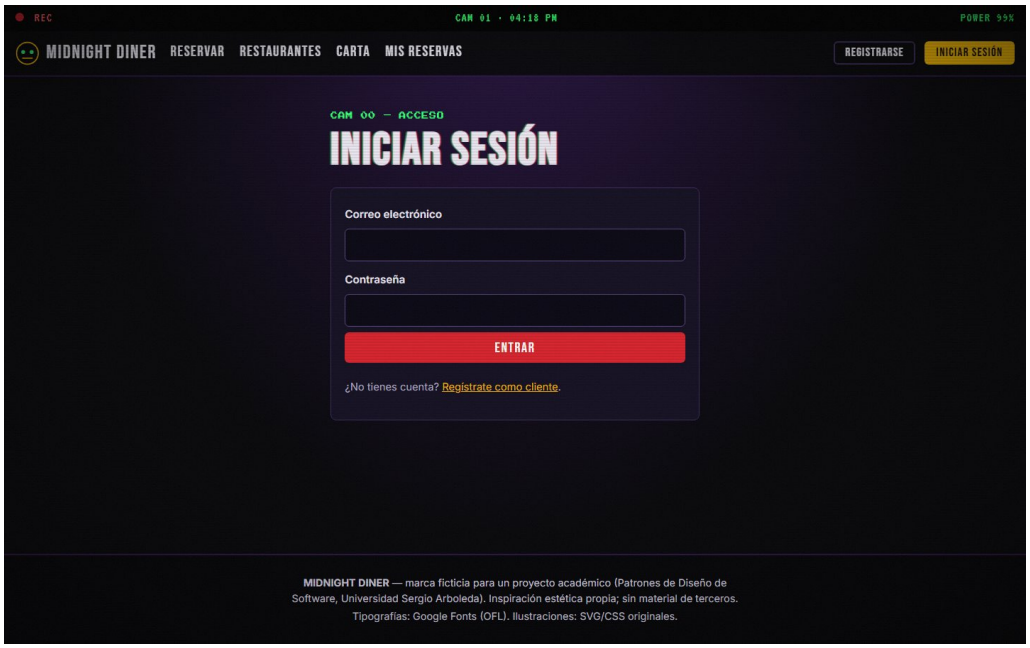


Figura 56. Pantalla de inicio de sesión (`login`).

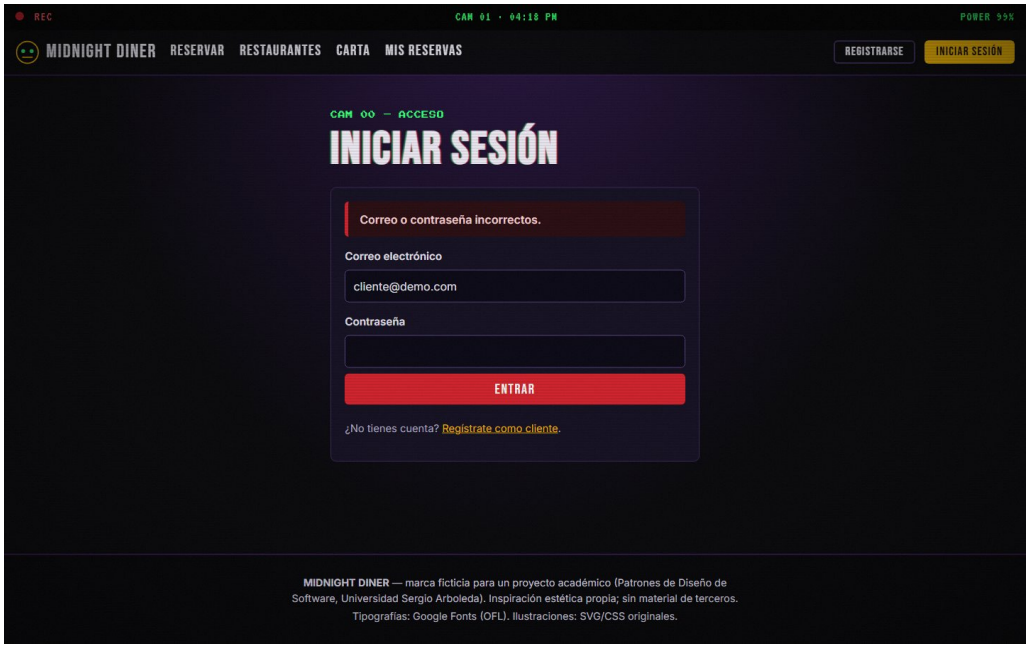


Figura 57. Inicio de sesión con credenciales incorrectas: mensaje genérico, sin revelar si falló el correo o la contraseña.

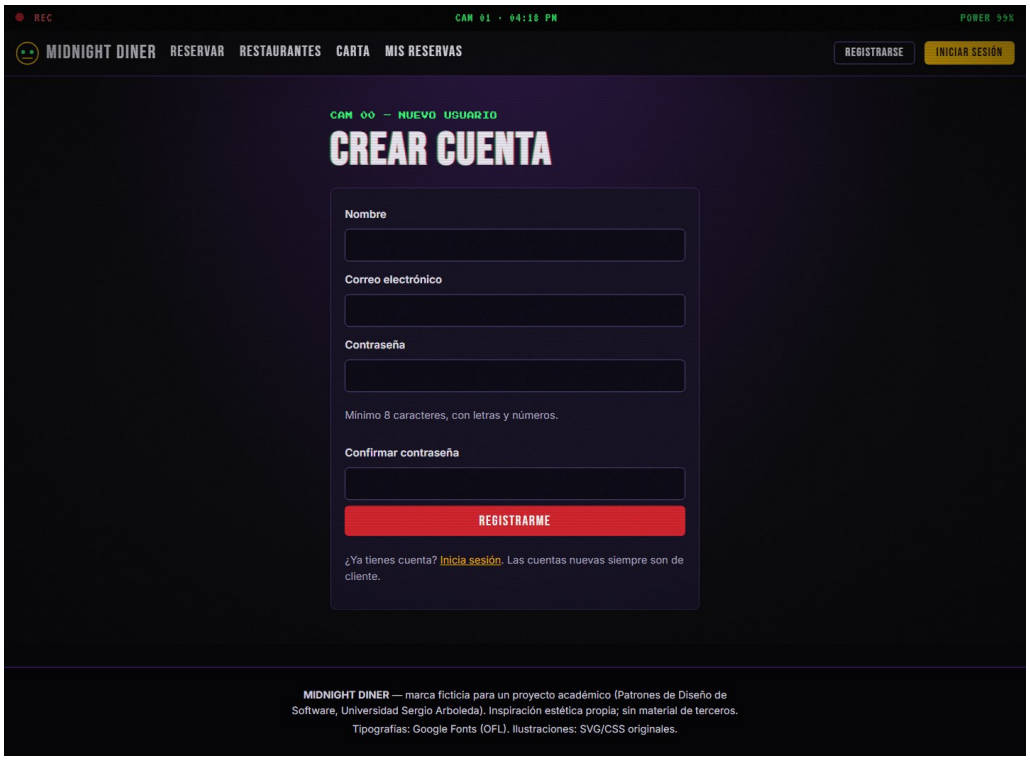


Figura 58. Registro de un cliente (`registro`).

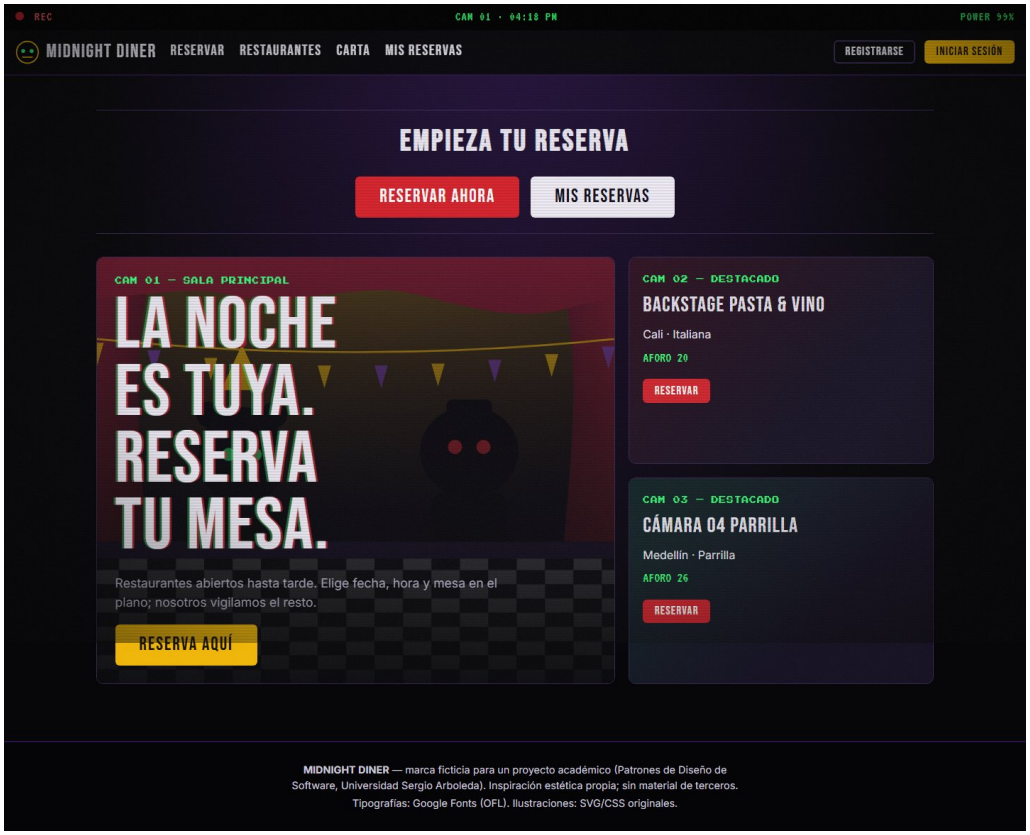


Figura 59. Página de inicio.

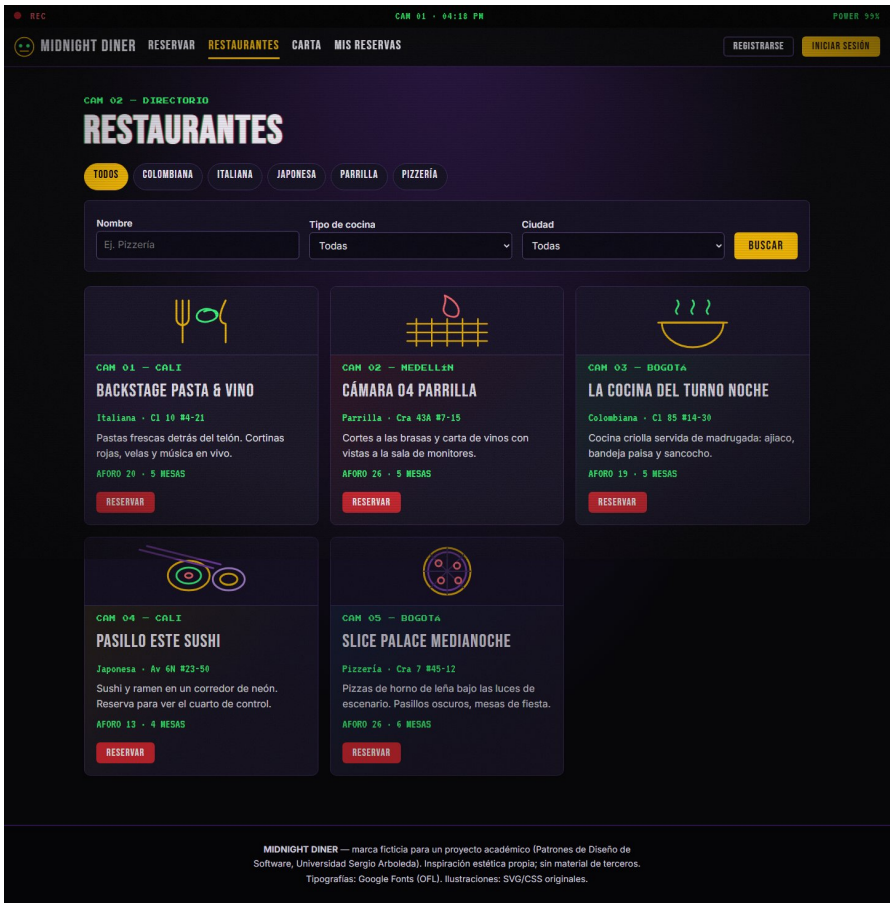


Figura 60. Listado de restaurantes con atajos por tipo de cocina y filtros (`restaurantes`).

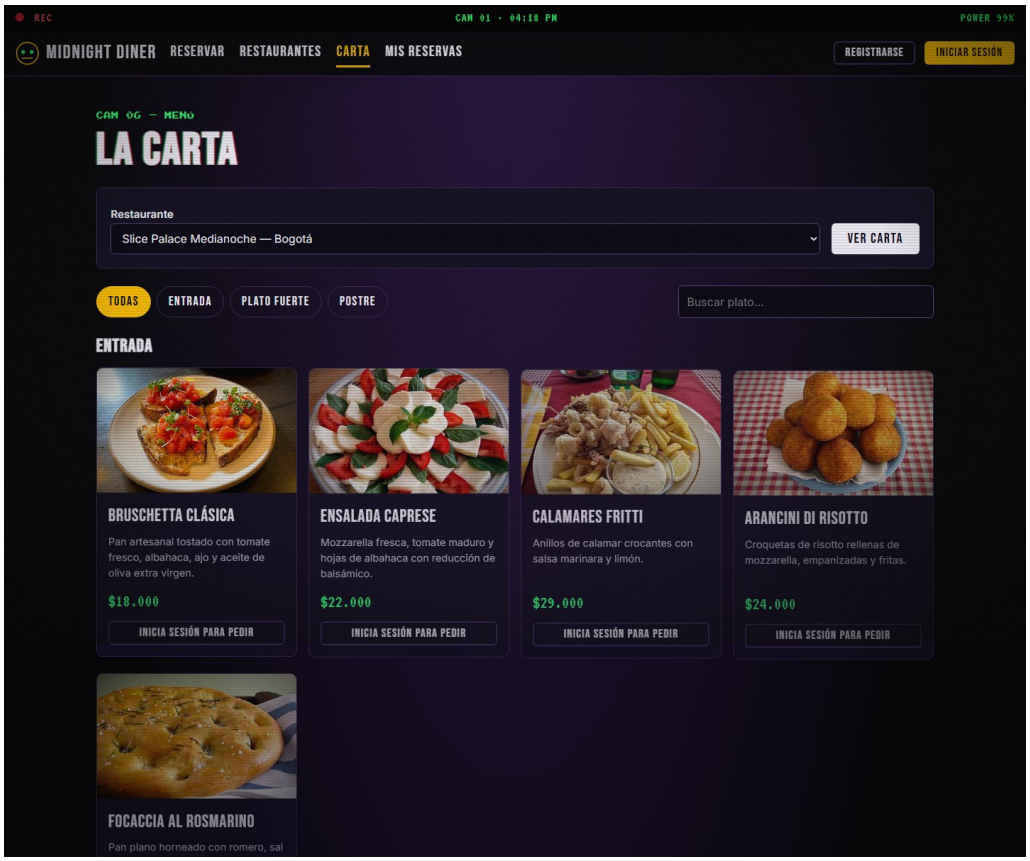


Figura 61. Carta de un restaurante, por categorías y con buscador (`carta`); se muestra el inicio de la página.

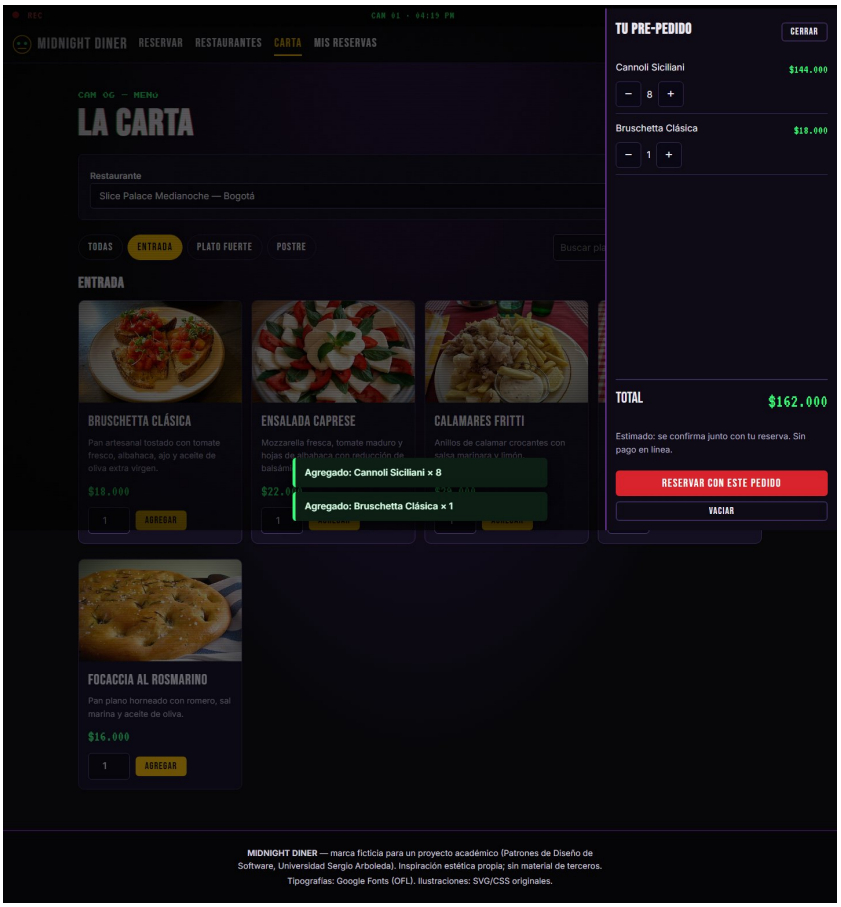


Figura 62. Carta con el cajón del pre-pedido abierto.

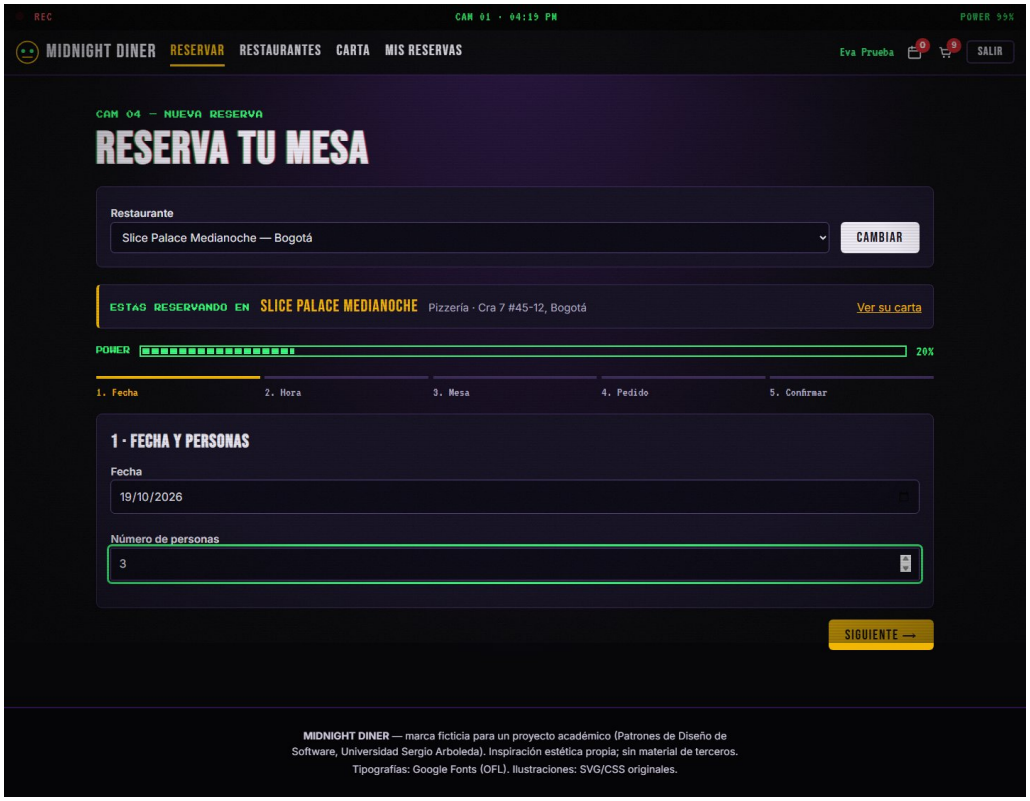


Figura 63. Formulario de reserva, paso 1: aviso del restaurante elegido, fecha y número de personas.

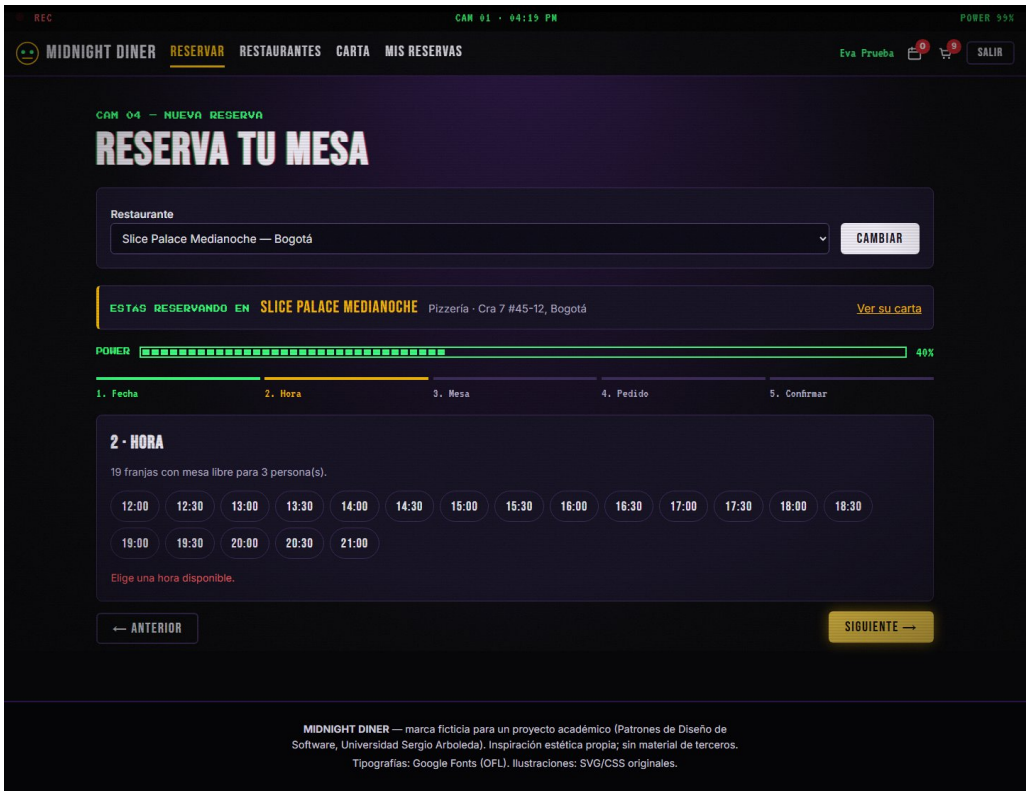


Figura 64. Paso 2 sin hora elegida: el formulario bloquea el avance y lo indica.

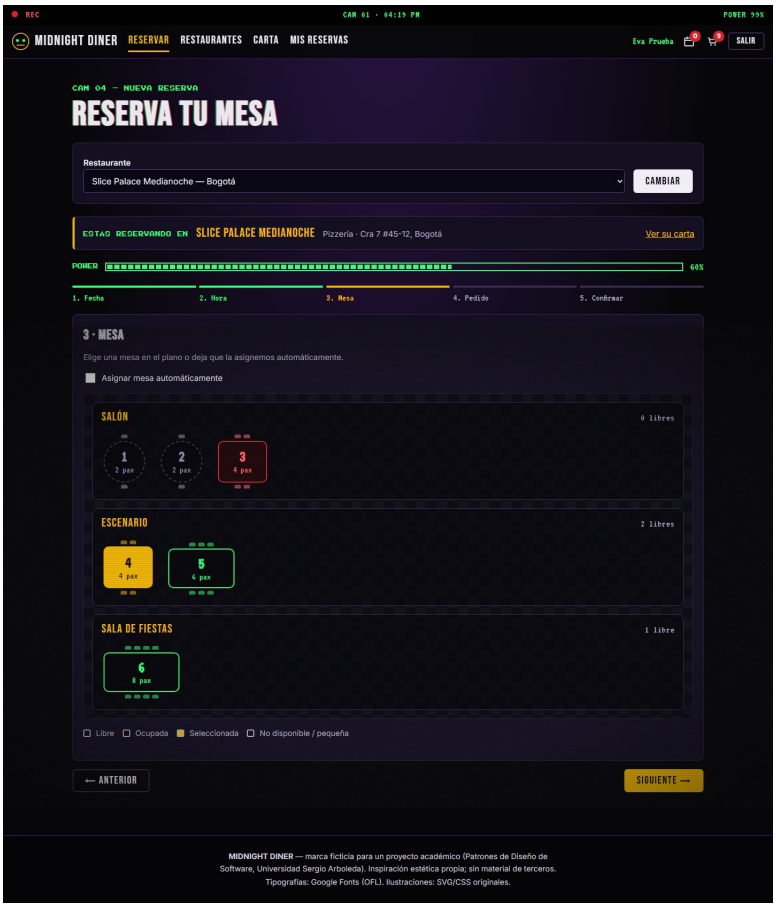


Figura 65. Paso 3: plano de mesas por zonas (libres, ocupadas, seleccionada y no disponible por capacidad).

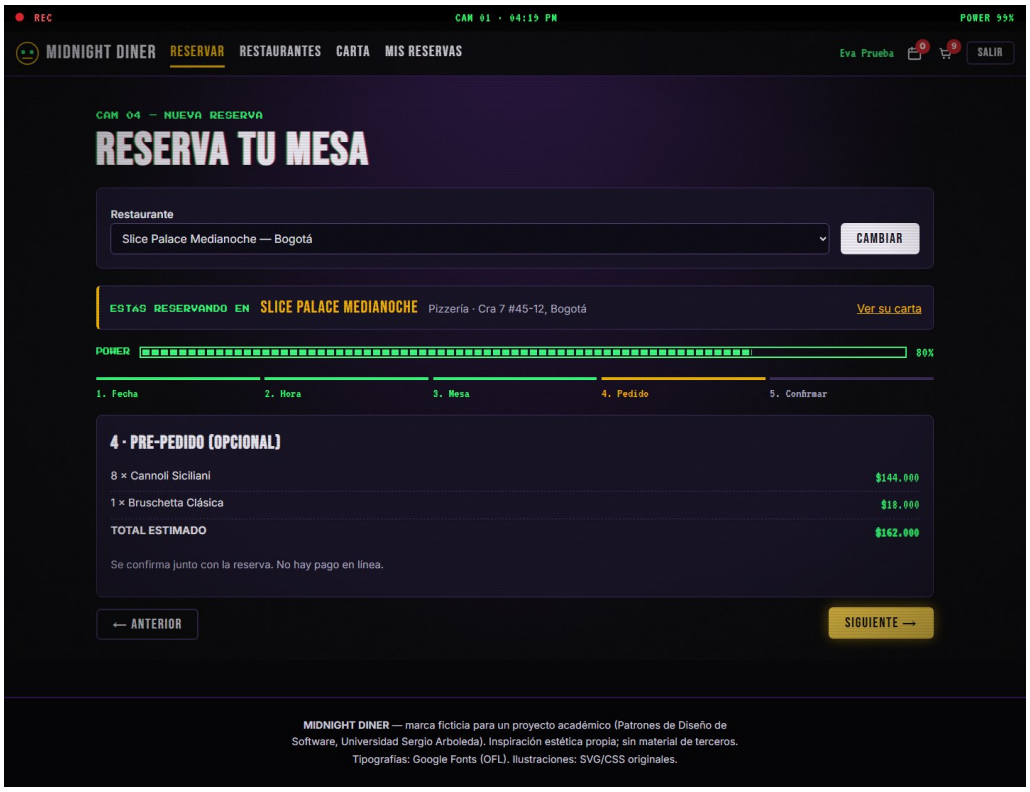


Figura 66. Paso 4: pre-pedido opcional, con el subtotal.

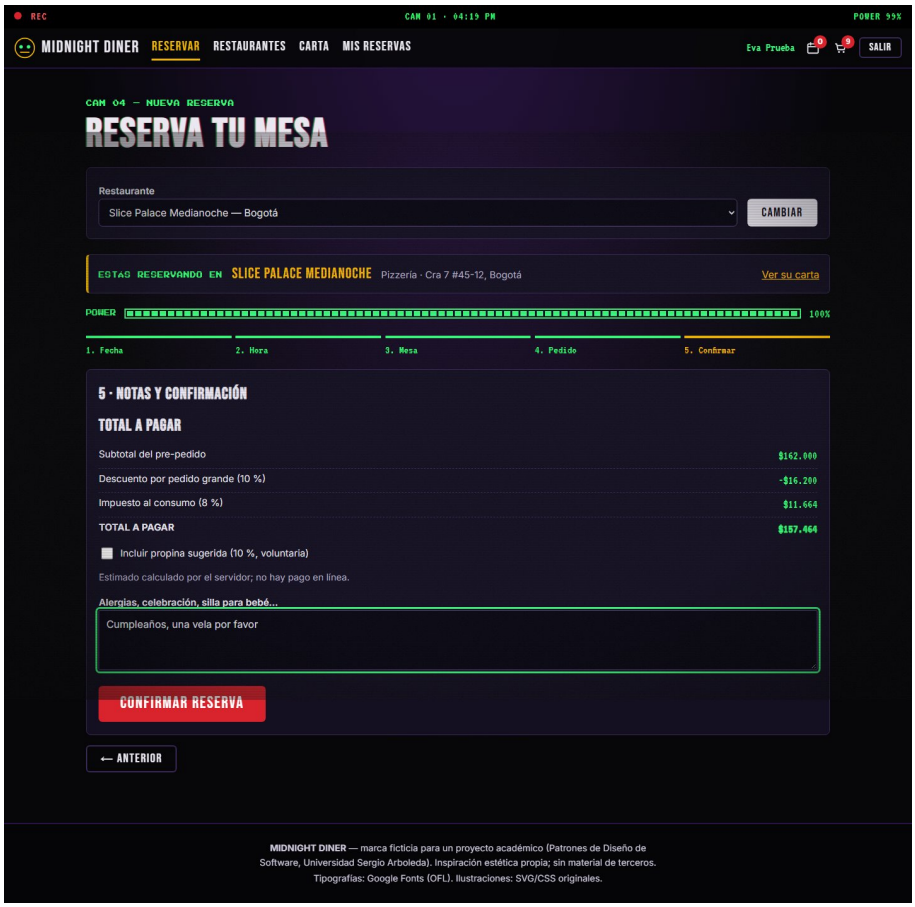


Figura 67. Paso 5: desglose del total a pagar calculado con el Decorator (sin propina).

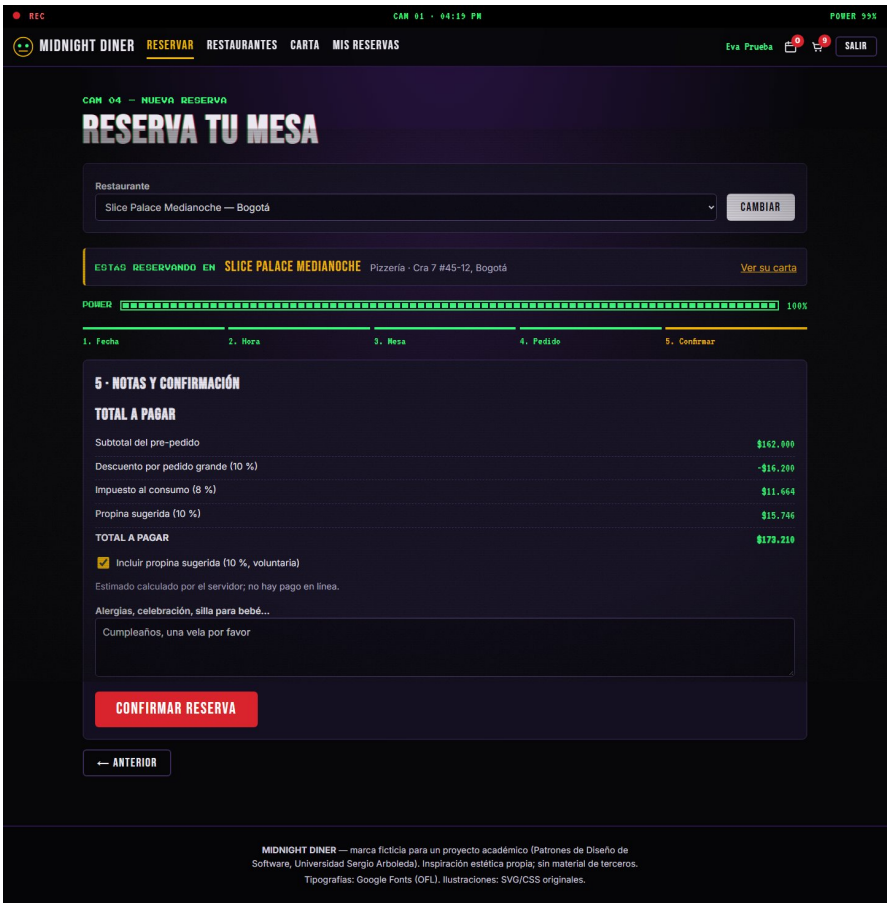


Figura 68. Paso 5 con la propina sugerida incluida (el total cambia en el servidor).

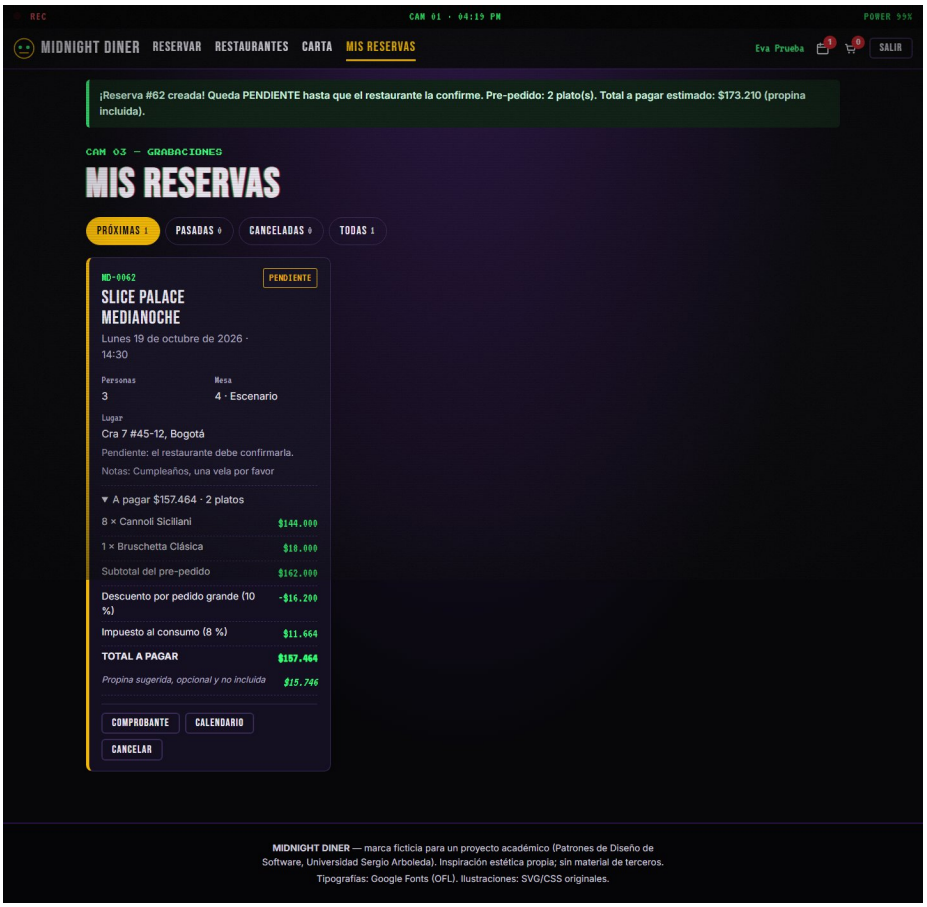


Figura 69. Mis reservas: tarjetas con código, estado, pre-pedido y total, con filtros.

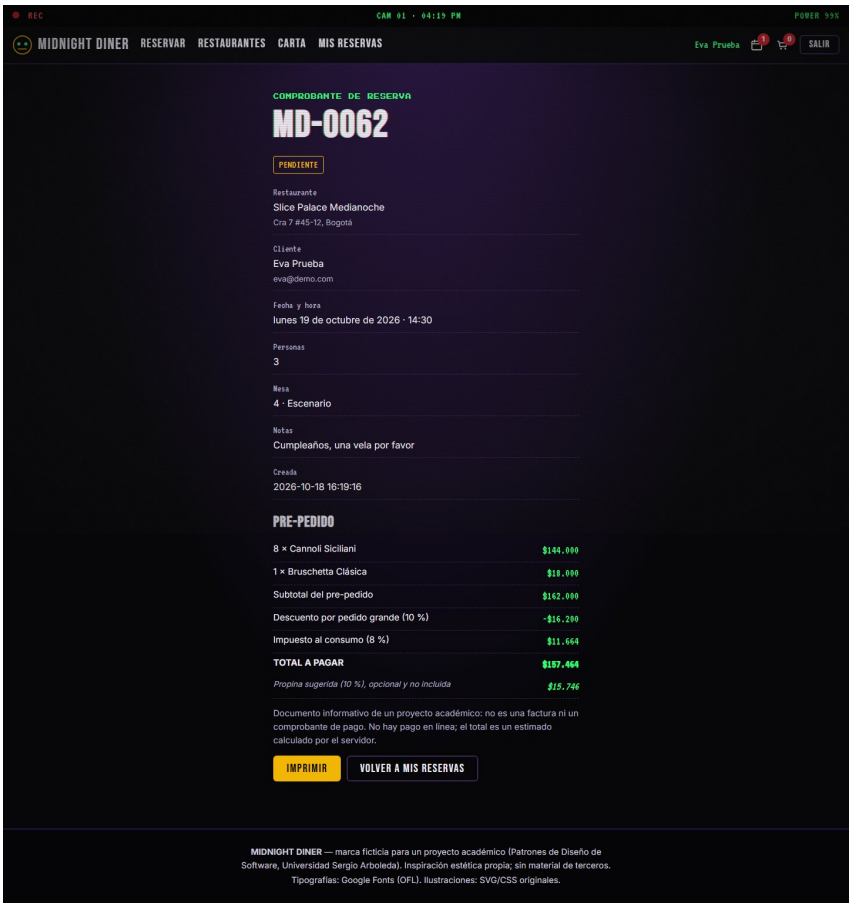


Figura 70. Comprobante imprimible de una reserva (con descarga del archivo `.ics`).

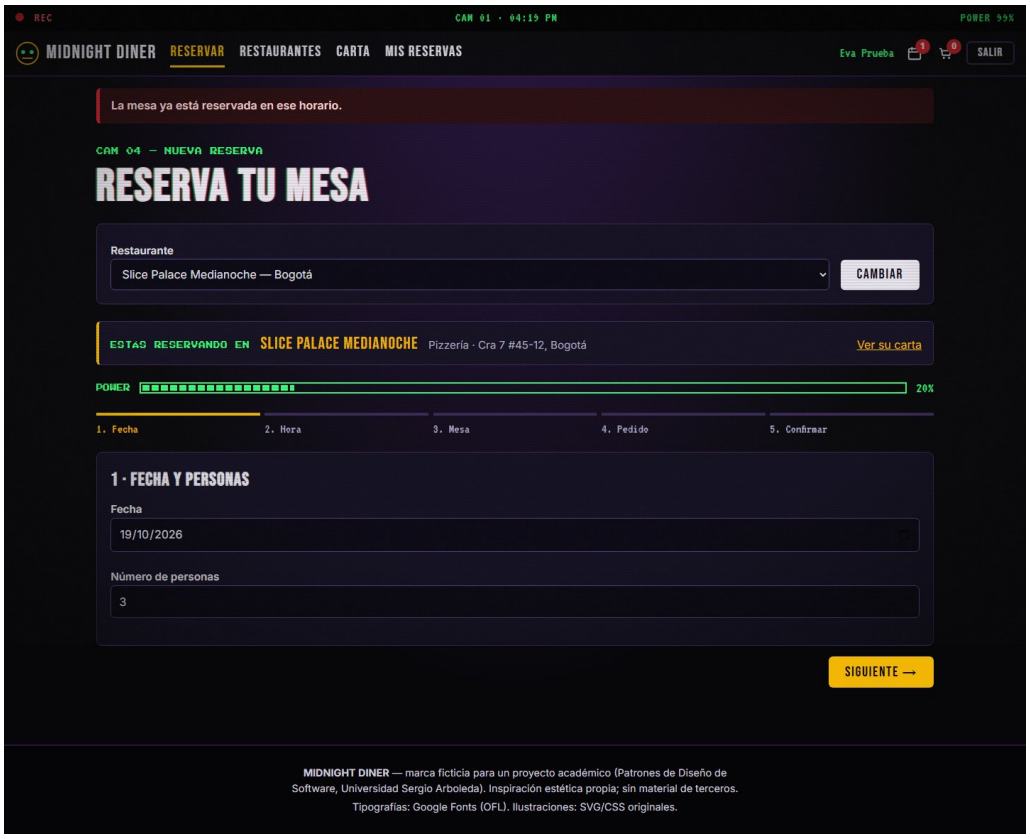


Figura 71. Intento de reservar una mesa ya ocupada en ese horario: la regla RN-3 rechaza la reserva y conserva los datos escritos.

8.3 Flujo del administrador

Cómo entra un administrador

1. Abrir `/login` (botón *Ingresar* de la barra superior).
2. Escribir el correo y la contraseña de un administrador, por ejemplo `admin@demo.com` y `Admin123!` (Tabla 13).
3. El sistema lo redirige a `/admin`, el panel de su restaurante. Un cliente, con el mismo formulario, llega a `/restaurantes`: el rol guardado en la base decide el destino (`ruta_inicio()` de cada producto de la fábrica de usuarios).

Desde el panel, el administrador gestiona las reservas (buscar, filtrar, confirmar, cancelar con confirmación, exportar a CSV), las mesas, los horarios semanales y la carta.

Cómo crear más administradores

El alta de administradores es una operación *controlada por el operador del sistema*, no por el público (Sección 2.2). Se ejecuta en el servidor con un *script* de apoyo que usa las mismas funciones de la aplicación (la misma conexión y el mismo *hash* `scrypt`) y crea el

usuario junto con su restaurante. Se guarda como `nuevo_admin.py`, se editan los datos y se ejecuta con `python nuevo_admin.py` desde la carpeta del proyecto:

```
from app.datos.conexion import ConexionBD
from app.seguridad import hashear_password

NOMBRE, EMAIL, PASSWORD = "Ana Pérez", "ana@resto.com", "Clave2026"
RESTAURANTE = dict(nombre="La Esquina", direccion="Cra 5 #10-20",
                   ciudad="Bogotá", tipo_cocina="Italiana", descripcion="")

ConexionBD().inicializar_esquema()
with ConexionBD().sesion() as con:   # una sola transacción
    uid = con.execute(
        "INSERT INTO usuario (nombre, email, password_hash, rol) "
        "VALUES (?,?,?, 'administrador')",
        (NOMBRE, EMAIL.lower(), hashear_password(PASSWORD))).lastrowid
    con.execute(
        "INSERT INTO restaurante (nombre, direccion, ciudad, tipo_cocina,"
        " descripcion, admin_id) VALUES (?,?,?,?,?,?)",
        (*RESTAURANTE.values(), uid))
```

Este *script* se probó contra la aplicación: el nuevo administrador entra por `/login`, es llevado a `/admin`, abre sus cinco secciones sin error y recibe 403 en `/reservar`. Es un *script* de apoyo para el operador; la aplicación y sus 154 pruebas automáticas no lo incluyen. Notas: se usa un correo único y una contraseña de al menos 8 caracteres con letras y números (la misma regla del registro); el restaurante nuevo empieza sin mesas ni horarios y queda abierto a reservas cuando su administrador los carga desde el panel; y en la demostración de Render (plan gratuito) los administradores son los cinco del *seed*, que se recrean en cada arranque.

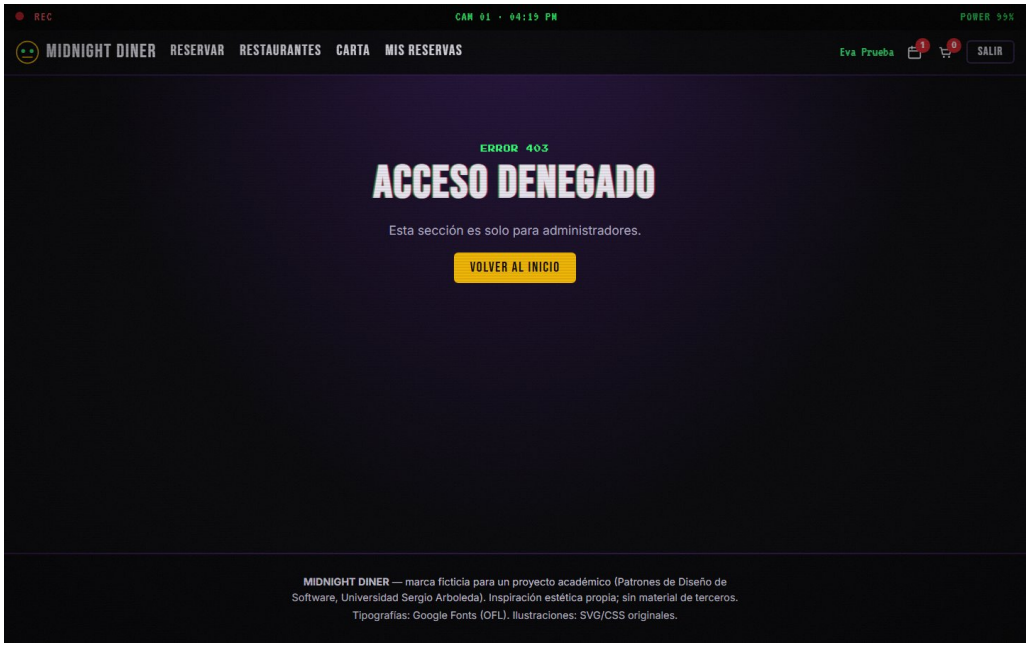


Figura 72. Un cliente que intenta entrar a `admin` recibe 403 (control de acceso por rol).

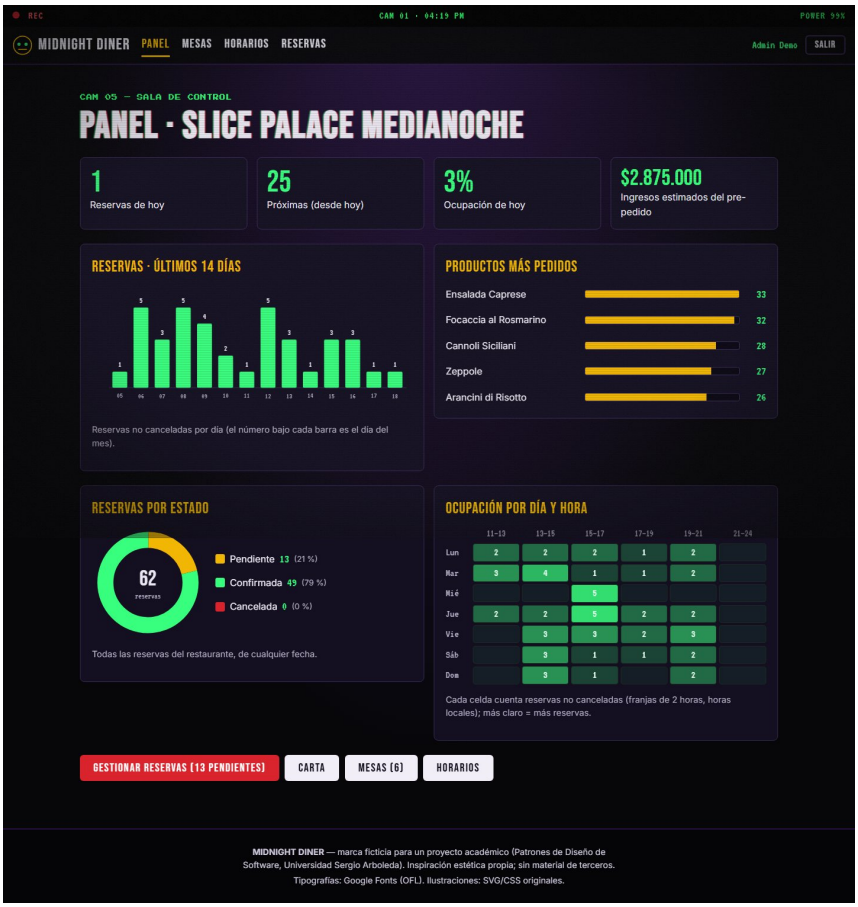


Figura 73. Panel del administrador: indicadores, dona de estados, mapa de calor de ocupación, reservas de los últimos 14 días y productos más pedidos.

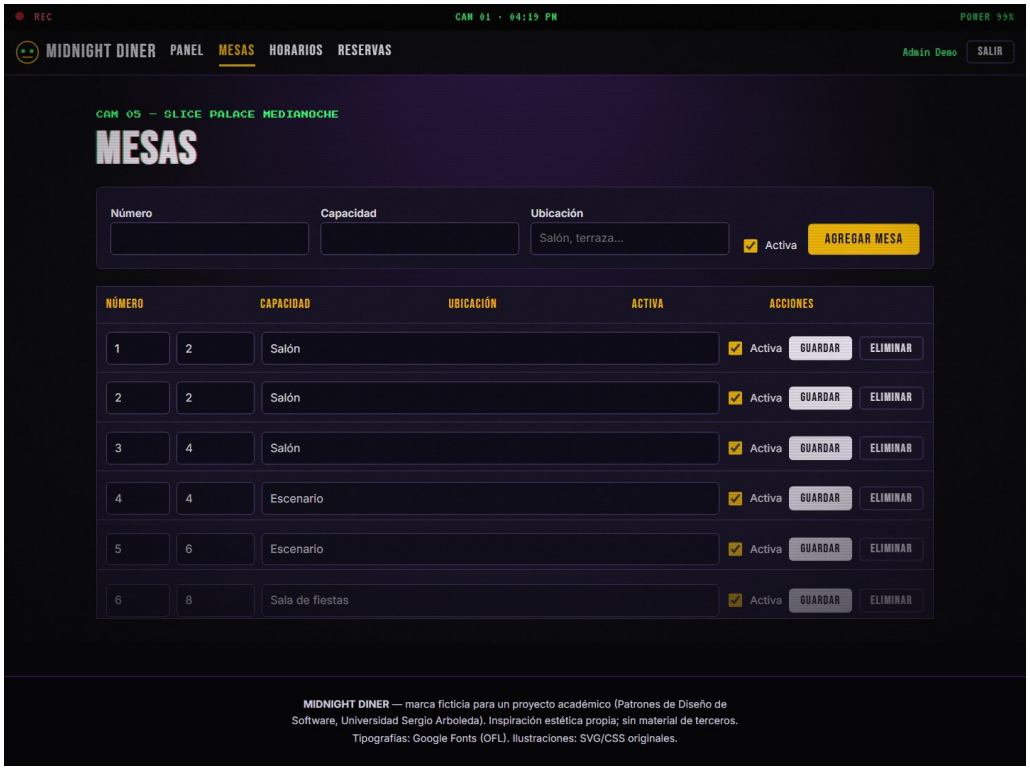


Figura 74. Gestión de mesas (`mesas`).

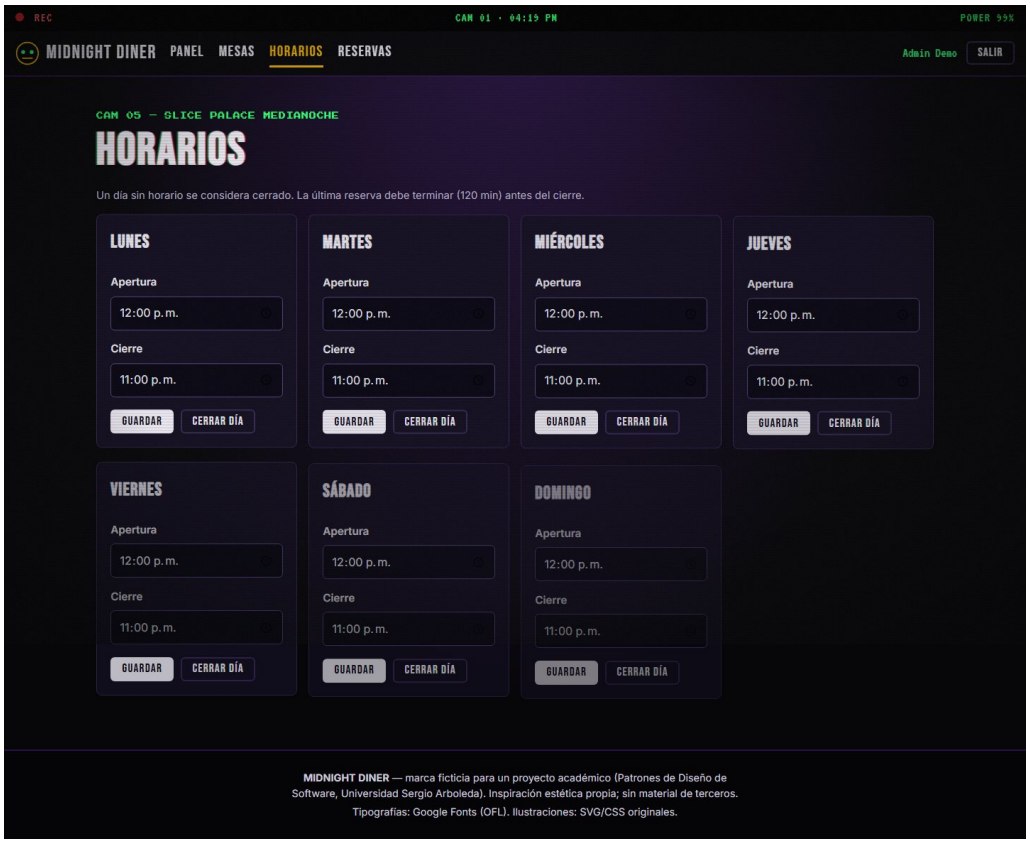


Figura 75. Gestión del horario semanal (`horarios`).

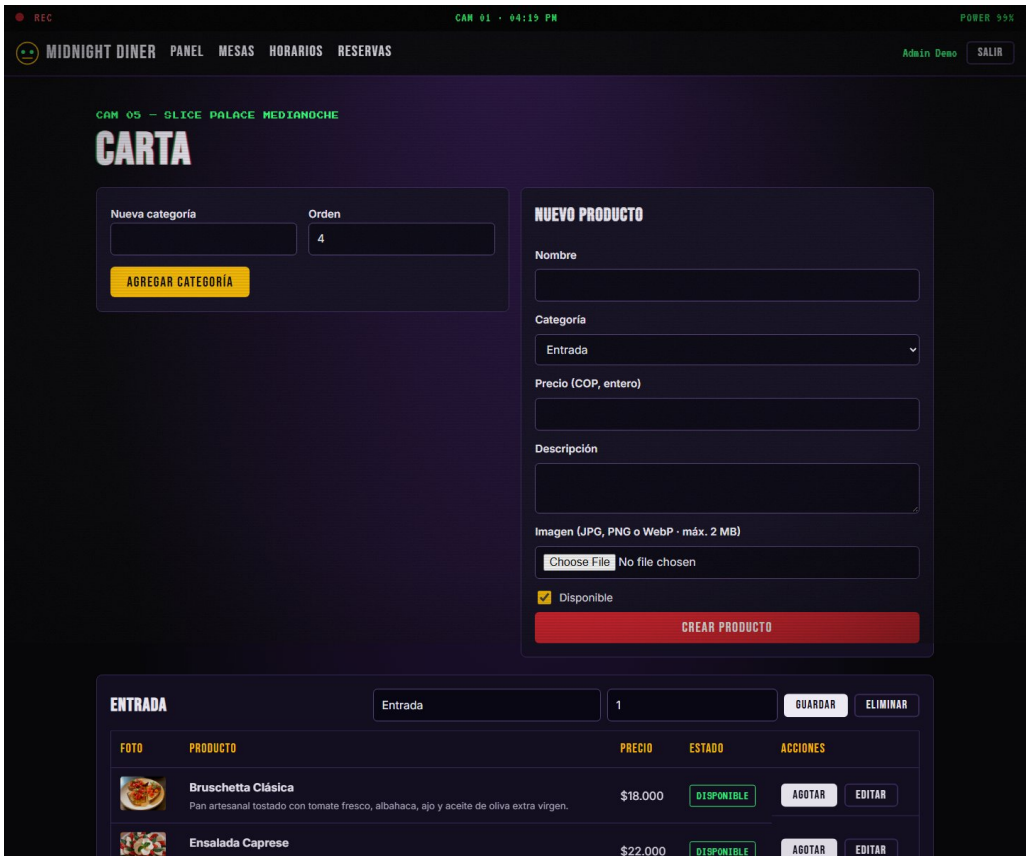


Figura 76. Gestión de la carta: categorías y productos (`carta`); se muestra el inicio de la página.

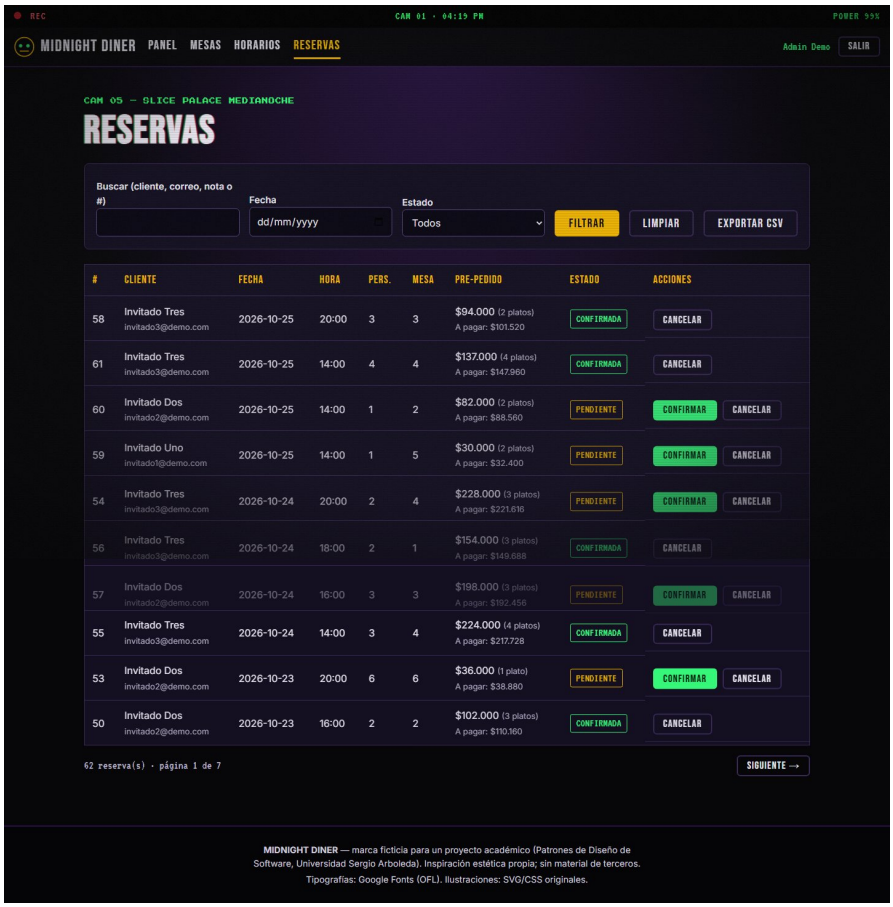


Figura 77. Reservas del restaurante, con búsqueda por cliente, filtros por fecha y estado, exportación a CSV y paginación.

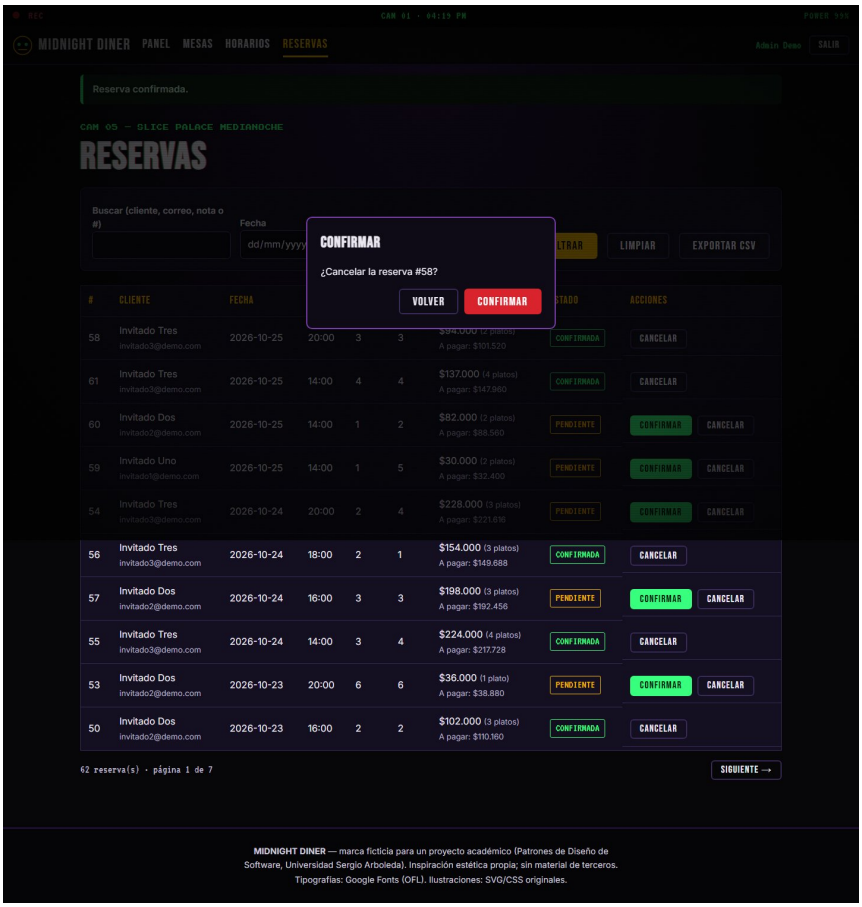


Figura 78. Confirmación antes de cancelar una reserva (cuadro de diálogo propio).

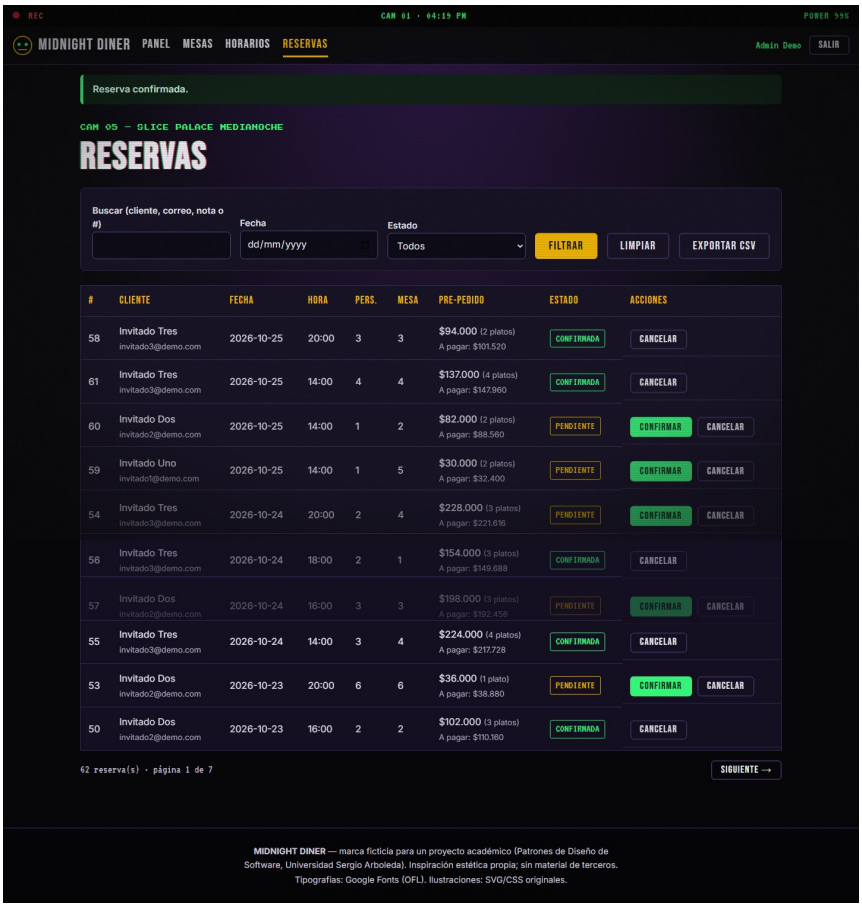


Figura 79. Reserva confirmada por el administrador (aviso verde y estado actualizado).

9 Despliegue

La aplicación se ejecuta con `uvicorn` y se configura solo con variables de entorno. El comando de arranque de producción es:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT \
        --proxy-headers --forwarded-allow-ips='*'
```

`--proxy-headers` hace que el limitador de login vea la IP real del cliente cuando hay un proxy delante. Las variables son `SECRET_KEY` (firma de la cookie), `HTTPS_ONLY=1`, `RESERVAS_DB` (ruta de la base) y `AUTOSEMBRAR=1`, que siembra los datos de demostración al arrancar si la base está vacía y nunca borra datos existentes (`test_autosembrar_siembra_bd_vacia_y_no_borra_existente`). El repositorio incluye `Procfile`, `render.yaml` (Blueprint de Render) y `requirements-prod.txt`, con solo las dependencias de ejecución.

Se eligió Render y no Vercel porque SQLite necesita un sistema de archivos con escritura persistente durante la vida del servicio, mientras que las funciones de Vercel son efímeras y de solo lectura; con Vercel habría que cambiar SQLite por una base externa. En el plan gratuito de Render el disco es efímero: cada reinicio recrea y vuelve a sembrar la

base, lo que es aceptable para una demostración pero no conserva reservas. **Estado de la verificación:** el arranque con `requirements-prod.txt`, el comando anterior (incluido el formato de PowerShell con `-forwarded-allow-ips='*'`), la cookie `Secure` y la cabecera HSTS se probaron en entornos virtuales limpios en Windows; el despliegue público en Render todavía no se ha verificado.

10 Limitaciones

Estas limitaciones son conocidas y no se ocultan:

- **Concurrencia.** La garantía contra la doble reserva se apoya en el bloqueo de escritura de SQLite y se probó con hilos y conexiones propias dentro de un proceso; no se probó con varios procesos del servidor a la vez. SQLite admite un solo escritor a la vez, adecuado para esta escala.
- **Límite de intentos de inicio de sesión.** Se guarda en memoria de cada proceso: no se comparte entre varios procesos ni sobrevive a un reinicio. Detrás de un proxy hace falta `--proxy-headers` para identificar la IP real.
- **Formulario de reserva.** Necesita JavaScript habilitado para cargar las horas disponibles.
- **Tasas del cobro.** Impuesto, propina y descuento son supuestos de demostración sin verificar contra la normativa; la propina no se guarda con la reserva. El total sí se guarda, pero en las bases anteriores a ese cambio se calculó con las tasas vigentes al migrar.
- **Alta de administradores.** La hace el operador con un *script* (el registro público solo crea clientes); en el despliegue gratuito de Render, una cuenta creada así no sobrevive al reinicio, porque la base se recrea con el *seed*.
- **Datos de demostración.** Las credenciales del seed son públicas; el despliegue de demostración no debe usarse con datos reales.
- **Alcance de las pruebas.** Las 154 pruebas son automáticas y las de este documento se ejecutaron en Linux con Python 3.13, y la instalación limpia, el arranque y la ejecución de extremo a extremo con navegador se repitieron en Windows; no se sometió la aplicación a pruebas de carga.

11 Conclusiones

- La separación en tres capas dejó las reglas de reserva en un solo lugar (el Builder y los servicios), de modo que los controladores solo traducen entre HTTP y negocio.
- Cada patrón se justificó por un problema real del código: el Singleton evita olvidar las claves foráneas; el Factory Method elimina los condicionales por rol; el Builder

concentra las validaciones; el Facade reduce el acoplamiento del controlador; el Proxy aísla la autorización; y el Decorator evita una clase por cada combinación de conceptos del cobro.

- Inyectar el Factory Method en el DAO y entregar el Proxy a la fachada mantuvo el sentido de las dependencias y permitió probar cada pieza por separado, lo que se refleja en las 154 pruebas superadas.
- Llevar la garantía contra la doble reserva a una transacción de la base de datos, guardar el total cobrado junto con la reserva y limitar los intentos de acceso hicieron que el sistema sea verificable y no dependa solo de la corrección del código de interfaz.
- Un punto a mejorar es compartir el límite de intentos y el carrito entre procesos (por ejemplo, con una base externa) para escalar a varias instancias.

Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE patterns: Best practices and design strategies* (2.ª ed.). Prentice Hall.

Wikimedia Commons. (s. f.). *Fotografías de platos usadas en las cartas de demostración* [Archivos con licencias CC0, CC BY o CC BY-SA, verificadas con la API de Commons; autores y enlaces en el Anexo A].

A Créditos

Cartas de demostración. Los nombres, descripciones y precios de las cartas de los restaurantes de cocina italiana y de pizzería parten del menú de un repositorio de GitHub de un tercero, usado con su autorización; solo se tomaron nombres, descripciones y precios (sin imágenes ni diseño). Las cartas de los demás restaurantes (colombiana, japonesa y parrilla) son texto propio del proyecto, con precios de demostración. Hay 5 cartas publicadas para 5 restaurantes.

Tipografías e ilustraciones. Tipografías de Google Fonts (VT323, Press Start 2P, Bebas Neue e Inter; licencia OFL). Las ilustraciones son SVG y CSS propios. La marca MIDNIGHT DINER y los nombres de los restaurantes son ficticios; el proyecto no usa material de terceros salvo las fotografías de los platos.

Fotografías de los platos. Hay 62 fotografías, cada una en una sola carta; se recortaron a 8:5 y se comprimieron. La Tabla 14 se genera a partir de `CREDITOS.json`, el mismo archivo que la aplicación usa para mostrar el crédito junto a cada foto; los enlaces completos a cada página de origen están en `CREDITOS.json` y `CREDITOS.md`. Las licencias con atribución (CC BY) obligan a reconocer al autor, y las CC BY-SA obligan además a compartir las adaptaciones con la misma licencia.

Restaurante	Plato	Archivo de origen	Autor	Licencia	Fuente
Backstage Pasta & Vino	Antipasto Misto	`Antipasto_Misto_Con_Bruschette.jpg`	AlMare (usuario de Commons)	CC BY 2.0	Commons
Backstage Pasta & Vino	Burrata con Tomates Cherry	`Burrata_salad_with_cherry_tomatoes_and_rocket_from_La_Mensa_Grill_and_Bar_Plaza88.jpg`	Peachyeung316	CC BY-SA 4.0	Commons
Backstage Pasta & Vino	Carpaccio di Manzo	`Beef_Carpaccio_(3466985953).jpg`	Mack Male from Edmonton, AB, Canada	CC BY-SA 2.0	Commons
Backstage Pasta & Vino	Fettuccine Alfredo	`Fettuccine_Alfredo.jpg`	Robert Banh	CC BY 2.0	Commons
Backstage Pasta & Vino	Gnocchi al Pesto	`Gnocchi_with_pesto_(3269919059).jpg`	Karen and Brad Emerson	CC BY 2.0	Commons
Backstage Pasta & Vino	Lasagna alla Bolognese	`Meaty_Lasagna_8of8_(8736299782).jpg`	Breville USA	CC BY 2.0	Commons
Backstage Pasta & Vino	Panna Cotta	`Panna_Cotta_with_cream_and_garnish.jpg`	hiro	CC BY-SA 2.0	Commons
Backstage Pasta & Vino	Risotto ai Funghi	`Risotto_Champignons.jpg`	PixelBrûlé	CC BY-SA 4.0	Commons
Backstage Pasta & Vino	Semifreddo al Limone	`Semifreddo_nocciola.jpg`	Daniele2399	CC BY-SA 4.0	Commons
Backstage Pasta & Vino	Spaghetti alla Carbonara	`Espaguetis_carbonara.jpg`	Javier Somoza	CC BY-SA 4.0	Commons
Backstage Pasta & Vino	Tiramisù	`Tiramisu_-_Bistro_Don_Giovanni_-_Sarah_Stierch_2018_02.jpg`	Missvain	CC BY 4.0	Commons
Backstage Pasta & Vino	Torta Caprese	`Tortacapresespot.jpeg`	Mgdongarra	CC0	Commons
Backstage Pasta & Vino	Torta della Nonna	`Torta_della_nonna.jpg`	fugzu from Pistoia, Italia	CC BY 2.0	Commons
Backstage Pasta & Vino	Zuppa Minestrone	`Minestrone_soup.jpg`	Katrin Morenz from Aachen, Deutschland	CC BY-SA 2.0	Commons
Cámara 04 Parrilla	Brownie con Helado	`Walnut_brownie_with_ice-cream.jpg`	Sirsendu.mohanta	CC BY-SA 4.0	Commons

continúa en la página siguiente

Restaurante	Plato	Archivo de origen	Autor	Licencia	Fuente
Cámara 04 Parrilla	Chorizo Criollo	Chorizo_a_la_parrilla_de_Argentina.jpg	Horacio Cambeiro	CC BY 3.0	Commons
Cámara 04 Parrilla	Churrasco de Res	DFC_2244_Juicy_grilled_steak_smothered_in_savory_brown_gravy_served_with_roasted_potatoes_sauteed_vegetables_and_a_crisp_mixed_salad.jpg	PattayaPatrol	CC BY-SA 4.0	Commons
Cámara 04 Parrilla	Copa de Vino Tinto	A_glass_of_red_wine.jpg	Davide Restivo from Lenzburg, Aarau, Switzerland	CC BY-SA 2.0	Commons
Cámara 04 Parrilla	Costillas BBQ	BBQ_RIBS_pork_side_ribs.jpg	WorldwrestlingfederationVKM	CC BY-SA 3.0	Commons
Cámara 04 Parrilla	Flan de Caramelo	Creme_caramel_at_NB_Steak_JK.jpg	Oleg Yunakov	CC BY-SA 4.0	Commons
Cámara 04 Parrilla	Limonada de Hierbabuena	Homemade_Mint_Lemonade,Bangladesh.jpg	Evening42	CC BY-SA 4.0	Commons
Cámara 04 Parrilla	Mazorca Asada	Grilled_Corn_on_the_cob.JPG	Namitakhaire	CC BY-SA 4.0	Commons
Cámara 04 Parrilla	Picaña a la Brasa	Picanha_(14759358544).jpg	Christian Benseler	CC BY 2.0	Commons
Cámara 04 Parrilla	Pollo a las Brasas	Pollo_a_la_brasa_peruano.jpg	Manuel González Olaechea	CC BY-SA 3.0	Commons
Cámara 04 Parrilla	Provoleta a la Parrilla	Provoleta_con_cazuela_(porcion).jpg	Horacio Cambeiro	CC BY 3.0	Commons
Cámara 04 Parrilla	Salmón a la Parrilla	Grilled_plated_salmon_fillet.jpg	DanaTentis	CC0	Commons
La Cocina del Turno Noche	Agua de Panela con Queso y Pan	Aguapanela-Colombiana-3-2.jpg	Mappa Stellae	CC BY-SA 4.0	Commons
La Cocina del Turno Noche	Ajiaco Santafereño	Ajiaco_la_puerta_falsa.jpg	Cesar.cely	CC BY-SA 4.0	Commons
La Cocina del Turno Noche	Arepa con Queso	Arepa_de_chocolo.jpg	Nelson Fernando Sotelo Castro	CC BY 2.0	Commons
La Cocina del Turno Noche	Bandeja Paisa	Bandeja_paisa,_plato_Colombiano.jpg	Wilfredor	CC0	Commons
La Cocina del Turno Noche	Brevas en Almíbar	Incir_Kompostosu_(Fig_in_Syrup).JPG	Miansari66	CC0	Commons
La Cocina del Turno Noche	Café Colombiano	A_small_cup_of_coffee.JPG	Julius Schorzman	CC BY-SA 2.0	Commons
La Cocina del Turno Noche	Chocolate con Queso y Arepa	Colombian_classic_-_hot_chocolate_with_arepa_and_cheese_(in_porcelain_with_pink_flowers).jpg	Quinn Comendant	CC BY-SA 2.0	Commons
La Cocina del Turno Noche	Empanadas Colombianas	Empanada_colombiana.jpg	Francesc Fort	CC BY-SA 4.0	Commons
La Cocina del Turno Noche	Limonada de Coco	Coconut_Drink_(Unsplash).jpg	Alexander Mils alexandermils	CC0	Commons
La Cocina del Turno Noche	Patacones	Patacones_-_Barranquilla.jpg	Jdvillalobos	CC BY 3.0	Commons
La Cocina del Turno Noche	Sancocho de Gallina	Sancocho_cruzado_de_gallina,_rabo_y_costilla_con_arepa.jpg	Rodolfo pimentel	CC BY-SA 4.0	Commons
Pasillo Este Sushi	California Roll	California_Sushi_mit_Kaviar_(26545022496).jpg	Tim Reckmann from Hamm, Deutschland	CC BY 2.0	Commons
Pasillo Este Sushi	Cheesecake de Matcha	Olea_Matcha_Cheesecake_with_olive_oil_-_Olea_2025-07-03.jpg	Andy Li	CC0	Commons
Pasillo Este Sushi	Dorayaki	Dorayaki_004.jpg	Ocdp	CC0	Commons
Pasillo Este Sushi	Edamame	Edamame_by_jark.jpg	jark	CC BY-SA 2.0	Commons

continúa en la página siguiente

Restaurante	Plato	Archivo de origen	Autor	Licencia	Fuente
Pasillo Este Sushi	Gyozas de Cerdo	Pork_gyoza_-_Massachusetts.jpg	Daderot	CC0	Commons
Pasillo Este Sushi	Mochi Helado	Mochi_Ice_Cream.jpg	Charles Nguyen	Public domain	Commons
Pasillo Este Sushi	Nigiri de Salmón	Nigiri_Sushi_(25966163204).jpg	Tim Reckmann from Hamm, Deutschland	CC BY 2.0	Commons
Pasillo Este Sushi	Ramen Tonkotsu	Tonkotsu_Ramen_-_Goemon_Ramen_Bar_2023-06-06.jpg	Andy Li	CC0	Commons
Pasillo Este Sushi	Sashimi de Salmón	Salmon_sashimi_Yuichiro_Haga.jpeg	Yuichiro Haga	CC BY 2.0	Commons
Pasillo Este Sushi	Sopa Miso	Miso_Soup_001.jpg	Ocdp	CC0	Commons
Pasillo Este Sushi	Tempura de Camarón	Tempura_Prawns_at_Southport_Surf_Life_Saving_Club,_Main_Beach,_Queensland.jpg	Kgbo	CC BY-SA 4.0	Commons
Pasillo Este Sushi	Té Verde Japonés	2018_Uji_lightly_steamed_sencha_-_first_infusion.jpg	Difference engine	CC BY-SA 4.0	Commons
Pasillo Este Sushi	Yakisoba de Pollo	Chicken_yakisoba_Merelymel13.jpg	Merelymel13	CC BY-SA 2.0	Commons
Slice Palace Medianoche	Affogato al Caffè	Vinoteca,_Smithfield,_London_(4485849609).jpg	Ewan Munro from London, UK	CC BY-SA 2.0	Commons
Slice Palace Medianoche	Arancini di Risotto	Arancini_002.jpg	No machine-readable author provided. Gmelfi assumed (based on copyright claims).	Public domain	Commons
Slice Palace Medianoche	Bruschetta Clásica	Bruschetta_at_The_Bar_at_MacArthur_Place_-_Sarah_Stierch.jpg	Missvain	CC BY 4.0	Commons
Slice Palace Medianoche	Calamares Fritti	Calamari_fritti_in_Izola_Isola_(52976884396).jpg	domjisch from Graz, Austria	CC BY 2.0	Commons
Slice Palace Medianoche	Cannoli Siciliani	Tiny_sicilian_cannoli_at_lunch_(5071817409).jpg	judywitts	CC BY 2.0	Commons
Slice Palace Medianoche	Ensalada Caprese	Caprese_salad_in_homemade.jpg	spiralmushroom	CC BY 2.0	Commons
Slice Palace Medianoche	Focaccia al Rosmarino	Rosemary_Focaccia.jpg	Parkerman & Christie from San Diego, USA	CC BY 2.0	Commons
Slice Palace Medianoche	Pizza Margherita	Margherita_Originale.JPG	Mario56	CC BY-SA 3.0	Commons
Slice Palace Medianoche	Pizza Pepperoni	Pepperoni_Pizza_from_Fellini's_Pizza.jpg	BullDawg2021	CC BY 4.0	Commons
Slice Palace Medianoche	Pizza Quattro Formaggi	Four_Cheese_-_Pizza_500_2023-11-10.jpg	Andy Li	CC0	Commons
Slice Palace Medianoche	Pollo alla Parmigiana	Tony's_of_North_Beach_-_December_2022_-_Sarah_Stierch_05.jpg	Missvain	CC BY 4.0	Commons
Slice Palace Medianoche	Zeppole	Zeppole_stacked_on_a_plate.jpg	Luca Sartoni	CC BY-SA 2.0	Commons

Tabla 14. Créditos de las fotografías de las cartas. Fuente: `CREDITOS.json` del proyecto.